

第 7 章：String Fundamentals (字符串基础)

原书：《Learning Python》（6th Edition），作者 Mark Lutz 本章地位：本章是对字符串对象类型（str）的深入二次巡礼。在第 4 章已初见字符串的基础上，本章补上所有遗漏细节：字面量的各种写法（单引号、双引号、三引号、转义、原始字符串）、序列运算（拼接、索引、切片）、43+ 个字符串方法，以及三种字符串格式化工具（% 表达式、.format 方法、f-string）。本章也是全书对象类型深入之旅的第一站——它定义的"序列（sequence）"与"不可变（immutable）"两个类别概念，将直接适用于后面列表、元组、字典等所有核心类型。Unicode 的完整细节本书推迟到第 37 章，本章只需建立"字符串是 Unicode 码点序列、而非字节数组"的心智模型。

7.1 开篇引言

英文原文

So far, we've studied numbers and explored Python's dynamic typing model. The next major type on our in-depth object tour is the Python string—an ordered collection of characters used to store and represent text- and bytes-based information. We looked briefly a

t strings in Chapter 4. Here, we will revisit them to fill in details we skipped earlier. Before we get started, let's get clear on what we won't be covering here. Chapter 4 also briefly previewed Unicode strings and file s—tools for dealing with non-ASCII text. Unicode is a key tool for programmers, especially those who work in the internet domain. It can pop up, for example, in web pages, emails, GUI toolkits, file-processing tools, XML and JSON text, and more. At the same time, Unicode can be a heavy topic for programmers just starting out, and a complete understanding of it relies on tools that we haven't yet studied in full, like files. In light of that, this book splits its strings coverage between the essentials here, and their extension to Unicode and byte strings in Chapter 37 of its advanced topics part. That is, this chapter tells only part of the string story in Python—the part that most scripts use, and most Python learners need to know up front. Despite this limited scope, everything we learn here will apply directly to Unicode and bytes processing, too, because Python text strings are Unicode, even if they're simple ASCII text, and byte strings are simply strings constrained to byte values. After you've learned the basics here, Chapter 37 is recommended reading for the rest of the string saga, and most programmers will want to follow up with its coverage eventually. Unicode is rarely optional in programming today, though it's best deferred until you've had a chance to master

strings in general. So let's get started!

中文翻译

到目前为止，我们已经学习了数字（number）并探索了 Python 的动态类型模型。在深入对象之旅中，下一种主要类型是 Python 的字符串（string）——一种有序的字符集合，用来存储和表示基于文本（text）和基于字节（bytes）的信息。我们在第 4 章已经粗略看过字符串，本章将重新审视它，补充之前跳过的细节。在开始之前，先明确本章不会讲什么。第 4 章还简要预告了 Unicode 字符串和文件——处理非 ASCII 文本的工具。Unicode 对程序员（尤其是互联网领域的程序员）来说是不可或缺的工具。它可能出现在网页、电子邮件、GUI 工具包、文件处理工具、XML 和 JSON 文本等各处。但同时，Unicode 对刚起步的程序员是个沉重的话题，要完全理解它，依赖我们尚未完整学习的工具，比如文件。有鉴于此，本书把字符串的内容一分为二：基础部分放在本章，Unicode 和字节字符串的扩展放在高级主题部分的第 37 章。也就是说，本章只讲述 Python 字符串故事的一部分——大多数脚本都在用、大多数 Python 学习者需要先知道的那部分。尽管范围有限，我们在这里学到的一切也会直接适用于 Unicode 和字节处理，因为 Python 的文本字符串就是 Unicode——即使它只是简单的 ASCII 文本；而字节字符串不过是限制为字节值的字符串而已。学完本章基础之后，建议继续阅读第 37 章以了解字符串故事的其余部分，大多数程序员最终都会想读完它。在今天，Unicode 在编程中几乎不是可选项，但它最好等你先掌握了字符串的一般用法之后再深入学习。那么，让我们开始吧！

深度理解

·核心概念：Python 的 str 从定义上讲就是 Unicode 字符串——这不是“高级特性”，而是默认行为。你写的每个'a bc' 都是 Unicode 字符串，所以本章学的所有操作（切片、方法、格式化）天然适用于任何语言的文本。

·设计原因：为什么把 Unicode 推迟到第 37 章？因为完整理解 Unicode 依赖编码（encoding）/解码（decoding）、字节流、文件 I/O 等尚未学到的工具。作者选择“先学通用的字符串操作，再学多字节文本的特殊细节”，符合学习曲线。

·实际问题：今天的网络应用（网页、JSON、数据库）几乎全是 UTF-8 编码的文本。若不懂字符串基础，后面处理任何用户输入都会出问题；但一上来就啃 Unicode 细节，新手很容易被概念淹没。

·初学者误区：以为“Unicode 是另一套字符串系统”。恰恰相反——所有文本字符串都是 Unicode，\uXXXX 之类的转义只是显式写出码点（code point）的语法糖。本章只需记住一条：字符串 ≠ 字节，len() 数的是字符，不是字节。

7.2 字符串对象基础 (String Object Basics)

英文原文

From a functional perspective, strings can be used to represent just about anything that can be encoded as

text or bytes. In the text department, this includes symbols and words (e.g., your name), contents of text files loaded into memory, internet addresses, Python source code, and so on. Strings can also be used to hold the raw bytes used for media files and network transfers, and both the encoded and decoded forms of non-ASCII Unicode text. You may have used strings in other languages, too. Python's strings serve the same role as character arrays in languages such as C, but they are a somewhat higher-level tool than arrays. Unlike in C, strings in Python come with a powerful set of precoded processing tools. Also unlike languages such as C, Python has no distinct type for individual characters; instead, you just use one-character strings for one-character info. Strictly speaking, Python strings are categorized as immutable sequences, meaning that the characters they contain have a left-to-right positional order and cannot be changed in place. In fact, strings are the first representative of the larger class of objects called sequences that we will explore here. Pay attention to the sequence operations covered in this chapter, because they will work the same on other sequence types you'll meet later, such as lists and tuples. As a first step, Table 7-1 previews common string literals and operations discussed in this chapter, by abstract example (don't expect its code snippets to run!). As it shows, empty strings are written as a pair of quotation marks (single or double) with n

othing in between, and there are a variety of ways to code strings. For processing, strings support expression operations such as concatenation (combining strings), slicing (extracting sections), indexing (fetching by offset), and so on. Besides expressions, Python also provides a set of string methods that implement common string-specific tasks, as well as modules for more advanced text-processing tasks such as pattern matching.

中文翻译

从功能角度看，字符串几乎可以表示任何能被编码为文本或字节的东西。在文本领域，这包括符号和单词（比如你的名字）、加载到内存中的文本文件内容、网址、Python 源码等等。字符串还可以用来存放媒体文件和网络传输所用的原始字节，以及非 ASCII Unicode 文本的编码和解码两种形式。你可能也在其他语言里用过字符串。Python 的字符串与 C 等语言中的字符数组（character array）扮演相同角色，但它是比数组更高级的工具。与 C 不同，Python 的字符串自带一套强大的预编码处理工具；同时，Python 也没有独立的“单字符”类型——单字符信息就用一个字符的字符串来表示。严格来说，Python 字符串被归类为不可变序列（immutable sequence）：它们包含的字符具有从左到右的位置顺序，且不能就地（in place）修改。实际上，字符串是我们这里将要探索的更大对象类别——序列（sequence）——的第一个代表。请重点关注本章讲解的序列操作，因为它们对后面会遇到的列表（list）、元组（tuple）等其他序列类型同样适用。作为第一步，表 7-1 以抽象示例

的方式预览了本章要讨论的常见字符串字面量和操作（不要指望这些代码片段真的能运行！）。如表所示，空字符串写作一对引号（单引号或双引号），中间什么都不放；字符串有多种写法。在操作方面，字符串支持表达式（expression）运算，如拼接（concatenation，组合字符串）、切片（slicing，提取片段）、索引（indexing，按偏移取字符）等等。除了表达式，Python 还提供了一套方法（method）来实现常见的字符串专属任务，以及用于模式匹配等高级文本处理任务的模块（module）。

深度理解

·核心概念：本章第一个关键分类——字符串是“不可变序列”。拆开看：序列意味着有顺序、可按下标访问；不可变意味着不能改原对象，只能造新对象。这个二元标签是理解后面所有类型（列表、元组）的总纲。

·底层实现：CPython 中 str 是 PyUnicodeObject，内部存储的是 Unicode 码点序列，并按内容的复杂程度选择 1/2/4 字节的紧凑表示（compact representation）以省内存。这与 C 语言“char 数组 + 末尾 \0”的模型完全不同——所以 Python 的 len() 直接返回字符数，且任何字符（包括 \0）都不终止字符串。

·设计原因：没有独立的 char 类型、用“单字符串”表示单字符，是为了简化类型系统：一切字符串操作的对象统一为 str，无需像 C 那样区分 char 与 char。而“方法集随对象走”的设计（而不是像 C 那样靠库函数 strlen/strcpy）让字符串自带处理能力。

·实际问题：字符串可表示任意文本或字节数据——配置内容、日志、URL、JSON、二进制块。本章的操作（拼接、切片、方法、格式化）就是处理这些数据的"第一批武器"。

·初学者误区：把字符串想成"字符数组"。Python 字符串不是数组——它不能按下标赋值（不可变），也没有 C 的 \0 结束符概念。另一个误区是认为字符串只能装 ASCII 文本，实际上它从出生起就是 Unicode。

代码分析

表 7-1 的抽象示例（仅供预览，不能原样运行）：

```
S = '' #
S = "app's" #
S = 'c\no\td\x00e' #
S = """...
multiline
...""" #
S = r'\temp\data.txt' #
B = b'h\xc4ck' # 4 37
S = 'py\U0001F40D' # Unicode 4 37
S = u'py\U0001F40D' # Python 2.X 37
S1 + S2 / S * 3 #
S[i] / S[i:j] / len(S) #
S1 > S2, S1 == S2 #
'a %s coder' % kind #
'a {0} coder'.format(kind) #
f'a {kind} coder' # 3.6+
S.find('od') # 43 7.6
S.rstrip() #
S.replace('od', 'ood') #
S.split(',') # Unicode Unicode 7-3
S.isdigit()
```



```

S.lower()
S.endswith('thon')
S.join(strlist)
S.encode('utf8')
B.decode('latin1')
'py' in S.lower()           #
for x in S: print(x)
[c * 2 for c in S]
map(ord, S)
re.match('Py(.*?)on', line) #

```

解释器如何处理：表 7-1 的每一行都对应解释器的一个处理路径——字面量在编译期被解析成对应的字符串对象（转义序列在这一步就被展开成真正的字符），而表达式运算与方法调用在运行期执行。注意 `b'...'` 与 `'...'` 在编译期就会产生不同类型的对象（bytes vs str），两者在 Python 3 中互不自动转换。格式化字面量 `f'...'` 也是编译期识别——但它的 `{}` 内表达式要等运行期才求值（详见 7.9.3 节）。

7.3 字符串字面量 (String Literals)

英文原文

By and large, strings are fairly easy to use in Python. The first thing you need to know about them, though, is that there are many ways to write them in your code: - Single quotes: `'cod'e'` - Double quotes: `"cod'e"` - Triple quotes: `'''...code...'''`, `"""...code..."""` - Escape sequences: `"c\tto\nd\0e"` - Raw strings: `r"C:\new\test.bin"` (Chapter 37) - Bytes literals: `b'co\x01de'` (Chapter 37)

- Unicode literals: `'h\u00c4ck'` The single- and double-quoted forms are by far the most common; the others serve specialized roles, and we're postponing further discussion of the last two advanced forms until Chapter 37. Let's take a quick look at all the other options in turn.

中文翻译

总的来说, Python 中字符串用起来相当简单。但你需要知道的第一件事是: 写字符串的方式有很多种: - 单引号: `'code'` - 双引号: `"code"` - 三引号: `'''...code...'''`、`"""...code..."""` - 转义序列: `"c\t\n\d\0e"` - 原始字符串: `r"C:\new\test.bin"` (第 37 章) - 字节字面量: `b'co\x01de'` (第 37 章) - Unicode 字面量: `'h\u00c4ck'` 单引号和双引号形式是目前最常见的; 其余形式服务于专门用途, 最后两种高级形式我们推迟到第 37 章再讨论。下面依次快速看看所有其他选项。

深度理解

·核心概念: 字符串字面量 (string literal) 是"代码里书写字符串的方式"。同一种数据可以有多种语法外壳, 本质都是生成 `str` 对象。

·底层实现: 解释器在编译 (词法分析) 阶段读取字面量: 先看引号类型定界字符串范围, 再处理转义序列、原始字符串前缀 `r`、字节前缀 `b`、Unicode 前缀 `u` 等。这个阶段产生的错误 (如未闭合引号) 是 `SyntaxError`, 在程序运行前就会报出。

·设计原因：提供多种引号是为了"把字符串包起来时不用转义内部引号"；提供 `r` 是为了 Windows 路径和正则表达式；提供 `b/u` 前缀是历史遗留与 Python 2 兼容的需要。

·实际问题：写 Windows 路径 `'C:\new\text.dat'` 若不注意转义，路径会被悄悄篡改（`\n` 变换行）——这是新手最常见的问题之一。

·初学者误区：以为引号种类有语义差别。单双引号完全等价，只是风格选择；把 `"..."` 或 `'...'` 与"强/弱字符串"混为一谈（那是其他语言的概念）是大忌。

7.3.1 单引号与双引号等价 (Single and Double Quotes Are the Same)

英文原文

Around Python strings, single- and double-quote characters are interchangeable, though they must match, and must be straight quotes (beware tools that autocorrect to slanted quotes!). That is, string literals can be written enclosed in either two single or two double straight quotes—the two forms work the same and return the same type of object. For example, the following two strings, coded at the usual REPL, are identical once they are read by Python:

```
python >>> 'python', "python" ('python','python')
```

The reason for supporting both is that it allows you to embed a quote character of the other variety inside a string without escaping it with a backslash. You may embed a double-quote character in a string enclosed in single-quote characters, and vice versa, without having to use the escapes you'll meet in a moment:

```
python >>>'python"s', "python's" # 混用引号, 无需转义('python"s', "python's")
```

This book generally prefers to use single quotes around strings just because they are marginally easier to read, except in cases where a single quote is embedded in the string. This is a purely subjective style choice, but Python displays strings this way, too, and most Python programmers do the same today, so you probably should too. Note that the comma is important in the preceding code. Without it, Python automatically concatenates adjacent string literals of any kind. It is almost as simple to add a `+` operator between them to invoke concatenation explicitly, but adjacent literals are concatenated early when your code is read (and as you'll learn ahead, wrapping this form in parentheses also allows it to span multiple lines for larger blocks of text that can't use triple quotes):

```
python >>> title = "Learning "Python" " 6E" # 读取代码时隐式拼接>>> title'Learning Python 6E'
```

Adding commas between these strings would result in a tuple, not a string. Also notice in all of these outp

uts that Python prints strings in single quotes unless they embed one. If needed, you can also embed quote characters by escaping them with backslashes:

```
python >>>'python\'s', "python\'s" ("python's",'python's')
```

To understand why, you need to move on to learn how escapes work in general.

中文翻译

在 Python 字符串中，单引号和双引号字符是可以互换的，不过必须成对匹配，而且必须是直角引号（小心那些会把引号自动修正为弯引号的工具！）。也就是说，字符串字面量可以用两个单直引号或两个双直引号括起来——两种形式工作方式相同，返回的对象类型也一样。例如，下面两条在常见 REPL 中写下的字符串，一旦被 Python 读入就完全一样：python >>>'python', "python" ('python','python') 支持两种引号的原因是：允许你在字符串中嵌入另一种引号，而无需使用反斜杠转义。你可以在单引号字符串里嵌入双引号字符，反之亦然，不必使用马上要学的转义：python >>>'python"s', "python's" # 混用引号，无需转义('python"s', "python's") 本书一般偏好用单引号包裹字符串，因为略微易读一些，除非字符串内嵌了单引号。这纯粹是主观风格选择，但 Python 本身也这样显示字符串，如今大多数 Python 程序员也这么做，所以你大概也应该照做。注意上面的代码中逗号很重要。没有逗号时，Python 会自动拼接（concatenate）任意相邻的字符串字面量。在它们之间加一个 + 运算符显式调用拼接也很简单，但相邻字面量是在代码被读取时提前拼接的（而且你稍后会学到：用括号把这

个形式包起来，还能让长文本跨越多行，适用于不能用三引号的大段文本）：`python >>> title = "Learning "Python' " 6E" # 读取代码时隐式拼接>>> title'Learning Python 6E'`如果在这些字符串之间加上逗号，得到的将是元组（tuple），而不是字符串。另外请注意所有这些输出中：除非字符串内嵌了单引号，否则 Python 用单引号显示字符串。需要时，你也可以用反斜杠转义来嵌入引号字符：`python >>>'python\'s', "python\'s" ("python's",'python's')`要理解为什么会这样，你需要继续学习转义的一般工作方式

深度理解

·核心概念：单双引号等价 + 隐式拼接（adjacent literal concatenation）。前者是自由度，后者是编译期的便利语法。

·底层实现：隐式拼接发生在编译期——`"Learning "Python' " 6E"`在词法分析时就被合并成单个字符串常量，运行时只有一个字符串对象，没有任何拼接运算的字节码。这比 `"a" + "b"`（运行期执行 BINARYOP + 生成新对象）更高效。

·设计原因：提供两种引号让“字符串内嵌引号”无需转义，可读性更好；而隐式拼接则让长字符串可以自然断行、分段书写，还允许混合使用不同引号风格。

·实际问题：写长 SQL、HTML 片段时，用括号包裹的隐式拼接是常用手法；%、.format 模板与文字混排时也常靠它断行。

·初学者误区：误以为'a''b' 是语法错误或会得到元组——加逗号才会得到元组。还有人忘记隐式拼接的存在，误以为相邻字符串彼此"孤立"。

7.3.2 转义序列是特殊字符 (Escape Sequences Are Special Characters)

英文原文

The prior example embedded a quote inside a string by preceding it with a backslash (`\`). This is representative of a general pattern in strings: backslashes are used to introduce special character codings known as escape sequences. Escape sequences let us embed characters in strings that cannot easily be inserted into a string literal or typed on a keyboard. The character `\`, and one or more characters following it in the string literal, are replaced with a single character in the resulting string object, which has the value specified by the escape sequence. For example, here is a five-character string that embeds a newline and a tab:

```
python >>> s = 'a\nb\tc'
```

The two characters `\n` stand for a single character—the newline character (technically speaking, code-point value 10, which means newline in Unicode and its ASCII subset). Similarly, the sequence `\t` is replaced with the tab character (code point 9). The way this string looks when printed depends on how you print it. The interactive echo shows the s

pecial characters as escapes, but print interprets the m instead:

```
python >>> s'a\nb\tc'
```

```
>>> print(s) a b c
```

To be completely sure how many actual characters are in this string, use the built-in len function—it returns the actual number of characters in a string, regardless of how it is coded or displayed:

```
python >>> len(s) 5
```

This string is five characters long: it contains an ASCII a, a newline character, an ASCII b, and so on. NOTE — But length is not bytes: If you're accustomed to all-ASCII text, it's tempting to think of this len result as meaning five bytes too, but you probably shouldn't. Really, "bytes" in today's Unicode world doesn't hold the meaning it once did. For one thing, the Python string object includes admin data that makes it larger in memory than its text alone. For another, string content and length both reflect code points (identifying numbers) assigned by Unicode, and a single character's code point does not necessarily map to a single byte—either when decoded in memory or encoded in files. Under encoding UTF-16, for example, ASCII characters are multiple bytes in files and may be any size at all in memory depending on how Python allocates their space. Moreover, code-point values of non-ASCII characters like (e.g., the snake emoji \U0001F40D fr

om Table 7-1) are far too large to fit in an 8-bit byte in any form. In fact, Python defines str strings formally as sequences of Unicode code points, not bytes, to make this clear. There's much more on how Unicode text obviates bytes in Chapter 37 when you're ready to take the plunge. For now, to avoid confusion, think characters instead of bytes in strings. Notice that the original backslash characters in the preceding examples are not really stored with the string in memory; they are used only to describe special character values to be stored in the string. For coding such special characters, Python recognizes a full set of escape code sequences, listed for reference in Table 7-2.

Table 7-2. String backslash characters (字符串反斜杠字符)

转义	含义
\\	反斜杠 (存储一个 \)
\'	单引号 (存储')
\"	双引号 (存储 ")
\n	换行 (newline, 又名 linefeed)
\r	回车 (carriage return, 如 Windows 的 \r\n)
\t	水平制表符 (horizontal tab)
\v	垂直制表符 (vertical tab)
\a	响铃 (bell, 在支持处)
\b	退格 (backspace)
\f	换页 (formfeed)
\xhh	十六进制码点或字节值 (恰好 2 位十六进制数字)
\ooo	八进制码点或字节值 (最多 3 位, 上限 \377)

<code>\0</code>	空字符 (null) : 八进制二进制 0 字符 (不终止字符串)
<code>\uhhhh</code>	Unicode 字符码点, 16 位值 (恰好 4 位十六进制数字)
<code>\Uhhhhhhhhh</code>	Unicode 字符码点, 32 位值 (恰好 8 位十六进制数字)
<code>\N{name}</code>	Unicode 数据库中 ID 为 name 的字符
<code>\ 换行</code>	被忽略 (前置于续行)
<code>\ 其他</code>	原样保留, 但 3.12 中给出警告, 将来会变成错误

Some of Table 7-2's escapes come with usage rules. Again for reference, here are the fine points: - The `\x`, `\u`, and `\U` escape sequences require exactly two, four, and eight hexadecimal digits, respectively. Use digits 0–9 and A–F (uppercase or lowercase) for h. - The `\o` escape accepts one to three octal digits and issues a warning for values over `\377` in Python 3.12 because values too big for a byte cause issues in byte strings. Use digits 0–7 for o. - Both `\u` and `\U` are recognized only in str text-string literals (e.g., `'...'`), where they give a character's Unicode code-point value. This is the code point's decoded value. - `\x` and `\o` escapes work in both bytes byte-string literals (`b'...'`), where they give a byte's absolute value; and in str text-string literals, where they give a character's Unicode code-point value. - Lettered escapes like `\n` stand for their Unicode code points in text and their ASCII encodings in bytes, even if the two values agree (there is more on Unicode escapes in Chapter 37). Let's get back to running code. Some string escape sequences allow you to

o embed absolute values as characters of a string. When you do this, you're really giving the code-point value of the desired character. For instance, here's a five-character string that embeds two characters with zero values (coded as octal escapes of one digit):

```
python >>> s = 'a\0b\0c'>>> s'a\x00b\x00c'
>>> len(s) 5
```

The character associated with code point 0 is generally known as NULL (or NUL). Importantly, in Python, a character like this does not terminate a string the way a "null byte" typically does in C. Instead, Python keeps both the string's length and text in memory. In fact, no character terminates a string in Python. Here's a string that is all absolute escape codes—an absolute 1 and 2 (coded in octal), followed by an absolute 3 (coded in hexadecimal), and nonprintables all:

```
python >>> s = '\001\002\x03'>>> s'\x01\x02\x03'
>>> len(s) 3
```

Notice that Python displays nonprintable characters in hex, regardless of how they were specified. When needed, you can freely combine characters, absolute-value escapes, and the more symbolic escapes in Table 7-2. To demo, the following string contains the characters "HACK", a tab and newline, and an absolute zero character coded in hex:

```
python >>> S = 'H\tA\nC\x00K'>>> S'H\tA\nC\x00K'
>>> len(S) 7
>>> print(S) H A CK
```

This becomes more important to know when you process binary data files in Python. Because their contents are represented as strings in your scripts, it's OK to process binary files that contain any sorts of binary byte values. When opened in binary modes, files return raw bytes from the external file as bytes—a string variant that supports most of the syntax and tools in this chapter (you'll find more on files and bytes in Chapters 4, 9, and 37). Two limits in Table 7-2 merit callouts. First of all, octal escapes with values too large for a byte issue warnings as of Python 3.12 and will be errors soon—despite the fact that these escapes in text strings denote code points, not bytes:

```
python >>> '\400'<stdin>:1: SyntaxWarning: invalid octal escape sequence'\400'Ä'
```

This seems unlikely to break much code, but the last entry in Table 7-2 may: if Python does not recognize the character after a `\` as being a valid escape code, it simply keeps the backslash in the resulting string—at least for the moment:

```
python >>> x = 'C:\py\code'<stdin>:1: SyntaxWarning: invalid escape sequence'\p'
```

```
>>> x # 原样保留 \ (并显示为 \\) 'C:\\py\\code'
```

```
>>> len(x) # 但这种行为不会持久：别再依赖它了！ 10
```

Despite this behavior being expected (and even relied upon) for three decades, it has recently been judged bad and has started issuing a warning when used. In Python 3.12, it invokes a syntax warning that doesn't stop your program but clutters your output with `ngs`. Worse, this will be treated as a programming-ending error in a future Python, so you should not use this going forward—and are expected to change all the code you've written in the past that does! Even without this backward-incompatible Python change, though, strings that rely on this behavior seem as likely to lose backslashes in escapes as to retain them elsewhere. Instead, code literal backslashes explicitly such that they are retained in your strings in all Pythons, by either doubling them with `\\` (an escape for one `\`), or using raw strings—the topic of the next section.

中文翻译

前面的示例通过在引号前加反斜杠 (`\`) 把引号嵌入字符串。这代表了字符串中的一种通用模式：反斜杠用来引入称为转义序列 (escape sequence) 的特殊字符编码。转义序列让我们能在字符串中嵌入那些难以直接写进字面量、或难以在键盘上敲出的字符。字面量中的 `\` 及其后面一个或多个字符，在最终字符串对象中会被替换为一个字符，其值由转义序列指定。例如，下面是一个嵌入了换行符和制表符的五字符字符串：`python >>> s = 'a\nb\tc'` 字符对 `\n` 代表一

个字符——换行 (newline) 字符 (严格说, 是码点值 10, 在 Unicode 及其 ASCII 子集中都表示换行)。类似地, `\t` 被替换为制表符 (tab) (码点 9)。这个字符串打印出来什么样, 取决于你怎么打印它。交互式回显把特殊字符显示为转义形式, 而 `print` 会把它们解释成实际字符: `python >>> s'a\nb\tc'>>> print(s)` a b c 要想确切知道这个字符串到底有多少个实际字符, 请使用内置的 `len` 函数——它返回字符串中实际的字符个数, 无论字符串如何编码或显示: `python >>> len(s)` 5 这个字符串有 5 个字符: 一个 ASCII 的 a、一个换行字符、一个 ASCII 的 b, 等等。注意——长度不等于字节数: 如果你习惯全 ASCII 文本, 很容易把这里的 `len` 结果也当作 5 个字节, 但你不该这样想。说真的, 在今天这个 Unicode 世界里, "字节 (bytes)" 已不再有过过去那种含义。其一, Python 字符串对象包含管理数据, 使它在内存中比其文本本身更大; 其二, 字符串的内容与长度都反映 Unicode 分配的码点 (code point) (标识数字), 而单个字符的码点未必映射到单个字节——无论是在内存中解码, 还是在文件中编码。例如在 UTF-16 编码下, ASCII 字符在文件中占多个字节, 在内存中可能占任意大小 (取决于 Python 如何分配空间)。况且, 非 ASCII 字符 (如 蛇形表情符号, `\U0001F40D`) 的码点值在任何形式下都太大, 放不进一个 8 位字节。事实上, Python 正式把 `str` 字符串定义为 Unicode 码点序列而不是字节序列, 就是为了把这件事说清楚。关于 Unicode 文本如何取代字节的更多内容在第 37 章, 等你准备好再深入。眼下, 为避免混淆, 在字符串里请想字符 (character) 而不是字节 (byte)。注意, 前面示例中的原始反斜杠字符并不会真的随字符串存进内存; 它们只是用来描述要存储的特殊字符

值。为了编写这些特殊字符，Python 识别一整套转义码序列，供参考列在表 7-2 中。

表 7-2 字符串反斜杠字符（见上方英文原文下方表格，中文见“含义”列）

表 7-2 中的一些转义带有使用规则。同样供参考，细则如下：- \x、\u、\U 转义分别恰好需要 2、4、8 位十六进制数字。h 用数字 0-9 和字母 A-F（大小写均可）。- \o 转义接受 1 到 3 位八进制数字，在 Python 3.12 中，值超过 \377 会发出警告，因为对字节来说太大的值会在字节字符串中引发问题。o 用数字 0-7。- \u 和 \U 都只在 str 文本字符串字面量（如'...'）中被识别，给出字符的 Unicode 码点值，即码点的解码值。- \x 和 \o 转义在 bytes 字节字符串字面量（b'...'）中给出字节的绝对值；在 str 文本字符串字面量中给出字符的 Unicode 码点值。- 像 \n 这样的字母型转义，在文本中代表其 Unicode 码点、在字节中代表其 ASCII 编码，即使两者数值相同（Unicode 转义的更多内容见第 37 章）。让我们回到运行代码。某些字符串转义序列允许你把绝对值作为字符嵌入字符串。做这件事时，你实际上给出的是期望字符的码点值。例如，下面是一个嵌入了两个零值字符（用一位八进制转义编码）的五字符字符串：

```
python >>> s = 'a\0b\0c'>>> s'a\x00b\x00c'>>> len(s) 5
```

码点 0 对应的字符一般称为 NULL（或 NUL）。重要的是，在 Python 中，这样的字符不会像 C 语言中的“空字节”那样终止字符串。相反，Python 在内存中同时保存字符串的长度和文本。事实上，没有任何字符能终止 Python 字符串。下面是一个全由绝对转义码组成的字符串——绝对 1 和 2（八进制编码），后面跟一个绝对 3（十六进制编码），全部不可打印：python >>> s = '\001\002\x03'>

>> s'\x01\x02\x03'>>> len(s) 3 注意，无论你当初用什么方式指定，Python 都一律用十六进制显示不可打印字符。需要时，你可以自由组合普通字符、绝对值转义和表 7-2 中更符号化的转义。演示一下：下面的字符串包含字符"HA CK"、一个制表符、一个换行符和一个用十六进制编码的绝对零字符：python >>> S='H\tA\nC\x00K'>>> S'H\tA\nC\x00K'>>> len(S) 7 >>> print(S) H A CK 当你在 Python 中处理二进制数据文件时，了解这一点变得更加重要。因为二进制文件内容在脚本中以字符串表示，所以处理包含任意二进制字节值的文件完全没问题。以二进制模式打开文件时，文件把外部文件中的原始字节作为 bytes 返回——这是字符串的一种变体，支持本章大部分语法和工具（文件和 bytes 的更多内容见第 4、9、37 章）。表 7-2 中有两个限制值得一提。首先，值太大、放不进一个字节的八进制转义从 Python 3.12 起会发出警告，并且很快就会变成错误——尽管这些转义在文本字符串中表示的是码点而不是字节：python >>> '\400'<stdin>:1: SyntaxWarning: invalid octal escape sequence'\400'Ã'这似乎不太可能破坏多少代码，但表 7-2 中最后一项可能会：如果 Python 不把 \ 后面的字符识别为有效转义码，它就直接把反斜杠保留在结果字符串中——至少目前是这样：python >>> x = 'C:\py\code'<stdin>:1: SyntaxWarning: invalid escape sequence'\p'>>> x # 原样保留 \（并显示为 \\）'C:\\py\\code'>>> len(x) # 但这种行为不会持久：别再依赖它了！10 尽管这一行为三十年里一直被视为预期行为（甚至被依赖），最近它被判定为不良行为，并在使用时开始发出警告。在 Python 3.12 中，它会引发语法警告（SyntaxWarning）——不会中断程序，但会用唠叨污染你的输出。更

糟的是，在未来某个 Python 版本中，这会被当作直接终止程序的错误（error）处理，所以今后不要再这样写——而且你过去写的所有这样写的代码都要改！不过，即使没有这个破坏向后兼容性的 Python 变更，依赖该行为的字符串也既可能保住转义外的反斜杠、也可能在转义中丢失反斜杠。正确的做法是显式写出字面反斜杠，让字符串在所有 Python 版本中都保留它们：要么把 `\\` 加倍（一个 `\\` 转义成一个 `\`），要么使用原始字符串——这就是下一节的主题。

深度理解

- 核心概念：转义序列是“源代码中的多字符记号 → 内存中的单字符”的编译期替换机制。写的是 `\n` 两个字符，存的是换行一个字符——反斜杠从不进内存。
- 底层实现：词法分析器（tokenizer）扫描字面量时处理 `\` 后接字符的组合，把结果编码进字符串对象内部（Unicode 码点）。因为对象自带长度字段（而 C 用 `\0` 结尾），所以 `\0` 嵌入字符串中间完全安全——`len()` 照样正确数出 5、3、7 个字符。
- 设计原因：不能用键盘直接输入换行、制表符、Unicode 字符，也不能把回车真正“写进”源代码（否则会结束代码行）。转义序列让任意码点都能以纯文本形式表达。`len()` 数的是码点数而非字节数，是为了与“`str = Unicode 码点序列`”的正式定义一致。
- 实际问题：处理二进制文件、生成带换行的文本、嵌入 `\r` `\n` 与 `\t` 分隔的日志格式时，转义是日常操作；`\u`/`\U`/`\N`{ } 让非 ASCII 字符可跨编辑器安全书写。

·初学者误区：(1) 以为 `\n` 是两个字符——`len('a\nb\tc')` 是 5 不是 7；(2) 以为 `\0` 会像 C 一样截断字符串；(3) 在 3.12 仍写 `'\400'` 或 `'\p'` 这类非法转义，依赖“原样保留”的旧行为——它已告警并将成为错误，必须改成 `\\` 或原始字符串。

代码分析

```
>>> s = 'a\0b\0c'          # \0
>>> s
'a\x00b\x00c'             # \x00
>>> len(s)                  # 5
5

>>> S = 'H\tA\nC\x00K'     # + + +
>>> len(S)
7                           # H\tA\nC\x00K  7
>>> print(S)               # print
H A
CK
```

解释器如何处理：这两段代码展示了“编码时（转义形式）→ 存储时（码点序列）→ 显示时（两种格式）”的完整链路。`'\001\002\x03'` 中八进制 `\001`、`\002` 与十六进制 `\x03` 在编译期都被换算成数值 1、2、3 三个码点；字符串对象只存“码点 + 长度”，不存反斜杠。回显（`repr`）为了让你能复制回代码，用转义形式输出；`print` 则走显示路径，把控制字符直接解释输出——所以 `print(S)` 里制表符真的把文本推开了，`\x00` 只是不打印。这就是“同一字符串，两种面貌”的底层原因：`repr` 面向程序员，`print` 面向用户。

7.3.3 原始字符串抑制转义 (Raw Strings Suppress Escapes)

英文原文

As we've already seen, escape sequences are handy for embedding special characters in strings. Sometimes, though, the special treatment of backslashes for introducing escapes can lead to trouble. It's surprisingly common, for instance, to see Python newcomers trying to open a file on Windows with a filename argument that looks something like this: `python myfile = open('C:\new\text.dat','w')` thinking that they will open a file called `text.dat` in the directory `C:\new`. The problem with this is that `\n` is taken to stand for a newline character, and `\t` is replaced with a tab. In effect, the call tries to open a file named `C:(newline)ew(tab)ext.dat`, with usually less-than-stellar results. This is just the sort of thing that raw strings are meant to address. If the letter `r` (uppercase or lowercase, but usually the latter) appears just before the opening quote of any string literal covered in this chapter, it turns off the escape mechanism. The result is that Python retains your backslashes literally, exactly as they appear in the string. Hence, to fix the filename problem, just remember to add the letter `r` on Windows: `python myfile = open(r'C:\new\text.dat','w')` # 可行: 禁用 \ 转义 Alternatively, because, as noted in the preceding section,

two backslashes are really an escape sequence for one backslash, you can keep your backslashes by simply doubling them up when they should be taken verbatim: `python myfile = open('C:\\new\\text.dat','w')` # 同样可行: \\ 表示 \

In fact, Python itself sometimes uses this doubling scheme when it prints strings with embedded backslashes: `python >>> path = r'C:\new\text.dat'` # 原始字符串: 保留所有 \

`>>> path` # 以 Python 代码形式显示 `'C:\\new\\text.dat'`

`>>> print(path)` # 用户友好格式 `C:\new\text.dat`

`>>> len(path)` # 字符串长度: 每个 \ 算一个字符 15

As covered in Chapter 5, the default format at the interactive prompt prints results as if they were code, and therefore escapes backslashes in the output. The `print` statement provides a more user-friendly format that shows that there is actually only one backslash in each spot. To verify this is the case, you can check the result of the built-in `len` function, which returns the number of characters in the string, independent of display formats. If you count the characters in the `print(path)` output, you'll see that there really is just 1 character per backslash, for a total of 15. Besides directory paths on Windows, raw strings are also commonly used for regular expressions in text pattern matching, supported with the `re` module introduced in Chapter 37. Also note that Python scripts can usually use forward slashes in directory paths on both Windows and Unix because this slash is interpreted portably (e.g., `'C:/new/text.dat'` works when openi

ng Windows files, too). Raw strings are useful for paths using native Windows backslashes, though, and any other time you want to ensure that Python will leave your `\` alone. They also work for triple-quoted strings to suppress escapes (and future invalid-escape errors!) in text of the sort up next.

NOTE — Raw-string quirk: Despite its role, even a raw string cannot end in a single backslash, because the backslash escapes the following quote character. That is, `r'...\'` is not a valid string literal: you still must escape the surrounding quote character to embed it in the string, and Python assumes that is your intent. The upshot is that a raw string cannot end in an odd number of backslashes, including one. If you need to end a raw string with a single backslash, you can use two and slice off the second (`r'...\[:-1]`), tack one on manually (`r'...' + '\\'`), or skip the raw string syntax and just double up the backslashes in a normal string (`'...\\'`). All three of these forms create the same two-character string with a Unicode ellipsis and one backslash.

中文翻译

如前所见，转义序列在字符串中嵌入特殊字符很方便。但有时，反斜杠引入转义的这种特殊处理也会带来麻烦。比如，惊人的常见情形是：Python 新手想在 Windows 上打开文件，文件名参数写成这样：`python myfile = open('C:\new\text.dat','w')` 以为会打开 C:\new 目录下名为 text.dat

的文件。问题在于 `\n` 被解读为换行字符，`\t` 被替换为制表符。实际上，这个调用试图打开的文件名叫 `C:(换行)ew(制表符)ext.dat`，结果通常不会太好。这正是原始字符串（raw string）要解决的问题。如果在本章涵盖的任何字符串字面量开始引号前出现字母 `r`（大写或小写均可，但通常用小写），它就关闭转义机制。结果是 Python 原样保留你的反斜杠，与字符串中出现的完全一致。因此，要修复文件名问题，只需记住在 Windows 上加字母 `r`：`python myfile = open(r'C:\new\text.dat','w')` # 可行：禁用 \ 转义另外，正如前一节所述，两个反斜杠其实是一个反斜杠的转义序列，所以你也可以在需要原样保留反斜杠的地方把它们加倍：`python myfile = open('C:\\new\\text.dat','w')` # 同样可行：\\ 表示 \ 事实上，Python 自己在打印含有嵌入反斜杠的字符串时，有时也采用这种加倍方案：`python >>> path = r'C:\new\text.dat' # 原始字符串：保留所有 \ >>> path # 以 Python 代码形式显示'C:\\new\\text.dat'>>> print(path) # 用户友好格式C:\new\text.dat >>> len(path) # 字符串长度：每个 \ 算一个字符15` 如第 5 章所述，交互式提示符的默认格式会像打印代码一样输出结果，因此在输出中转义反斜杠。`print` 语句提供更用户友好的格式，显示每个位置实际上只有一个反斜杠。要验证这一点，可以检查内置 `len` 函数的结果——它返回字符串中字符个数，与显示格式无关。如果你数一下 `print(path)` 输出中的字符，会发现每个反斜杠确实只有 1 个字符，总共 15 个。除了 Windows 目录路径，原始字符串也常用于第 37 章引入的 `re` 模块所支持的模式匹配正则表达式。另外注意，Python 脚本在 Windows 和 Unix 的目录路径中通常都可以使用正斜杠，因为这个斜杠被跨平台解释（比如 `'C:/new/text.dat'`）

在打开 Windows 文件时同样有效)。不过, 对于使用 Windows 原生反斜杠的路径, 以及任何你希望 Python 别碰你的 \ 的场合, 原始字符串都很管用。它们对三引号字符串同样生效, 可以抑制下一节那种文本中的转义 (以及未来的非法转义错误!)。

注意——原始字符串的怪癖: 尽管名为原始, 即便是原始字符串也不能以单个反斜杠结尾, 因为反斜杠会把后面的引号字符转义掉。也就是说, `r'...\'` 不是合法的字符串字面量: 你仍然必须转义周围的引号字符才能把它嵌入字符串, 而 Python 会假定你正是这个意图。结论是: 原始字符串不能以奇数个反斜杠结尾——哪怕只有一个也不行。如果你需要让原始字符串以单个反斜杠结尾, 可以用两个然后切片去掉第二个 (`r'...\'[:-1]`), 手动补一个 (`r'...' + '\\'`), 或者干脆不用原始字符串语法, 在普通字符串里把反斜杠加倍 (`'...\\'`)。这三种写法都生成同一个两字符字符串: 一个 Unicode 省略号加一个反斜杠。

深度理解

·核心概念: 原始字符串是一个编译期开关——`r` 前缀告诉词法分析器“本字面量内部不要处理转义序列”, 反斜杠和后随字符一律按字面保留 (仍有唯一例外: `\'` 仍会转义引号, 因为引号决定字面量边界)。

·底层实现: 加 `r` 后, tokenizer 不再把 `\n`、`\t` 等展开为单个码点, 而是把 `\` 当普通字符存入字符串对象; `path` 的内存内容就是 `C:\new\text.dat` (15 个字符)。交互式回显用 `repr` 输出, `repr` 会再次把 `\` 显示成 `\\` 以便复制回代码, 所以显示的是 17 个字符的样子, 但 `len()` 诚实报

告 15。

·设计原因：Windows 路径和正则表达式充满反斜杠，若强制转义，代码会变成反斜杠地狱。r 前缀让这些领域里的反斜杠"所见即所得"。

·实际问题：open(r'C:\new\text.dat')、re.compile(r'\d+\.\d+') 是标准用法。正斜杠'C:/new/text.dat' 在 Python 的 open 中通常也能用，但很多第三方工具（如子进程命令）仍按反斜杠理解路径。

·初学者误区：(1) 以为 r 能"修复"所有反斜杠问题——它照样受 3.12 的非法转义警告影响（如 r'\p' 其实没有 \p 可转义，但 r 下 \p 合法，因为不尝试转义）；(2) 以为原始字符串能以单个 \ 结尾——语法上不允许，这是最经典的反直觉陷阱；(3) 把 r'...' 与"二进制原始数据"混淆，r 控制的是转义不是编码。

代码分析

```
>>> path = r'C:\new\text.dat'    # \n
>>> path                          # repr  \  \
'C:\\new\\text.dat'
>>> print(path)                  # 15
C:\new\text.dat
>>> len(path)                    #
15
```

解释器如何处理：词法分析阶段，r 前缀让 tokenizer 跳过转义展开处理，把 C:\new\text.dat 的每个字符（含两个反斜杠）原样写进字符串常量池；运行时这个对象就是 15 个码点。回显走 repr()：repr 的目标是"输出能被当作 Python 代码重新读入"，所以每个 \ 被写成 \\（你对这一

个反斜杠所做的第二次转义编码，是 repr 在显示时做的，与原始字符串本身无关）。print 走 str()：str 的目标是给人看，所以直接输出原始内容。len() 绕开所有显示层，直接读对象内部的长度字段——这就是 15。设计思想：编码方式、内存表示、显示方式三者可各自不同，判断真相的唯一权威是 len() 与 ord()。

7.3.4 三引号与多行字符串 (Triple Quotes and Multiline Strings)

英文原文

So far, you've seen single quotes, double quotes, escapes, and raw strings in action. Python also has a triple-quoted string literal format, sometimes called a block string, that is a syntactic convenience for coding multiline data. This form begins with three quotes (of either the single or double variety), is followed by any number of lines of text, and is closed with the same triple-quote sequence that opened it. Single and double quotes embedded in the string's text may be, but do not have to be, escaped—the string does not end until Python sees three unescaped quotes of the same kind used to start the literal. For example:

```
python >>> quip = """Python strings ... sure have ...
a lot of options""" >>> quip'Python strings\n sure ha
ve\na lot of options'
```

This string spans three lines. As you learned in Chapter 3, the interactive prompt changes to ... on continuation lines like this in some interfaces, but not in others. This book omits the dots in some examples to enable cut-and-paste, but don't type them yourself if they're listed as they are here but absent in your REPL, and extrapolate as needed. Prompts aside, Python collects all the triple-quoted text in this example into a single multiline string, with embedded newline characters (`\n`) at the places where your code has physical line breaks. Notice that, as in the literal, the second line in the result has leading spaces, but the third does not—what you type is truly what you get. To see the string with the newlines interpreted, print it instead of echoing:

```
python >>> print(quip) Python strings sure have a lot of options
```

In fact, triple-quoted strings will retain all the enclosed text, including any to the right of your code that you might intend as comments. So don't do this—put your comments above or below the quoted text, or use the automatic concatenation of adjacent strings mentioned earlier, with explicit newlines if needed, and surrounding parentheses to allow line spans (you'll learn more about this latter form when you study syntax rules in Chapters 10 and 12):

```
python >>> menu = """ ... Open # Comments here a
```

```
added to string! ... Save # Ditto ... """ >>> menu'\nOpen # Comments here added to string!\nSave # Ditto\n'
```

```
>>> menu = ( ...'Open\n' # Comments here ignored .  
..'Save\n' # But newlines not automatic ... ) >>> menu'  
u'Open\nSave\n'
```

So why use triple-quoted strings? For one thing, they are useful anytime you need multiline text in your program—for example, to embed multiline error messages, or HTML, XML, JSON, or YAML code in your Python source code files. You can usually embed such blocks directly in your scripts by triple-quoting without resorting to external text files or concatenation and newline characters. Triple-quoted strings are also commonly used for docstrings (documentation strings), which are string literals that are taken as comments when they appear at specific points in your file (and are covered in full in Chapter 15). These don't have to be triple-quoted blocks, but they usually are to allow for multiline comments and may need to be triple-quoted raw strings (e.g., `r'''`) to avoid invalid escape errors in the future (see the 3.12 syntax warnings noted previously). Finally, triple-quoted strings are also sometimes used as a "horribly hackish" way to temporarily disable lines of code during development. Really, it's not too horrible, and it's actually a fairly common practice today, but it wasn't the original intent of the literal. If you wish to turn off a few lines of code and run y

our script again, simply put three quotes above and below them, like this:

```
python X = 1 """
```

```
import os # 临时禁用这段代码print(os.getcwd()) """
```

Y = 2 This was tagged as "hackish" because Python really might make a string out of the lines of code disabled this way, but this is probably not significant in terms of performance. For large sections of code, it's also easier than manually adding hash marks before each line and later removing them. This is especially true if you are using a text editor that does not have support for editing Python code specifically. In Python, practicality often beats aesthetics.

中文翻译

到目前为止，你已经看过单引号、双引号、转义和原始字符串的实际用法。Python 还有一种三引号字符串字面量格式，有时称为块字符串（block string），是编写多行数据的语法便利。这种形式以三个引号开头（单引号或双引号都可以），后面接任意行数的文本，并以同样三个引号序列（与开头同种引号）闭合。字符串文本中嵌入的单、双引号可以但不必须转义——在 Python 看到三个未转义的同类引号之前，字符串不会结束。例如：

```
python >>> quip = """Python strings ... sure have ... a lot of options""" >>> quip
```

`Python strings\n sure have\n a lot of options'`这个字符串横跨三行。如第 3 章所学，某些交互界面在这样续行时提示符会变成 ...，另一些则不会。本书有些示例省略了

这些点以便复制粘贴；但如果像这样列出而你的 REPL 里没有这些点，不要自己输入它们，按需变通即可。提示符问题放一边。Python 把本示例中三引号内的所有文本收集成一个多行字符串，在有物理换行的位置嵌入换行字符（\n）。注意，如字面量所示，结果的第二行有前导空格，第三行没有——你写什么就是什么。要想看到换行被解释的字符串，用 print 而不是回显：python >>> print(quip) Python strings sure have a lot of options 事实上，三引号字符串会保留所有被括起来的文本，包括代码右侧你原本当作注释的那些内容。所以别这么干——把注释放在引用文本的上方或下方，或者用前面提到的相邻字符串自动拼接，需要时显式写换行，并用括号包裹以便跨行（这个后一种形式在你学第 10、12 章语法规则时会了解更多）：python >>> menu = """ ... Open # 这几句"注释"被加进了字符串！ ... Save # 同上... """ >>> menu'\nOpen # Comments here added to string!\nSave # Ditto\n'>>> menu = (...'Open\n' # 这里的注释被忽略...'Save\n' # 但换行不会自动添加...) >>> menu'Open\nSave\n'那为什么要用三引号字符串？其一，任何时候程序里需要多行文本它们都很有用——比如在 Python 源文件中嵌入多行错误消息，或 HTML、XML、JSON、YAML 代码。通常你可以直接用三引号把这样的块嵌进脚本，不必求助于外部文本文件或拼接加换行字符。三引号字符串也常用于文档字符串（docstring）：当字符串字面量出现在文件的特定位置时，会被当作注释（第 15 章完整讲解）。文档字符串不一定是三引号块，但通常是，以便容纳多行注释；而且可能需要写成三引号原始字符串（如 r'''），以避免将来出现非法转义错误（见前面提到的 3.12 语法警告）。最后，三引号字符串有时也被

用作一种"相当 hack"的方式，在开发期间临时禁用若干行代码。其实没那么糟，如今这也是相当常见的做法，但它本非这种字面量的本意。如果你想关掉几行代码再跑一遍脚本，只需在它们上下各放三个引号，像这样：`python X = 1 "`
`""" import os # 临时禁用这段代码print(os.getcwd()) """`
`Y = 2` 这被贴上"hack"的标签，是因为 Python 确实可能把这样禁用的代码行变成一个字符串，但对性能来说这大概无关紧要。对较大段的代码，这也比手动在每行前加井号、之后再删掉更省事——尤其在你使用一个不支持 Python 代码编辑功能的文本编辑器时。在 Python 里，实用性常常胜过美观。

深度理解

·核心概念：三引号字面量 = 跨物理行的字符串常量。它不在运行时"拼接多行"，而是在编译期把多行文本连同其间的换行符直接合成一个字符串对象。被...内嵌的引号无需转义，因为结算点 (terminator) 是连续三个同类未转义引号。

·底层实现：tokenizer 从开头的三引号一直读到结尾的三引号，过程中的物理换行被转换成 `\n` 码点存入字符串，行首缩进原样保留（quip 第二行 `sure have` 的空白就在）。对比普通单行指定拼 `'Open\n' + 'Save\n'`：显式换行法由你自己控制每个 `\n`。

·设计原因：多行文本（消息模板、HTML/XML 片段、文档字符串）若用单引号+`\n` 拼写会冗长难读；若用外部文件又增加资源管理负担。三引号让"源代码即文本"。

·实际问题：docstring ('''...''' 紧随函数/类/模块声明之后成为 .doc)；临时注释大段代码（比逐行 # 更易撤销）；模板片段直接内嵌。

·初学者误区：(1) 以为行内 # 注释会生效——三引号里的 # 是普通字符；(2) 以为缩进会被自动去除——不会，textwrap.dedent 才是你要的工具；(3) 用它"注释掉"代码时，注意字符串内若出现三连同类引号会提前结束；(4) 把三引号当注释功能用，遮盖了它的真实身份——它永远先是一个字符串字面量。

代码分析

```
>>> quip = """Python strings
... sure have
... a lot of options"""
```

解释器如何处理：tokenizer 读入三行文本物理行，每处换行存入 \n，生成码点序列 Python strings + \n + sure have + \n + a lot of options——注意第二行前面那个空格被保留（回显中的 'sure have'），这正是"所见即所得"。这不是运行时操作，字节码里直接是一条 LOADCONST 加载这个已构造好的常量。对比括号+隐式拼接版本：('Open\n' # 注释忽略\n'Save\n') 中 # 注释被词法分析器跳过（它们在字面量之外、位置合法），而 'Open\n' 与 'Save\n' 两个相邻字面量在编译期合并为 'Open\nSave\n'。设计思想：边界决定语义——注释的位置（引号外/内）决定了它是被忽略还是成为字符内容，理解 tokenizer 的定界规则就能预见这些行为。

7.4 字符串实战 (Strings in Action)

英文原文

Once you've created a string with the literal expressions we just met, you will almost certainly want to do things with it. This section and the next two demonstrate string expressions, methods, and formatting—the first line of text-processing tools in the Python language.

中文翻译

一旦你用刚才学过的字面量表达式创建了字符串，你几乎肯定想对它做点什么。本节以及接下来两节将演示字符串表达式、方法和格式化——它们是 Python 语言中文本处理工具的第一梯队。

深度理解

- 核心概念：字符串"实战"分三块武器库：表达式（+、索引、切片等），方法（.find、.replace 等 43 个），以及格式化（% 表达式、.format 方法、f-string）。表达式与内置函数跨类型通用，方法则类型专属。
- 底层实现：三者各自拥有不同的执行路径——表达式大多是编译成字节码后由解释器直接执行（如 + 走 BINARYOP 指令），方法是在对象的方法表里查找并调用 C 函数或 Python 函数，格式化则由 str 的 format 协议驱动。
- 设计原因：把"通用结构操作"（拼接、取长度）与"字符

串专属操作" (改大小写、去空白) 分开, 避免了 C 标准库里 strlen/strcpy/strstr 这种"散装函数"的命名负担, 也让用户自定义类型能够复用序列协议。

·实际问题: 可读日志、URL 解析、数据清洗、模板生成——所有文本任务都由这三套工具组合完成。

·初学者误区: 把 .find 当 in 用 (返回值不同), 或把方法结果当就地修改 (字符串不可变, 必须接收返回值)。

7.4.1 基本操作 (Basic Operations)

英文原文

Let's begin by interacting with the Python interpreter to illustrate the basic string operations listed earlier in Table 7-1. You can concatenate strings using the + operator and repeat them using the operator:

```
python >> len('abc') # 长度: 元素个数 3 >> 'abc' + 'def'
# 拼接: 得到新字符串'abcdef' >> 'Py!' * 4 # 重复: 相当于
'Py!' + 'Py!' + 'Py!' + 'Py!' 'Py!Py!Py!Py!'
```

The len built-in function here returns the length of a string (or any other object with a length). Formally, adding two string objects with + creates a new string object, with the contents of its operands joined, and repetition with is like adding a string to itself a given number of times (minus one).

In both cases, Python lets you create arbitrarily sized strings; there's no need to predeclare anything in Pyt

hon, including the sizes of data structures—you simply build string objects as needed and let Python manage the underlying memory space automatically, as we learned in Chapter 6.

Repetition may seem a bit obscure at first, but it comes in handy in a surprising number of contexts. For example, to print a line of 80 dashes, you can count up to 80, or let Python count for you:

```
python >> print('-----') # 80  
个短横线: 笨办法 >> print('-' 80) # 80 个短横线: 简单  
办法
```

Notice that the operator overloading and polymorphism called out in Chapter 5 and earlier is at work here already: we're using the same + and operators that perform addition and multiplication when using numbers. Python does the correct operation because it knows the types of the objects being added and multiplied. But be careful: the rules aren't quite as liberal as you might expect.

For instance, Python doesn't allow you to mix numbers and strings in + expressions: 'abc'+9 raises an error instead of automatically converting 9 to a string (we'll fix this ahead).

As shown near the end of Table 7-1, you can also iterate over strings in loops using for statements, which repeat actions, and test membership for both characters and substrings with the in expression operator, w

hich is essentially a search.

For substrings, in is much like the str.find() method covered later in this chapter, but it returns a Boolean result instead of the substring's position (don't be alarmed if the following's print indents your prompt; its end=" " changes the default newline character at the end of the display to a space):

```
python >> myjob = 'hacker'>> for c in myjob: # 逐个取出元素，打印每个字符加空格
... print(c, end=" ") # 抑制每次打印后的换行... h a c k e r
>> 'k' in myjob # 找到了 True >> 'z' in myjob # 没找到 False
>> 'HACK' in 'abcHACKdef' # 子串搜索，不返回位置 True
```

The for loop, previewed in Chapter 4, assigns a variable to successive items in a sequence (here, a string) and executes one or more statements (normally indented) for each item. In effect, the variable `c` becomes a cursor stepping across the string's characters. Because iteration turns out to be a big idea in Python, we will discuss iteration tools like these and others listed in Table 7-1 in more detail later in this book (see Chapters 14 and 20).

中文翻译

让我们开始与 Python 解释器交互，演示表 7-1 前面列出的基本字符串操作。你可以用 + 运算符拼接 (concatenate) 字符串，用运算符重复 (repeat) 它们：python >>>

`len('abc')` # 长度：元素个数3 >>> `'abc' + 'def'` # 拼接：得到新字符串 `'abcdef'` >>> `'Py!' * 4` # 重复：相当于 `'Py!' + 'Py!' + 'Py!' + 'Py!'` `'Py!Py!Py!Py!'` 这里的 `len` 内置函数返回字符串（或任何有长度的对象）的长度。严格地说，用 `+` 把两个字符串对象相加会创建一个新字符串对象，操作数的内容被连在一起；用重复则类似于把字符串自身相加给定次数（再减一）。两种情况都允许你创建任意大小的字符串——在 Python 里无需预声明任何东西，包括数据结构的大小——你只需按需构建字符串对象，让 Python 自动管理底层内存空间，如第 6 章所学。重复操作一开始可能显得有点偏门，但它在多得出奇的场景中派得上用场。比如要打印一行 80 个短横线，你可以自己数到 80，也可以让 Python 帮你数：`python >>> print('-----`
`---')` # 80 个短横线：笨办法 >>> `print('-' * 80)` # 80 个短横线：简单办法注意，第 5 章及之前提到的运算符重载（operator overloading）和多态（polymorphism）已经在这里起作用：我们用的是同一对 `+` 和运算符，在数字上它们执行加法与乘法。Python 之所以选对操作，是因为它知道被加、被乘对象的类型。但要小心：规则并没有你想象得那么宽松。比如，Python 不允许在 `+` 表达式里混用数字和字符串：`'abc'+9` 会引发错误，而不是自动把 9 转成字符串（这个问题稍后我们会给出解决办法）。如表 7-1 末尾所示，你还可以用 `for` 语句在循环中迭代字符串（重复动作），并用 `in` 表达式运算符测试字符和子串的成员资格（membership）——`in` 本质是一次搜索。对子串而言，`in` 很像本章后面要讲的 `str.find()` 方法，但它返回布尔结果而不是子串的位置（如果下面 `print` 缩进了你的提示符，别惊慌：它的 `end=""` 把显示末尾默认的换行字符改成了空格）：`py`

```
thon >>> myjob = 'hacker'>>> for c in myjob: # 逐个
取出元素，打印每个字符加空格... print(c, end='') # 抑制
每次打印后的换行... h a c k e r >>> 'k' in myjob # 找到
了True >>> 'z' in myjob # 没找到False >>> 'HACK' in 'a
bcHACKdef' # 子串搜索，不返回位置True 第 4 章预览过的
for 循环，把变量依次赋给序列（这里是字符串）中的连续
元素，并为每个元素执行一条或多条（通常缩进的）语句
。实际上，变量 c 成为在字符串字符间步进的游标。由于迭
代在 Python 中被证明是一个大主题，本书后面（见第 14
、20 章）会更详细地讨论此类及表 7-1 列出的其他迭代工
具。
```

深度理解

- 核心概念：字符串是序列，因此继承序列的通用表达式：
+ 拼接、重复、len() 长度、for 迭代、in 成员测试。这些操作对字符串是“代工”——同一批运算符会在列表、元组上复现。
- 底层实现：'Py!' 4 编译为 BINARYOP，CPython 的字符串乘法在 C 层按新长度一次分配内存再复制（长度公式 $len(a) \cdot n$ ，重复 0 次或与原串拼接时走优化/常量化路径）；
+ 拼接同样新建对象（Python 3.11+ 对单元素拼接有 C 层内联优化；重复 += 在旧版本是 $O(n^2)$ ，这是后面要学“用 .join 拼字符串”的动机）。in 走 contains，本质是 $O(n)$ 的子串搜索。
- 设计原因：运算符属于“类型知道自己的加法是什么”——int.add、str.add 各自实现，解释器按左侧操作数的类型分发。这使新手不必记忆几十个拼接函数名。

·实际问题：分隔线、进度条的重复填充；逐字符迭代做字符级处理；用 `in` 做快速关键词/子串存在性检查（比 `.find()` 后判 `-1` 更直白）。

·初学者误区：(1) `'abc' + 9` 报错让新手错愕——Python 拒绝隐式类型转换（“宁缺毋滥”的设计哲学），必须 `'abc' + str(9)`；(2) 以为乘法只对数字有效；(3) 在循环里用 `print(c, end='')` 却发现 REPL 提示符被推到下一行——因为 `print` 不再以换行收尾。

代码分析

```
>>> myjob = 'hacker'
>>> for c in myjob:                # c h,a,c,k,e,r
...     print(c, end=' ')          # end=' ' '\n',
h a c k e r
>>> 'k' in myjob                   # __contains__ True
True
```

解释器如何处理：for 循环先对 `myjob` 调用 `iter()`，得到字符串迭代器（`striterator`），然后循环体反复调用迭代器的 `next` 取出下一个字符（每次取一个 Unicode 码点），直到 `StopIteration` 异常触发循环终止——这正是“游标步进”的底层本质。`print(c, end='')` 中 `end` 是关键字参数，取代默认 `end='\n'`，所以输出是一行空格分隔的 `h a c k e r`。`'k' in myjob` 调用 `str.contains`，内部用快速子串搜索（CPython 的 `unicodecontains`，长串走改进的搜索算法）返回 `True`。设计思想：`in` 测试与 `for` 迭代都是“把字符串当序列处理”，统一由迭代协议与容器协议支撑。

7.4.2 索引与切片 (Indexing and Slicing)

英文原文

Because strings are ordered collections (a.k.a. sequences) of characters, we can access their components by position. As introduced in Chapter 4, characters in a string are fetched by indexing—providing the numeric offset of the desired component in square brackets after the string. You get back the one-character string at the specified position.

As in most C-like languages, Python offsets start at 0 and end at one less than the length of the string (and the "start at 0" part may be a short-lived hurdle if you're accustomed to counting from 1). Unlike C, however, Python also lets you fetch items from sequences such as strings using negative offsets. Technically, a negative offset is added to the length of a string to derive a positive offset, but you can also think of negative offsets as counting backward from the end. The following interaction demonstrates:

```
python >> S='code'>> S[0], S[-2] # 从前或从后索引 ('c','d')>> S[1:3], S[1:], S[:-1] # 切片: 提取一段 ('od','ode','cod')
```

In this code, the first line defines a four-character string and assigns it to the name `S`. The next line indexes it in two ways: `S[0]` fetches the item at offset 0 from the left—the one-character string `'c'` at the front; and

`S[-2]` gets the item at offset 2 back from the end—or equivalently, at offset $(4 + (-2))$ from the front.

The last line in the foregoing example demonstrates slicing, a generalized form of indexing that returns an entire section, instead of a single item. It can be used to extract columns of data, chop off prefixes and suffixes, and more. Slicing can also be viewed as a type of parsing (decomposing content), especially when applied to strings, because it's an easy way to extract substrings. In fact, we'll explore slicing in the context of text parsing later in this chapter.

Slicing works like this: when you index a sequence object such as a string on a pair of offsets separated by a colon, Python returns a new object containing the contiguous section identified by the offset pair. The left offset is taken to be the lower bound (inclusive), and the right is the upper bound (noninclusive). That is, Python fetches all items from the lower bound up to but not including the upper bound and returns a new object containing the fetched items. If omitted, the left and right bounds default to 0 and the length of the object you are slicing, respectively.

For instance, in the example we just ran, `S[1:3]` extracts the items at offsets 1 and 2—it grabs the second and third items and stops before the fourth item at offset 3. Next, `S[1:]` gets all items beyond the first—the upper bound, which is not specified, defaults to the l

length of the string, which is off the end. Finally, `S[:-1]` fetches all but the last item—the lower bound defaults to 0, and `-1` refers to the last item, noninclusive. In more graphic terms, indexes and slices map to cells as shown in Figure 7-1.

Figure 7-1. Indexes and slices: positives start from the left (0) and negatives from the right (-1). (本图为插图：字符串 code 的每个字符下方标注正负两种下标；示意图)：

	字符	c	o	d	e		---	---	---	---	---		正下标
0		1		2		3		4		5		6	
	负下标	-4		-3		-2		-1		0		-5	

All of which may seem confusing at first glance, but indexing and slicing are simple and powerful tools to use once you get the knack. Remember, if you're unsure about the effects of a slice, try it out interactively. In the next chapter, you'll see that it's even possible to change an entire section of another object in one step by assigning to a slice (though not for immutables like the strings we're studying here). For now, here's a cheat sheet of the details for reference:

Indexing — `S[l]` — fetches components at offsets in sequences:

- The first item is at offset 0.
- Negative indexes mean counting backward from the end or right.
- Out-of-bounds offsets are an error.
- `S[0]` fetches the first item.
- `S[-2]` fetches the second item from the end (like `S[len(S)-2]`).

Slicing — `S[l:j]` — extracts contiguous sections of sequences:

- The upper bound is noninclusive.
- Slice bounds default to 0 and the sequence length, if omitted.
- Out-of-bounds offsets are adjusted to be in bounds.

ds. - `S[1:3]` fetches items at offsets 1 up to but not including 3. - `S[1:]` fetches items at offset 1 through the end (the sequence length). - `S[:3]` fetches items at offset 0 up to but not including 3. - `S[:-1]` fetches items at offset 0 up to but not including the last item. - `S[:]` fetches items at offsets 0 through the end—making a top-level copy of `S`.

Extended slicing — `S[l:j:k]` — accepts a step (or stride) `k`, which defaults to `+1`: - Allows for skipping items and reversing order—see the next section.

The second-to-last bullet item listed here turns out to be a common technique: `S[:]` makes a full top-level copy of a sequence object—an object with the same value, but a distinct piece of memory (you'll find more on copies in Chapter 9). This isn't very useful for immutable objects like strings, but it comes in handy for objects that may be changed in place, such as lists: making a copy can avoid the side effects of shared references shown in Chapter 6.

Also per the cheat sheet, slices differ from indexes in their policy on out-of-bounds (off the end) offsets: they're always errors in indexing, because the offset does not exist, but scaled to be in bounds in slicing, because this can be useful in programs that need to accommodate sizes flexibly:

```
python >> S = 'code'>> S[99] IndexError: string index out of range >> S[1:99]'ode'
```

Because we're going to explore this oddity in an end-of-part exercise, though, we'll cut the story short here.

中文翻译

因为字符串是字符的有序集合（即序列），我们可以按位置访问其组成部分。如第 4 章介绍，字符串中字符通过索引（indexing）获取——在字符串后的方括号里提供所需分量的数字偏移量，返回指定位置的那个单字符字符串。与大多数 C 系语言一样，Python 的偏移从 0 开始，到字符串长度减一结束（如果你习惯从 1 数起，“从 0 开始”这关可能短暂卡一下）。然而，与 C 不同，Python 还允许你用负偏移（negative offset）从字符串这样的序列中取元素。严格讲，负偏移会与字符串长度相加得出一个正偏移，但你也可以把负偏移理解为“从末尾往回数”。下面这段交互演示了这一点：

```
python >>> S = 'code' >>> S[0], S[-2] # 从前或从后索引('c','d') >>> S[1:3], S[1:], S[:-1] # 切片：提取一段('od','ode','cod')
```

这段代码中，第一行定义一个四字字符串并赋给名字 S。下一行用两种方式索引它：S[0] 从左边取偏移 0 处的元素——最前面的单字符字符串 'c'；S[-2] 取末尾往回两个偏移处的元素——等价于从前数 $(4 + (-2))$ 偏移处。上面例子中最后一行演示切片（slicing），它是索引的推广形式，返回一个完整的片段，而不是单个元素。切片可用来提取数据列、切除前后缀等等。切片也可以看作一种解析（parsing，分解内容）——尤其用于字符串时，因为它提取子串非常容易。事实上，我们在本章后面还会在文本解析的背景下探索切片。切片的工作原理：当你用一对冒号分隔的偏移索引字符串这样的序列对象时，Python

返回一个新对象，包含这一对偏移标识出的连续片段。左偏移被视为下界（含），右偏移是上界（不含）。也就是说，Python 取下界起、到上界为止（不含上界）的所有元素，装进一个新对象返回。若省略，左、右边界分别默认为 0 和你所切对象的长度。例如在我们刚运行过的例子里，`S[1:3]` 提取偏移 1 和 2 的元素——取走第二、第三个元素，在偏移 3 的第四个元素之前停下。接着，`S[1:]` 取第一个之后的所有元素——未指定的上界默认是字符串长度，即越出末尾。最后，`S[:-1]` 取除最后一个外的所有元素——下界默认 0，-1 指最后一个元素（不含）。用更形象的话说，索引与切片映射到格子如插图 7-1 所示。图 7-1（文字还原）：字符串 'code' 每个字符下的正负下标：

字符	c	o	d	e
正下标	0	1	2	3
负下标	-4	-3	-2	-1

这一切乍看可能令人困惑，但一旦上手，索引和切片就是简单又强大的工具。记住：如果不确定某个切片的效果，就在交互环境里试试。下一章你会看到，甚至可以通过对切片赋值一步改变另一个对象的整个片段（不过对我们这里研究的字符串这类不可变对象不行）。现在把细节整理成速查表备用：

- 索引——`S[i]`——取序列中偏移动处的元素：
 - 第一个元素在偏移 0。
 - 负索引表示从末尾或右边往回数。
 - 越界偏移是错误。
 - `S[0]` 取第一个元素。
 - `S[-2]` 取倒数第二个元素（相当于 `S[len(S)-2]`）。
- 切片——`S[l:j]`——提取序列的连续片段：
 - 上界不含（noninclusive）。
 - 切片边界省略时默认为 0 和序列长度。
 - 越界偏移会被调整到界内。
 - `S[1:3]` 取偏移 1 起到（不含）3 的元素。
 - `S[1:]` 取偏移 1 起到末尾（序列长度）的元素。
 - `S[:3]` 取偏移 0 起到（不含）3 的元素。
 - `S[:-1]` 取偏移 0 起到（不含）最后一个元素。
 - `S[:]` 取偏移 0 到尾——生成 S

的顶层副本。扩展切片——`S[l:j:k]`——接受步长 (step, 又称 stride) `K`, 默认 `+1`: - 允许跳过元素和反转顺序——见下一节。上面倒数第二条是个常用技巧: `S[:]` 生成序列对象的完整顶层副本 (copy) ——值相同但内存独立的对象 (第 9 章会详细讲副本)。对字符串这类不可变对象用处不大, 但对可原地修改的对象 (如列表) 很有用: 复制可以避免第 6 章展示的共享引用副作用。同样按速查表, 切片和索引在越界 (超出末尾) 偏移上的策略不同: 索引遇越界总是错误, 因为该偏移不存在; 而切片会把越界按比例收缩到界内, 因为这对需要灵活容纳大小的程序很有用: `python >>> S = 'code'>>> S[99] IndexError: string index out of range >>> S[1:99]'ode'` 不过, 因为我们要在一部分末尾的练习中探讨这个怪癖, 这里就先不展开了。

深度理解

- 核心概念: 索引取单个元素 (`S[i]`), 切片取连续片段 (`S[l:j]`, 左含右不含), 扩展切片加步长 (`S[l:j:k]`)。这是序列类型三大支柱里的结构提取支柱。
- 底层实现: 切片本质是把 `l:j:k` 封装成一个 slice 对象, 再调用序列的 `getitem(sliceobject)`。CPython 在 C 层先规范化偏移 (处理负数: 加长度), 对不可变 str 则按切片长度新建一个字符串并复制字符 (PyUnicodeSubstring), 因此 `S[1:99]` 中 99 被安全钳制到 4。负偏移 `S[-2]` 等价于 `S[len(S)-2]`——这是解释器执行的真正变换。
- 设计原因: "左含右不含" (half-open interval) 的约定使 `S[:i] + S[i:] == S` 恒成立, 也便于指定"去掉第 *i* 个"这类逻辑; 越界"索引报错、切片容忍"是两条不同契约: 前

者保护你不越界（C 里越界是未定义行为），后者让解析代码不必反复算哨兵长度。

·实际问题：line[:-1] 剥行尾换行、sys.argv[1:] 去掉程序名、data[:3]/data[-3:] 切定长列、s[:] 复制（对可变序列防共享引用副作用）。

·初学者误区：(1) 以为 S[1:3] 含下标 3——不含，结果是两个字符；(2) 想取"倒数第 i 个"却写 S[-0]——-0 == 0，取的是第一个；(3) 以为字符串能用下标赋值去改——不可变，会抛 TypeError；(4) 混淆 in（测试）、[:]（复制）与 find（返回位置）。

代码分析

```
>>> S = 'code'
>>> S[0], S[-2]          #
('c', 'd')
>>> S[1:3], S[1:], S[:-1] #
('od', 'ode', 'cod')
```

解释器如何处理：S[0] 翻译为 BINARYSUBSCR 指令，对整数参数走 str.getitem(0)，返回一个字符的字符串子串。S[-2] 中解释器先把 -2 加长度 4 得正偏移 2，再取'd'——所以 -2 与 2 都指向同一个底层格子。S[1:3] 翻译时构造 slice(1, 3) 对象，调用 getitem；CPython 规范化后取下标 1、2（不含 3）复制成 2 字符新串。S[1:] 的右边界用 None 填充然后代之以长度 4；S[:-1] 左边界 None 代 0，右边界 -1 加 4 得 3、再不含——等价于取 0..2 即'cod'。设计思想：切片的所有边界在解释器层统一换算成"绝对半开区间"，所以穷举 [0]、[:], [len:] 这些边缘情

7.4.3 扩展切片：第三个界限与 slice 对象 (Extended Slicing)

英文原文

Though not commonly used, slice expressions also support an optional third index, used as a step (sometimes called a stride). The step is added to the index of each item extracted.

With it, the full-blown form of a slice is `S[l:j:k]`, which means "extract all the items in `S`, from offset `l` through `j-1`, by `k`." The third limit, `k`, defaults to `+1`, which is why normally all items in a slice are extracted from left to right. If you specify an explicit value, however, you can use the third limit to skip items or to reverse their order.

For instance, `S[1:10:2]` will fetch every other item in `S` from offsets 1–9; that is, it will collect the items at offsets 1, 3, 5, 7, and 9. As usual, the first and second limits default to 0 and the length of the sequence, respectively, so `X[:2]` gets every other item from the beginning to the end of the sequence:

```
python >> S = 'abcdefghijklmnop'>> S[1:10:2] # 跳过  
元素'bdfhj'>> S[:2]'acegikmo'
```

You can also use a negative stride to collect items in the opposite order. For example, in the slicing expression `S[::-1]`, the first two bounds default to sequence length-1 and -1 (they really default to None and None, but that's unimportant here), and a stride of -1 indicates that the slice should go from right to left instead of the usual left to right. In much simpler terms, the effect is to reverse the sequence:

```
python >> S = 'hello'>> S[::-1] # 反转元素'olleh'
```

With a negative stride, the meanings of the first two bounds are essentially reversed. That is, the slice `S[5:1:-1]` fetches the items from 2 to 5, in reverse order (the result contains items from offsets 5, 4, 3, and 2):

```
python >> S = 'abcedfg'>> S[5:1:-1] # 两个边界的角色  
互换'fdec'
```

Skipping and reversing like this are the most common use cases for three-limit slices, but see Python's standard-library manual for more details (or run a few experiments interactively). We'll revisit three-limit slices again later in this book, in conjunction with the `for` loop statement.

Later in the book, you'll also learn that slicing is equivalent to indexing with a slice object, a finding of importance to class writers seeking to support both operations:

```
python >> 'code'[1:3] # 切片语法'od'>> 'code'[slice(1,
```



```
3)] # 切片对象 + 索引语法'od'>>'code'[::-1]'edoc'>>'code'[slice(None, None, -1)]'edoc'
```

中文翻译

虽然不常用，但切片表达式还支持可选的第三个索引，用作步长（step）（有时称步伐/stride）。步长会加到每个被取元素的索引上。有了它，切片的最完整形式是 $S[l:j:k]$ ，意思是“取出 S 中从偏移 l 到 $j-1$ 、以步长 k 行进的所有元素。”第三个界限 k 默认为 $+1$ ，这就是为什么普通切片默认从左到右取出所有元素。不过，一旦你显式指定值，就可以用第三个界限来跳过元素或反转顺序。例如， $S[1:10:2]$ 将取 S 中偏移 1–9 里的每隔一个元素；也就是说，它收集偏移 1、3、5、7、9 处的元素。如常，第一、第二个界限分别默认 0 和序列长度，因此 $X[::2]$ 从序列开头到结尾取隔一个元素：
python >>> $S = \text{'abcdefghijklmnp'}$ >>> $S[1:10:2]$ # 跳过元素'bdfhj'>>> $S[::2]$ 'acegikmo'你也可以用负步长按相反顺序收集元素。例如，表达式 $S[::-1]$ 中，前两个边界默认是序列长度减一和 -1 （其实它们真的默认是 None 和 None ，但这里无关紧要），步长 -1 表明切片应自右向左而非通常的自左向右。用更简单的说法：其效果是反转（reverse）序列：
python >>> $S = \text{'hello'}$ >>> $S[::-1]$ # 反转元素'olleh'当步长为负时，前两个边界的含义基本上互换。也就是说，切片 $S[5:1:-1]$ 取偏移 2 到 5、按相反顺序的元素（结果含偏移 5、4、3、2 的元素）：
python >>> $S = \text{'abcedfg'}$ >>> $S[5:1:-1]$ # 两个边界的角色互换'fdec'像这样跳过和反转是三界切片最常见的用例，更多细节参见 Python 标准库手册（或交互式多跑几个

实验)。本书后面还会结合 for 循环语句再次讨论三界限切片。在后面你还会学到：切片等价于用切片对象 (slice object) 做索引，这一点对想同时支持两种操作的类作者很重要：
`python >>>'code'[1:3] # 切片语法'od'>>>'code'[slice(1, 3)] # 切片对象 + 索引语法'od'>>>'code'[::-1]'edoc'>>>'code'[slice(None, None, -1)]'edoc'`

深度理解

·核心概念：三个界限 `[I:J:K]` = "从 I 到 J，每 K 步取一个"。步长为负则从右向左走，且起始/结束边界角色互换；省略的边界语义也随负步长调整。

·底层实现：`[::-1]` 中冒号处省略的 None 组成 `slice(None, None, -1)`。CPython 的切片归一化逻辑 (`PySliceAdjustIndices`) 在负步长下把"缺省起点"解释为 `len-1`、缺省终点为 `-1` (含遍历到 0 号元素)，按公式 `start + nk` 生成下标序列。这就是 `'hello'[::-1]` 得到 `'olleh'` 的引擎。`slice()` 内置类型让你能在运行时构造等价切片，动态传给 `getitem`。

·设计原因：步长让"隔行取列/倒序"这类高频需求免去手写循环；把 `I:J:K` 提升为一等对象 (slice object) 则让自定义类能统一接收三界限语义。

·实际问题：`s[::-1]` 是最常用的字符串反转惯用法 (面试常考)；字符串 `[:2]` 提取规则间隔字符 (如身份证、银行账号排版校验)；`text.split()[::-1]` 反转单词顺序。

·初学者误区：(1) 以为 `S[::-1]` 是专设的反转语法——其实它只是步长 `-1` 的特例；(2) 负步长时边界记反——`S[5:1:`

-1] 含 5、不含 1、取'fdec'（原串 abcdefg 的 5..3 位置）；(3) 以为负步长对所有序列都高效——对 list 也成立，但对非随机访问类型语义不同。

代码分析

```
>>> 'code'[1:3]           #  
'od'  
>>> 'code'[slice(1, 3)]   # slice  
'od'  
>>> 'code'[::-1]         # -1  
'edoc'
```

解释器如何处理：S[l:j:k] 语法在编译时被改写成 S.getitem(slice(l, j, k))——切片对象是幕后的一等对象。[::-1] 实际是 slice(None, None, -1)：解释器在步长 -1 时调整索引界面为 start=len-1、stop=-1、步进 -1，于是从末尾倒着收集每个字符并组装新字符串'edoc'。对字符串而言 getitem 收到 slice 后调 PyUnicodeSubstring/步长版 PyUnicodeExtract，一次性分配结果内存。设计思想：切片表达式与服务端（序列）通过 slice 对象解耦，因此你在自定义类里也能自由决定"切成什么"。

7.4.4 为什么你会在乎：切片 (WHY YOU WILL CARE: SLICES)

英文原文

Throughout this book, you'll meet common use-case sidebars such as this one that give you a peek at how some of the language features being discussed are ty

pically used in real programs. Because you won't be able to make much sense of realistic use cases until you've seen more of the Python picture, these sidebars necessarily contain many references to topics not introduced yet; at most, you should consider them previews of ways that you may find these abstract language concepts useful for practical programming tasks. For instance, you'll see later that the argument words listed on a system command line used to launch a Python program are made available in the `argv` attribute of the built-in `sys` module: `python # 文件 echo.py import sys print(sys.argv) text $ python3 echo.py -a -b -c ['echo.py', '-a', '-b', '-c']` Usually, you're interested only in inspecting the arguments that follow the program name. This leads to a typical application of slices: a single slice expression can be used to return all but the first item of a list. Here, `sys.argv[1:]` returns the desired list, `['-a', '-b', '-c']`. You can then process this list without having to accommodate the program name at the front. Slices are also often used to clean up lines read from input files, of the sort we'll study in Chapter 9. If you know that a line will have a newline character at the end (a `\n`), you can get rid of it with a single expression such as `line[:-1]`, which extracts all but the last character in the line. In both cases, slices do the job of logic that must be explicit in a lower-level language. Having said that, calling the `line.rstrip` method is often preferred for stripping newline characters.

cters because this call leaves the line intact if it has no newline character at the end—a common case for files created with some text-editing tools. Slicing works only if you're sure the line is properly terminated.

中文翻译

贯穿全书，你会遇到像这样的“常见用例”侧边栏，让你一窥目前讨论的语言特性在实际程序中通常怎么用。因为在你见识更多 Python 全貌之前，很难理解真实用例的意义，这些侧边栏必然大量引用尚未介绍的主题；充其量，你应把它们当作“你可能发现这些抽象语言概念对实用编程任务有帮助”的预告。例如，你稍后会看到：启动 Python 程序时命令行上列出的参数字符串，都放在内置 sys 模块的 argv 属性中可查：`python # 文件 echo.py import sys print(sys.argv)` `text $ python3 echo.py -a -b -c ['echo.py','-a','-b','-c']` 通常，你只关心程序名之后的参数。这就引出了切片的典型应用：一个简单的切片表达式就能返回列表除第一项外的所有内容。这里 `sys.argv[1:]` 返回想要的列表 `['-a','-b','-c']`，之后你就可以处理这个列表，而不必顾及最前面的程序名。切片还常用于清理从输入文件读出的行，这类内容第 9 章会研究。如果你确定某行末尾有换行字符（`\n`），一条表达式如 `line[:-1]` 就能把它去掉——它取出该行除最后一个字符外的全部内容。两种情况下，切片都完成了在低级语言里必须显式编写的逻辑。话虽如此，剥换行时人们通常更偏好调用 `line.rstrip` 方法，因为如果行尾没有换行字符（某些文本编辑器创建的文件常常如此），这个调用能让行保持原样。切片只在你能确定行正确地以换行结尾时才可靠。

深度理解

- 核心概念：切片是把"枚举逻辑"压缩成一条表达式的抽象——`sys.argv[1:]` 替掉"写循环跳过头元素"，`line[:-1]` 替掉"读长度减一"的尾部判断。
- 底层实现：`sys.argv` 是命令行文本经解释器启动流程解析成的字符串列表；`sys.argv[1:]` 走 list 的 `getitem(slice)`，C 层复制尾段元素构造新列表。所谓"低级语言要做的事"，指 C 里你得写 `for (i=1;i<argc;i++)` 加下标管理。
- 设计原因：切片把"集合的局部视图"做成语法级操作，减少边界错误的藏身之处；`.rstrip` 与切片的取舍反映了另一个设计准则：切片承诺"精确位置"，方法承诺"容错处理"。
- 实际问题：写命令行工具、读配置文件按行清理、日志分析去掉时间戳前缀——全是切片整套操作的高频现场。
- 初学者误区：把 `line[:-1]` 当"安全去换行"——对以 `\n` 结尾的行 OK，对末尾没有换行的行会误删最后一个字符；处处用 `.rstrip()` 又无法表达"精确去掉第 n 位"。工具与场景要匹配。

代码分析

```
import sys
print(sys.argv)          # argv
$ python3 echo.py -a -b -c
['echo.py', '-a', '-b', '-c'] # argv[0]
sys.argv[1:]              # ['-a', '-b', '-c']
```

解释器如何处理：CPython 启动时 (`PySysSetArgv`) 把

OS 提供的 `argv` 转成一个 Python 字符串列表绑定为 `sys.argv`，首元素是脚本路径。运行时 `sys.argv[1:]` 触发 list 切片，复制除首元素外的剩余部分为新列表。这与 C 语言 `argv[1..argc-1]` 的语义一致，但切片把"从下标 1 到末尾"的区间描述直接放进语法。设计思想：把常见的"跳过第一个"编程需求固化为一条表达式，reduce 掉循环样板。

7.4.5 字符串转换工具 (String Conversion Tools)

英文原文

One of Python's design mottos is that it refuses the temptation to guess. As a prime example, you cannot add a number and a string together in Python, even if the string looks like a number (i.e., is all digits):

```
python >> '62' + 1
TypeError: can only concatenate str (not "int") to str
```

This is by design: because `+` can mean both addition and concatenation, the choice of conversion would be ambiguous—do you want `'621'` or `63`? Instead, Python treats this as an error. In Python, magic is generally omitted if it will make your coding life more complex.

What to do, then, if your script obtains a number as a text string from a file or user interface? The trick is th

at you must simply employ conversion tools before you can treat a string like a number, or vice versa. For instance:

```
python >> int('62'), str(62) # 从字符串转换 / 转换到字符串 (62,'62')
```

The `int` function converts a string to a number, and the `str` function converts a number to its string representation (essentially, what it looks like when printed). Now, although you can't mix strings and number types around operators such as `+`, you can manually convert operands before that operation if needed:

```
python >> S='62'>> I = 1 >> S + I TypeError: can only concatenate str (not "int") to str >> int(S) + I # 强制做加法 63 >> S + str(I) # 强制做拼接 '621'
```

Similar built-in functions handle floating-point-number conversions to and from strings, if you need to mix the two in expressions:

```
python >> float('1.5') + 2.8 4.3 >> '1.5' + str(2.8) '1.52.8'
```

The built-in `eval` function introduced in Chapter 5 runs a string containing Python expression code, and so can also convert a string to any kind of object. The functions `int` and `float` convert only to numbers, but this restriction means they are usually faster (and more secure, because they do not accept arbitrary expression code). As we also saw briefly in Chapter 5, string

formatting provides other ways to convert numbers to strings; more on it ahead.

中文翻译

Python 的设计箴言之一是“拒绝猜测的诱惑 (refuses the temptation to guess)”。一个典型例子：你不能在 Python 里把数字和字符串相加，哪怕这个字符串看起来像数字（即全是数字）：`python >>> '62' + 1` `TypeError: can only concatenate str (not "int") to str` 这是有意为之：因为 `+` 既可能是加法也可能是拼接，转换的选择就会有歧义——你到底想要 `'621'` 还是 `63`？于是 Python 把它当作错误处理。在 Python 里，如果魔法会让你的编码生活更复杂，它通常就被省略。那么，如果你的脚本从文件或用户界面拿到一个以文本字符串形式存在的数字，该怎么办？诀窍是：你必须先使用转换工具，才能把字符串当数字用（或反之）。例如：`python >>> int('62'), str(62)` # 从字符串转换 / 转换到字符串 `(62, '62')` `int` 函数把字符串转成数字，`str` 函数把数字转成它的字符串表示（本质上就是打印出来时的样子）。现在，虽然你不能在 `+` 这类运算符周围混用字符串和数字类型，但需要时可以在运算前手动转换操作数：`python >>> S = '62' >>> I = 1 >>> S + I` `TypeError: can only concatenate str (not "int") to str` `>>> int(S) + I` # 强制做加法 `63 >>> S + str(I)` # 强制做拼接 `'621'` 类似的几个内置函数处理浮点数与字符串的双向转换，如果你需要在表达式中混用两者：`python >>> float('1.5') + 2.8` `4.3 >>> '1.5' + str(2.8)` `'1.52.8'` 第 5 章介绍的内置 `eval` 函数会运行包含 Python 表达式代码的字符串，因此可以把字符串转换成任意种类的对象。`int` 和 `float` 只能转成数字，但这

一限制意味着它们通常更快、也更安全（因为它们不接受任意表达式代码）。如第 5 章也简略提过，字符串格式化提供了把数字转成字符串的其他途径；后面会详述。

深度理解

·核心概念：Python 坚持显式转换（explicit conversion）——类型间不自动猜。字符串数字的桥梁是 `int()/float()/str()`；`eval` 是“万能却危险”的进阶转换器。

·底层实现：`int('62')` 在 C 层解析数字串（支持 +/-、下划线、进制前缀），溢出时抛 `OverflowError`，非法输入抛 `ValueError` (`int('abc')`)。`str(62)` 调用数字的 `str` 生成十进制表示。`str + int` 触发类型检查失败抛 `TypeError`——+ 的分派先查 `add` 类型兼容性。`eval` 会真正编译并执行任意表达式，因此可构造任何对象，也意味着可执行任意代码。

·设计原因：“拒绝猜测”消除 `'62'+1` 到底是 63 还是 `'621'` 的歧义，避免隐式转换掩盖 bug——动态类型语言里隐式转换是隐蔽错误的温床。安全上，`int/float` 是受控解析，`eval` 是任意执行。

·实际问题：读 CSV/表单/API 返回的字符串数字做计算（必须先 `float/int`）；把数字拼进报错信息（必须 `str`）。

·初学者误区：(1) 直接 `'1.5' + str(2.8)` 得到 `'1.52.8'`——这是字符串拼接不是加法；(2) 以为 `int` 能容忍浮点字符串——`int('1.5')` 抛 `ValueError`，得用 `float`；(3) 在不可信输入上滥用 `eval`（SQL 注入式风险），首选 `int/float`。

代码分析

```
>>> int('62'), str(62)    # -> ->
(62, '62')
>>> int(S) + I            # S
63
>>> S + str(I)            # I
'621'
```

解释器如何处理：int('62') 调 C 的 PyLongFromString，跳过空白、检查符号、逐位累乘 10 并溢出检测，返回 PyLong；str(62) 调 PyLongType.tpstr，按预分配缓冲写入 ASCII 数字，返回 PyUnicode。int(S)+I 两次取值，解释器先执行 int(S) 产生 PyLong 62，随之 BINARYOP + 两操作数类型匹配（均为 int）走整数加法得 63。S + str(I) 同理，str(I) 生成'1' 后 + 走字符串拼接。设计思想：任何运算符都不参与类型换算——类型换算只由显式函数负责，运算符只做"同型运算"。这就是 Python 与 Perl 等"自动强转"语言的重大分野。

7.4.6 字符码转换 (Character-code Conversions)

英文原文

On the subject of conversions, it is also possible to convert a single character to its underlying integer code by passing it to the built-in ord function—this returns the numeric "ordinal" value used to represent the corresponding character in memory (technically, its

Unicode code point, as you'll learn in Chapter 37, but this isn't crucial yet). The `chr` function performs the inverse operation, taking an integer code and converting it to the corresponding character:

```
python >> ord('h') # 字符 => 码点编号 104 >> chr(104) # 编号 => 字符 (字符串) 'h' >> for c in 'hack': # 打印字符串中每个字符的码点
... print(c, ord(c)) ... h 104 a 97 c 99 k 107
```

You can use a loop to apply `ord` to all characters in a string as shown, but these tools can also be used to perform a simple sort of string-based math. To advance to the next character, for example, convert and do the math in integer:

```
python >> S = '5' >> S = chr(ord(S) + 1) >> S '6' >> S = chr(ord(S) + 1) >> S '7'
```

At least for single-character strings, this provides an alternative to using the built-in `int` function to convert from string to integer (though this only makes sense if character ordinals are ordered as your code expects!):

```
python >> int('5') 5 >> ord('5') - ord('0') 5
```

中文翻译

说到转换，还可以把单个字符转成它在底层对应的整数码：把它传给内置 `ord` 函数——它返回用于在内存中表示该字

符的数值"序数 (ordinal) " (严格说, 是它的 Unicode 码点, 详见第 37 章, 但眼下这不重要)。chr 函数执行逆操作: 接收整数码, 转成对应字符: `python >>> ord('h') # 字符 => 码点编号104 >>> chr(104) # 编号 => 字符 (字符串) 'h'>>> for c in 'hack': # 打印串中每个字符的码点..`
`. print(c, ord(c)) ... h 104 a 97 c 99 k 107` 如上所示, 你可以用一个循环把 ord 应用到字符串的每个字符上, 但这些工具也可以进行一种简单的"基于字符串的数学"。例如, 要前进到下一个字符, 先转换、再在整数域做数学: `python >>> S = '5'>>> S = chr(ord(S) + 1) >>> S'6'>>> S = chr(ord(S) + 1) >>> S'7'`至少对单字符字符串而言, 这提供了用内置 int 做字符串转整数的替代方案 (不过, 只有当字符序数的排布正如你代码所预期时, 这才有意义!) :
`python >>> int('5') 5 >>> ord('5') - ord('0') 5`

深度理解

·核心概念: ord/chr 是字符与其 Unicode 码点的双向桥。ord('h') 得 104 (ASCII 中'h'的码点), chr(104) 还原'h'。

·底层实现: ord 对长度为 1 的 str 返回内部存储的第一个码点 (PyUnicodeReadChar 或从紧凑表示直接读取); chr 把整数码点编码进新的 PyUnicode 对象 (超范围抛 ValueError) 。

·设计原因: 把"字符的下一个字符"等数字刻度数学 (如 Excel 列号 A→B 递增、密码穷举序号) 建立在码点数值上, 既统一又简单; 对比手写映射表, ord/chr 是内建且跨 Unicode 的。

·实际问题：小写转大写下一位、`'\t'/'\n'` 之类控制码判断、进制与校验和运算、加密散列的字符运算。`ord('5')-ord('0')` 是经典的"字符数字转数值"手法（源自 C 风格）。

·初学者误区：(1) `ord` 只接受长度恰为 1 的字符串，`ord('ab')` 抛 `TypeError`；(2) 想在"字母循环"里 `chr(ord('z')+1)` ——得到的不是 '{' 就是越界，字符序不是"字母表循环"；(3) 以为码点 == 字节值——ASCII 区两者重合，但 `ord('Ä')` 是 196，与 UTF-8 里的两字节截然不同。

代码分析

```
>>> S = '5'
>>> S = chr(ord(S) + 1)    # '5' -> 53 -> +1 -> 54 -> '6'
>>> S
'6'
>>> ord('5') - ord('0')    # 53 - 48 = 5
5
```

解释器如何处理：第一次 `ord('5')` 得整数 53（码点），加 1 得 54，`chr(54)` 构造单字符 '6'（ASCII 中数字 0–9 连续：'0'=48 ... '9'=57）。于是 `ord('5')-ord('0')` 恰好等于 5，与 `int('5')` 殊途同归——区别：前者基于字符编码的位置计算、只对"连续排序"的字符成立，后者走真正的数字解析（允许空白、符号、进制前缀）。设计思想：`ord+chr` 提供的是"编码刻度"视角，`int` 提供的是"数值语义"视角，两套工具按需选用。

7.4.7 字符串比较 (String Comparisons)

英文原文

Another reason for introducing ordinals here is that it helps us understand string comparisons: when we compare two text strings, Python automatically compares them left to right, character by character, and lexicographically—that is, by the same character code-point values returned by `ord`—until the first mismatch or end of either string. In the following, for example, the code point of `t` is greater than that of `k`, and the longer string at the end wins: `python >>> 'hack' == 'hack', 'hact' > 'hack', 'hacker' > 'hack' (True, True, True)` The same holds true for the byte strings you'll meet in Chapter 37 (they are compared byte for byte until a result is known), and the next chapter's richer collections like lists do similar (Python compares all their parts for you).

中文翻译

在这里引入序数的另一个原因，是它帮助我们理解字符串比较（string comparison）：比较两个文本字符串时，Python 会自动从左到右逐字符按字典序（lexicographically）比较——也就是按 `ord` 返回的那些字符码点值比较——直到出现第一个不同，或到达任一字符串的末尾。例如下面：`t` 的码点大于 `k` 的码点，而结尾较长的字符串获胜：`python >>> 'hack' == 'hack', 'hact' > 'hack', 'hacker' > 'hack' (True, True, True)`

深度理解

·核心概念：字符串比较 = 逐字符比较码点的字典序算法（类似词典编排）。'hack' 与前缀'hacker'：前 4 字符全同 → 较短者"先完"，较短者小于较长者，所以'hacker' > 'hack' 为 True。

·底层实现：CPython 对等长同类型做快速 memcmp (PyUnicodeEqual 先比较长度再比较内容)；对逐位比较先找公共前缀长度再定位首个差异码点 (unicodectcompare)。'hact' > 'hack'：公共前缀'hac'，第 4 位't'(116) vs 'k'(107)，116 > 107 → True。

·设计原因：码点序 × 字典序是"可计算、可预期、跨语言一致"的排序基准，也成为排序 (sort) 与集合去重的基础；与 C 的 strcmp 一致，但免去手写循环与指针。

·实际问题：字符串排序 (sorted(names))、关键词区间判断、按版本号/ID 前缀分区（注意：字典序 ≠ 数值序，"10" < "9"）。

·初学者误区：(1) 用 < 比版本号'3.12' vs '3.9' 会得出奇怪结果（数字作字符比）；(2) 以为比较是"长度优先"——是"首个差异优先"，'b' > 'aaaaaa' 恒真；(3) 混淆 ==（值相等）与 is（同一对象）；(4) 对中文等非 ASCII 文本，码点序 ≠ 用户认知的拼音/笔画序（那是 locale/标准化排序的领域）。

代码分析


```
>>> 'hack' == 'hack'      # => True
True
>>> 'hact' > 'hack'       # 'hac''t'(116) > 'k'(107)
True
>>> 'hacker' > 'hack'     #
True
```

解释器如何处理：三个比较都编译为 COMPAREOP == / >。== 先比长度（不等立即 False），长度相等时 C 层快速内容比较。> 走 unicodecompare：逐码点取出比较，直到 memcmp 命中差异或一方耗尽；'hact' 第 4 位确定胜出后立即返回 True，后面的字符不再看。'hacker' vs 'hack' 无差异但前者更长——“较短是前缀的则较短者更小”。这就是列表、元组递归比较在字符串层的实现基础。

7.5 修改字符串（第一部分）：序列操作 ("Changing" Strings Part 1: Sequence Operations)

英文原文

Remember the term immutable sequence? As we've seen, being a sequence means that strings support operations like concatenation, repetition, indexing, and slicing. The immutable part means that you cannot change a string in place—for instance, by assigning to an index:

```
python >>> S = 'text'>>> S[0] = 'n'
TypeError: 'str' object does not support item assignment
```

How to modify textual information in Python, then? To change a string, you generally need to build a new string using tools such as concatenation and slicing, and assign the result back to the string's original name if desired:

```
python >>> S = 'text'>>> S = S + 'ual!' # 要改变字符串，就构造一个新字符串>>> S
'textual!'
```

```
>>> S = S[:4] + ' processing' + S[-1]>>> S
'text processing!'
```

The first example adds a substring at the end of *S*, by concatenation. Really, it makes a new string and assigns it back to *S* to save it, but you can think of this as "changing" the original string. The second example replaces three characters with many by slicing, indexing, and concatenating. As you'll see in the next section, you can achieve similar effects with string methods like `replace`:

```
python >>> S = 'text'>>> S = S.replace('ex', 'hough')>>> S
'thought'
```

Like every operation that yields a new string value, string methods generate new string objects. If you want to retain those objects, you can assign them to variable names. Whether by sequence operations or methods, generating a new string object for each string c

hange is not as inefficient as it may sound—remember, as discussed in the preceding chapter, Python automatically garbage-collects (reclaims the space of) old unused string objects as you go, so newer objects reuse the space held by prior values. Python is usually more efficient than you might expect. But string methods can do much more, as the next section will explain.

NOTE — Except for bytearray: As previewed in Chapter 4 and to be covered in Chapter 37, Python has a string type known as bytearray, which is mutable and so may be changed in place. bytearray objects aren't really text strings; they're sequences of small, 8-bit integers. However, they support most of the same operations as normal strings and print as ASCII characters when displayed. Accordingly, they provide another option for large amounts of simple 8-bit text that must be changed frequently. Richer Unicode text and strings in general, though, require techniques shown here.

中文翻译

还记得术语不可变序列 (immutable sequence) 吗? 如我们所见, "是序列"意味着字符串支持拼接、重复、索引、切片这类操作; 而"不可变"意味着你不能就地修改字符串——例如, 不能对索引赋值: `python >>> S = 'text'>>> S[0] = 'n'` `TypeError: 'str' object does not support item assignment` 那么, Python 里到底怎么修改文本信息? 要改字

字符串，你通常需要用拼接、切片这类工具构造一个新字符串，需要时再把结果赋回字符串原来的名字：`python >>> S = 'text'>>> S = S + 'ual!'` # 要改变字符串，就构造一个新字符串`>>> S'textual!'>>> S = S[:4] + ' processing' + S[-1]>>> S'text processing!'`第一个例子通过拼接在 S 末尾加一个子串。其实它是做了一个新字符串并赋回 S 以保存它，但你可以把它想成“改变”了原字符串。第二个例子通过切片、索引、拼接，把三个字符替换成更多字符。如你在下一节将见到的，用 `replace` 这类字符串方法也能达到类似效果：`python >>> S = 'text'>>> S = S.replace('ex', 'hough')>>> S'thought'`与所有产生新字符串值的操作一样，字符串方法会生成新的字符串对象。如果你想保留这些对象，就把它们赋给变量名。无论是序列操作还是方法，为每次字符串修改都生成一个新对象，并不像听起来那么低效——记住，如上一章所述，Python 会随着运行自动垃圾回收（回收空间）旧的、不再使用的字符串对象，新对象会复用之前值所占的空间。Python 通常比你预期的更高效。但字符串方法还能做更多，下一节说明。

注意——除 `bytearray` 外：如第 4 章预览、第 37 章将详细讲，Python 有一种 `bytearray` 字符串类型，它是可变的（mutable），因此可以就地修改。`bytearray` 对象并不真是文本字符串；它们是小型 8 位整数的序列。不过，它们支持普通字符串的多数相同操作，显示时打印为 ASCII 字符。因此，对于需要频繁修改的大量简单 8 位文本，它们提供了另一种选择。但更丰富的 Unicode 文本以及一般的 `str` 字符串，需要本处所示的技术。

深度理解

·核心概念：修改字符串的唯一途径是造新字符串再重新绑定名字。S = S + 'ual!' 里的名字 S 只是"换了个指向"，旧对象交还给垃圾回收。真正不变的是：str 对象的内容从出生到回收恒为常量。

·底层实现：S[0]='n' 走 STORESUBSCR 指令，str 未实现元素赋值（类型描述符里没有 sqassitem），于是抛 TypeError:'str' object does not support item assignment——这个报错就是"immutable 在字节码层的体现"。
+ 则新建 PyUnicode。垃圾回收（CPython 引用计数+分代 gc）会回收没有引用的旧'text'对象——S 重新绑定后，两层对象被依次释放，底层内存可用于新对象。

·设计原因：不可变带来哈希性（字符串可作字典键/集合元素）、线程安全、无共享引用污染（第 6 章的别名陷阱），代价是需要时重建对象。bytearray 作为"可变的妥当出口"，兼顾需要就地高频修改 8 位数据的场景。

·实际问题：文本清洗、模板替换、日志脱敏——处处"新写不旧改"；高频拼接时注意 $O(n^2)$ ，批量改造成 list + ''.join()。

·初学者误区：(1) 反复 += 拼长字符串（三次+，旧版 $O(n^2)$ ）；(2) 想当然 S[2]='x' 就地改——那是列表行为，字符串会报错；(3) 混淆"重新赋值"与"原地修改"，误以为 S.replace(...) 忘了接返回值就能改掉 S。

代码分析

```
>>> S = 'text'
>>> S[0] = 'n'                                #
TypeError: 'str' object does not support item assignment
>>> S = S[:4] + ' processing' + S[-1]        #
>>> S
'text processing!'
>>> S = S.replace('ex', 'hough')              #
>>> S
'thought'
```

解释器如何处理：S[:4] 造'text'，S[-1] 造'!'，两次 + 各新建临时字符串，最终绑定给 S——中间临时对象即刻失去引用、被引用计数回收。replace 在 C 层先定位子串出现位置 (strstr 风格)，为新串按 (原长 - 被替长 + 新替长) 精确分配内存再拷贝拼装，返回全新 PyUnicode。关键点：没有任何一步改动过最初的'text'，最后一轮对 S.replace 的返回值必须显式接住 (赋回 S)，否则结果被丢弃。设计思想：immutable + 值语义让程序可以放心共享 str，需要"变化"时以函数式方式"推导新值"。

7.6 字符串方法 (String Methods)

英文原文

In addition to all the string operations already introduced, strings provide a set of methods that support more sophisticated text-processing goals. In Python, expressions and built-in functions may work across a range of types, but methods are generally specific to

object types—string methods, for example, work only on string objects. Some method names are used by multiple objects in Python for consistency (e.g., many have count and copy methods, and most mutables have a pop), but they are still more type specific than other tools.

中文翻译

除了已经介绍的全部字符串操作之外，字符串还提供一套方法（method），支持更复杂的文本处理目标。在 Python 中，表达式和内置函数可以跨类型工作，但方法通常类型专属（specific to object types）——例如，字符串方法只作用于字符串对象。为了保持一致性，Python 中某些方法名会被多种对象共用（比如很多对象有 count、copy 方法，多数可变对象有 pop），但它们仍然比其他工具更类型专门化。

深度理解

- 核心概念：方法与运算符/内置函数的二分法——len(x)、x+y 跨类型通用（多态）；x.replace(...) 要求 x 是字符串。方法 = "挂在特定类型上的函数"。
- 底层实现：str 类型描述符里有一张方法表（strmethods 数组），S.find 是取属性 find，实际是返回绑定方法（bound method）对象，随后调用把 S 作为隐式第一参数传入。
- 设计原因：把类型相关的行为收进类型内部（封装），命名空间整洁、发现性好（dir(S) 一目了然），也能与运算

符重载协同。

·实际问题：文本清洗链路就是方法串起来的：`.strip()` → `.lower()` → `.replace()` → `.split()`。

·初学者误区：拿方法当全局函数用（写 `find(S,'ac')`）会报错；忘记方法“非就地”、不接返回值。

7.6.1 方法调用语法 (Method Call Syntax)

英文原文

As introduced in Chapter 4, methods are simply functions that are associated with and act upon particular objects. Technically, they are attributes attached to objects that happen to reference callable functions which always have an implied subject. In finer-grained detail, functions are packages of code, and method calls combine two operations at once—an attribute fetch and a call:- Attribute fetches: An expression of the form `object.attribute` means "fetch the value of attribute in object."- Call expressions: An expression of the form `function(arguments)` means "invoke the code of function, passing zero or more comma-separated argument objects to it, and return function's result value." Putting these two together allows us to call a method of an object. The method call expression `object.method(arguments)` is evaluated from left to right—Python will first fetch the method of the object and then call it, passing in both object and the arguments. Or,

in plain words, the method call expression means this : Call method to process object with arguments. If the method computes a result, it will also come back as the result of the entire method-call expression. As a more tangible example: `python >>> S = 'hack'>>> result = S.find('ac')` # 调用 find 方法, 在字符串 S 中查找 'ac' This mapping holds true for methods of both built-in types, as well as user-defined classes we'll study later. As you'll see throughout this part of the book, most objects have callable methods, and all are accessed using this same method-call syntax. To call an object method, as you'll see in the following sections, you have to go through an existing object; methods cannot be run (and make little sense) without a subject.

中文翻译

如第 4 章介绍, 方法就是与特定对象关联、并作用于该特定对象的函数。严格说, 它们是挂在对象上的属性, 恰好引用了可调用函数, 且永远带一个隐含主体 (implied subject)。更细粒度地看: 函数是代码包, 而方法调用一次把两种操作合二为一——属性抓取 (attribute fetch) 和调用 (call) : - 属性抓取 (attribute fetch) : 形如 `object.attribute` 的表达式意思是 "取 object 中 attribute 的值"。 - 调用表达式 (call expression) : 形如 `function(arguments)` 的表达式意思是 "调用 function 的代码, 把零个或多个逗号分隔的 argument 对象传给它, 并返回 function 的结果值"。把两者合起来就能调用一个对象的方法。方法调用表达式 `object.method(arguments)` 从左到右求值——Pyth

on 先抓取 object 的 method，然后调用它，把 object 和 arguments 都传进去。用大白话说，方法调用表达式意思是："调用 method、用 arguments 处理 object。"如果方法计算出一个结果，它也会作为整个方法调用表达式的结果返回。用一个更具体的例子：`python >>> S = 'hack'>>> result = S.find('ac')` # 调用 find 方法，在字符串 S 中查找'ac'这种映射对内置类型和稍后我们要学的用户自定义类的方法都成立。如本书这一部分你将看到的：大多数对象都有可调用方法，都用这一套方法调用语法访问。要调用对象方法，如后续小节所示，你必须经由一个已存在的对象；没有主体，方法就无法被运行（也没什么意义）。

深度理解

- 核心概念：方法调用 = 属性抓取 + 调用两步组合；object 自动成为方法的隐式主体（即 C 风格里的 this/Java 的 this 但更透明）。
- 底层实现：S.find 经过描述符协议 PyObjectGetAttr，str 的无参函数 (str.find) 是 unbound/descriptor，绑定到实例 S 后成为 bound method (PyMethodNew)，调用时 S 作为第一实参传入 C 函数 unicodefind。
- 设计原因：显式的 S 前缀让"对谁操作"读代码即知，避免 C 全局函数 strstr(s, sub) 把主体混进参数表。
- 实际问题：链式调用 `line.lower().replace(",")`、作为一等对象传递 `f = S.find` 稍后调用。
- 初学者误区：只写方法名不接括号（拿到 bound method 对象而不是结果）、忘记括号、对结果不接收赋值。

代码分析

```
>>> S = 'hack'
>>> result = S.find('ac')    # S.find + ('ac')
```

解释器如何处理：字节码依次为 LOADNAME S → LOADMETHOD find (CPython 3.7+ 的专用指令，快路径避免完整属性调用开销) → LOADCONST'ac' → CALLMETHOD。执行时，LOADMETHOD 经类型描述符取到 C 实现 strfind，CALLMETHOD 把 S 放第一参数位、'ac' 放第二位，内部 unicodefind 从 0 开始搜索，命中偏移 1，于是 result 绑定为 1；没查到则返回 -1。设计思想：语法层面的".+("是两种机制的合成，学习识别这一步对理解用户自定义类的 getattr 与 call 都有地基作用。

7.6.2 全部字符串方法（今日版）（All String Methods (Today)）

英文原文

Table 7-3 summarizes the methods and call patterns for built-in string objects in Python 3.12. These change over time, so be sure to check Python's standard-library manual for the most up-to-date list, or run a dir or help call interactively on any string or the str type name, as shown in Chapter 4. In this table, S is a string object; optional arguments are enclosed in [] brackets; nested [] mean optional following an optional; X and X mean any number X; and the listed methods i

plement higher-level operations such as splitting and joining, case conversions, content tests, and substring searches and replacements.

Table 7-3. String method calls in Python 3.12 (Python 3.12 字符串方法调用)

方法	方法
S.capitalize()	S.ljust(width[, fill])
S.casefold()	S.lower()
S.center(width[, fill])	S.lstrip([chars])
S.count(sub[, start[, end]])	S.maketrans(x[, y[, z]])
S.encode([encoding[, errors]])	S.partition(sep)
S.endswith(suffix[, start[, end]])	S.removeprefix(prefix)
S.expandtabs([tabsize])	S.removesuffix(suffix)
S.find(sub[, start[, end]])	S.replace(old, new[, count])
S.format(fmtstr, args, kwargs)	S.rfind(sub[, start[, end]])
S.formatmap(mapping)	S.rindex(sub[, start[, end]])
S.index(sub[, start[, end]])	S.rjust(width[, fill])
S.isalnum()	S.rpartition(sep)
S.isalpha()	S.rsplit([sep[, maxsplit]])
S.isascii()	S.rstrip([chars])
S.isdecimal()	S.split([sep[, maxsplit]])
S.isdigit()	S.splitlines([keepends])
S.isidentifier()	S.startswith(prefix[, start[, end]])
S.islower()	S.strip([chars])
S.isnumeric()	S.swapcase()
S.isprintable()	S.title()
S.isspace()	S.translate(map)
S.istitle()	S.upper()
S.isupper()	S.zfill(width)
S.join(iterable)	

As you can see, strings have many methods, and we don't have space to cover them all here; omissions can be found in other resources when needed. To help you get started, though, let's work through some code that demonstrates some of the most commonly used methods in action and illustrates Python text-processing basics along the way.

中文翻译

表 7-3 汇总了 Python 3.12 内置字符串对象的方法与调用形式。这些方法会随时间变化，所以务必查阅 Python 标准库手册获取最新列表，或者如第 4 章所示，在任何字符串或 str 类型名上交互式地跑 dir 或 help。本表中，S 是一个字符串对象；可选参数用 [] 方括号括起；嵌套的 [] 表示“可选之后的再可选”；X 和 X 表示任意数量的 X；列出的方法实现了更高级的操作，如拆分与连接、大小写转换、内容测试、子串查找与替换。

表 7-3 Python 3.12 字符串方法调用（如上表，全表两个方法合成一行）

如你所见，字符串有很多方法，这里没有篇幅全部介绍；需要时缺失项可在其他资料中找到。不过，为了帮你起步，我们做一遍代码，演示最常用的几个方法在实际中的用法，顺带说明 Python 文本处理的基础。

深度理解

·核心概念：字符串方法三大族——查询族（find/index/count/startswith/endswith）、变换族（upper/replace

/strip/split/join)、测试族 (isdigit/isalpha/isspace 等 is)。命名有规律可循。

·底层实现：全部是 C 层 unicode 函数族：find→unicod
efind、split→unicodesplit、isopen 系调用 Unicode
数据库 (unicodedata) 的字符属性表。方法数随版本变
化 (3.9 加了 removeprefix/removesuffix, 那是 PEP 6
16)。

·设计原因：把这些高频操作内建，避免每个程序自造轮子
；is 测试族依托 Unicode 分类，跨语言一致。

·实际问题：清洗 (strip/lower)、分词 (split)、重组
(join)、校验 (isdigit/isspace) 构成日常文本处理的
主干。

·初学者误区：isalnum/isalpha 是 Unicode 语义 (含中
文"汉"也是 isalpha True)；index 找不到抛 ValueError
而 find 返回 -1 (选哪个取决于要不要异常)；isdigit 对'
2' 返回 True 但 isdecimal 返回 False。

7.6.3 修改字符串 (第二部分)：字符串方法 ("Changing" Strings, Part 2: String Methods)

英文原文

As we've seen, most strings cannot be changed in place directly because they are immutable. We explored changing strings with sequence operations in the preceding section, but let's resume that story here in the context of methods.

By way of review, to make a new text value from an existing string, you can construct a new string with sequence operations such as slicing and concatenation. For example, to replace two characters in the middle of a string, you can use code like this, much as we did in the prior section:

```
python >> S = 'textly!'>> S[:4] + 'ful' + S[-1] # 用序列操作构造新字符串'textful!'
```

But, if you're really just out to replace a substring, you can use the string replace method instead:

```
python >> S = 'textly!'>> S.replace('ly', 'ful') # 把 S 中所有'ly' 替换为'ful'"textful!"
```

The replace method is more general than this code implies. It takes as arguments the original substring (of any length) and a new substring (of any length) to replace the original, and performs a global search and replace—subject to an optional third argument that limits the number of replacements made:

```
python >> '--@--@--@--'.replace('@', 'PY', 2)'--PY--PY--@--'
```

In such a role, replace can be used as a tool to implement simple template replacements (e.g., in form letters). If you need to replace one fixed-size string that can occur at any offset, you can do a replacement again, or search for the substring with the string find method and then slice:

```
python >> S = 'xxxxPYxxxxPYxxxx'>> where = S.find('PY') # 搜索位置 >> where # 出现在偏移 4 4 >> S = S[:where] + 'CODE' + S[(where+2):] >> S'xxxxCODExxxxPYxxxx'
```

The find method returns the offset where a substring appears, or -1 if it is not found (it searches from the front by default, and its cousin rfind searches in reverse). As we saw earlier, this is a substring search operation just like the in expression, but find returns the position of a located substring. In this context, replace does the job easier, and can do more—in the following, replacing both multiple occurrences and multiple targets:

```
python >> S = 'xxxxPYxxxxPYxxxx'>> S.replace('PY','CODE', 1) # 只替换一个'xxxxCODExxxxPYxxxx'>> S.replace('PY','CODE') # 全部替换'xxxxCODExxxxCODExxxx'>> 'xxxxWHATxxxxHOWxxxx'.replace('WHAT','CODE').replace('HOW','PYTHON')'xxxxCODExxxxPYTHONxxxx'
```

As a reminder, replace returns a new string object each time (which is why two calls can be strung together here). Because strings are immutable, methods, like sequence operations, never really change the subject string in place—even if they are called "replace"! To save the new string object produced by a method call, assign it to a name:

```
python >> S = S.replace('PY','CODE') >> S'xxxxCODExxxxCODExxxx'
```


The fact that concatenation operations and the `replace` method generate new string objects each time they are run is a potential downside of using them to change strings: each interim result must create a full-fledged object with a fresh copy of its text. If you have to apply many changes to a very large string, you might be able to improve your script's performance by converting the string to an object that does support in-place changes:

```
python >> S = 'text'>> L = list(S) # 把字符串炸开成一个列表
>> L ['t','e','x','t']
```

The built-in `list` function (really, an object construction call) builds a new list out of the items in any sequence (or other iterable)—in this case, "exploding" the characters of a string into a list. Once the string is in this form, you can make multiple changes to it without generating a new copy for each change:

```
python >> L[0] = 'h' # 列表可以，字符串不行
>> L[3] = '!'>> L ['h','e','x','!']
```

After your changes, you can convert back to a string if needed (e.g., to write to a file) by using the `string join` method to "implode" the list back into a string:

```
python >> S = ''.join(L) # 把列表重新聚合回字符串
>> S
'hex!'
```

The `join` method may look a bit backward on first encounter. Because it is a method of strings (not of lists

), it is called through the desired delimiter string. join puts the strings in a list (or other iterable) together, with the delimiter between list items; in this case, it uses an empty string delimiter to convert from a list back to a string. More generally, any string delimiter and iterable of strings will do:

```
python >> 'PY'.join(['which', 'language', 'is', 'best', '?'])  
whichPYlanguagePYisPYbestPY?
```

Though subject to Python implementation, joining substrings all at once might run faster than concatenating them individually. The mutable bytearray string noted earlier may help with efficiency too; because it can be changed in place, it offers an alternative to this list/join combo for simple kinds of byte-sized text like ASCII.

中文翻译

如我们所见，大多数字符串不能直接就地修改，因为它们是不可变的。上一节我们用序列操作探索了修改字符串的方法，这里在方法语境下继续这个故事。复习一下：要想从现有字符串做出新的文本值，可以用切片、拼接等序列操作构造新字符串。例如，要替换字符串中间的某两个字符，可以写类似上一节的代码：

```
python >>> S = 'textly!'>>> S[:4] + 'ful' + S[-1]
```


用序列操作构造新字符串'textful!' 但如果你真的只想替换子串，直接用字符串 replace 方法更好：

```
python >>> S = 'textly!'>>> S.replace('ly', 'ful')
```


把 S 中所有'ly' 替换为'ful' 'textful!' replace 方法比这段代码暗示的更通用。它接收原始子串（任意长度）和要替换成的新

子串（任意长度）作为参数，执行全局搜索替换——并受一个限制替换次数的可选第三参数约束：python >>> '@--@--@--'.replace('@','PY', 2)--PY--PY--@--'在这种角色下，replace 可用来实现简单的模板替换工具（比如信函模板）。如果你需要替换一个可出现在任意偏移处的定长字符串，可以再做一次替换，或用 find 方法搜索子串再用切片：python >>> S='xxxxPYxxxxPYxxxx'>>> where = S.find('PY') # 搜索位置>>> where # 出现在偏移 4 4>>> S = S[:where] + 'CODE' + S[(where+2):]>>> S'xxxCODExxxxPYxxxx' find 方法返回子串出现的偏移，找不到则返回 -1（默认从前向后搜索，它的堂亲 rfind 从后向前搜索）。如我们前面所见，这是与 in 表达式相同的子串搜索操作，但 find 返回已定位子串的位置。在这种语境下，replace 做这活更省事、且能做更多——下面同时替换多处出现和多个目标：python >>> S='xxxxPYxxxxPYxxxx'>>> S.replace('PY','CODE', 1) # 只替换一个'xxxxCODExxxxPYxxxx'>>> S.replace('PY','CODE') # 全部替换'xxxxCODExxxxCODExxxx'>>> 'xxxxWHATxxxxHOWxxxx'.replace('WHAT','CODE').replace('HOW','PYTHON')'xxxxCODExxxxPYTHONxxxx'提醒一下：replace 每次返回一个新字符串对象（这正是这里能串两次调用的原因）。因为字符串不可变，方法和序列操作一样，从不会真正就地改变主体字符串——哪怕方法名字叫"replace"！要保存方法调用产生的新字符串对象，把它赋给一个名字：python >>> S = S.replace('PY','CODE')>>> S'xxxxCODExxxxCODExxxx'拼接操作和 replace 方法每次运行都生成新字符串对象，这一事实是用它们修改字符串的潜在缺点：每个中间结果都要创建一个带着文本全新副本的完整对象。如果你必须

对一个很大的字符串做很多次修改，可以把字符串转换成支持就地修改的对象来提升脚本性能：python >>> S='text'
>>> L = list(S) # 把字符串炸开成一个列表>>> L ['t','e',
'x','t'] 内置 list 函数（实际上是对象构造调用）从任何序列（或其他可迭代对象）的元素构建出一个新列表——本例中，就是把字符串的字符"炸开"成列表。一旦字符串进入这种形式，你就可以多次修改它，而不必为每次修改生成新副本：python >>> L[0]='h' # 列表可以，字符串不行>>> L[3]='!'
>>> L ['h','e','x','!'] 修改完成后，需要的话（比如写文件）可以用字符串 join 方法把列表"聚合"回字符串：python >>> S=''.join(L) # 把列表重新聚合回字符串>>> S'hex!' join 方法乍看有点"倒着长"。因为它是字符串的方法（不是列表的），要通过期望的分隔符（delimiter）字符串来调用。join 把列表（或其他可迭代对象）中的字符串串在一起，分隔符位于列表各项之间；本例用空字符串分隔符把列表转回字符串。更一般地，任意字符串分隔符和字符串可迭代对象都行：python >>> 'PY'.join(['which','language','is','best','?'])'whichPYlanguagePYisPYbestPY?' 虽然取决于 Python 实现，但一次性拼合子串通常比逐条拼接更快。前面提到的可变 bytearray 字符串在效率上也有帮助；因为它可以就地修改，对 ASCII 这类简单字节尺寸文本，它提供了 list/join 组合的替代方案。

深度理解

·核心概念：方法改字符串的两大范例——replace（整块替换）与 find + 切片（定位后重排）；以及"终极就地路线"：字符串 → 可变 list → 就地改 → ''.join 回字符串。

·底层实现：replace 用 C 层搜索-统计-分配-拷三段式：先统计出现次数（第三参数给定则截断）、按下式算新长 $len - nlen(old) + nlen(new)$ 、一次性分配、逐段拷贝。find 走子串搜索（长的模式用高效算法）。list(S) 迭代字符逐个装箱成 PyObject 列表元素。".join(L) 先对 iterable 求长并计算总长，一次性分配结果缓冲区再做 memcpy——所以比 for loop += 的多次分配快得多（这是性能建议中的经典）。

·设计原因：immutable 与性能的平衡——少次整合用 replace 干净直接，大量乱序编辑先入可变的中间形态 (list/bytearray)，最后一次拼回，避免复制放大。

·实际问题：批量替换模板变量、URL 参数清洗 (replace)、文件名/路径改造 (find+切片)、对用户输入逐字符过滤 (explode-edit-implode)。

·初学者误区：(1) replace 不指定次数会替换所有出现（想要只改一次记得传第三个参数）；(2) 忘记 replace/find 的返回值而丢弃——字符串永远返回新对象；(3) join 用错了对象——不是 L.join("") 而是 delimiter.join(L)；(4) list(S) 炸开的是字符不是"单词"。

代码分析

```
>>> L = list(S)           #
>>> L[0] = 'h'           #
>>> S2 = ''.join(L)       # 'hex!'
```

解释器如何处理：list(S) 先 iter(S) 得 striterator，逐个 next 取单字符、PyListAppend 追加，得到 ['t','e','x','t']。L[0]='h' 走 STORESUBSCR，list 实现了 sqassitem，

原地改第 0 个引用。".join(L) 内部：先遍历 L 求和各元素长度（并校验全是 str，否则 TypeError），按总长一次 PyUnicodeNew 分配，然后连续 memcpy 拼装，全程 O(n) 一次分配。对比 ans += c 循环（旧版本每次 + 都新分配，O(n²))，join 是"先算总账再一起搬"的思路。设计思想：把"会变的过程"交给可变类型，把"不变的结果"还给 immutable 字符串——各取所长。

7.6.4 更多字符串方法：解析文本 (More String Methods: Parsing Text)

英文原文

Another common role for string methods is as a simple form of text parsing—that is, analyzing structure and extracting substrings. To extract substrings at fixed offsets, we can employ slicing techniques:

```
python >> line = 'aaa bbb ccc' >> col1 = line[:3] >> col2 = line[4:8] >> col3 = line[-3:] >> col1, col2, col3 ('aaa', 'bbb', 'ccc')
```

Here, the columns of data appear at fixed offsets and so may be sliced out of the original string. This technique passes for parsing, as long as the components of your data have known positions. If instead some sort of delimiter separates the data, you can pull out its components by splitting. This will work even if the data may show up at arbitrary positions within the str

ing:

```
python >> line = 'aaa bbb ccc' >> cols = line.split() >  
> cols ['aaa','bbb','ccc']
```

The string split method chops up a string into a list of substrings, around a delimiter string. We didn't pass a delimiter in the prior example, so it defaults to whitespace—the string is split at groups of one or more spaces, tabs, and newlines, and we get back a list of the resulting substrings. In other applications, more tangible delimiters may separate the data.

To demo, the next example splits (and hence parses) the string at commas, a separator common in some database roles (string conversion tools covered earlier can change substrings here into numbers):

```
python >> line = 'Python,3.12,scripting,33' >> line.split(',') ['Python','3.12','scripting','33']
```

Delimiters can be longer than a single character, too:

```
python >> line = 'youPYarePYaPYstringPYcoder' >> line.split('PY') ['you','are','a','string','coder']
```

Although there are limits to the parsing potential of slicing and splitting, both run fast and can handle basic text-extraction chores. Comma-separated text data is also part of the CSV file format; for a more advanced tool on this front, see also the csv module in Python's standard library.

中文翻译

字符串方法的另一个常见角色是充当简单的文本解析 (parsing) ——即分析结构、提取子串。要在固定偏移处提取子串，可以用切片技术：python >>> line = 'aaa bbb ccc' >>> col1 = line[:3] >>> col2 = line[4:8] >>> col3 = line[-3:] >>> col1, col2, col3 ('aaa','bbb','ccc') 这里数据列出现在固定偏移处，因此可以从原字符串中切片取出。只要数据分量位置已知，这种技术就算解析。如果改用某种分隔符分隔数据，你可以通过拆分 (splitting) 取出分量——即使数据可能出现在字符串内的任意位置也能工作：python >>> line = 'aaa bbb ccc' >>> cols = line.split() >>> cols ['aaa','bbb','ccc'] 字符串 split 方法围绕一个分隔符字符串把字符串切成一系列子串。前面例子没传分隔符，所以它默认为空白 (whitespace) ——字符串在连续一个或多个空格、制表符、换行处拆分，我们得到结果子串的列表。其他应用中，更“实感”的分隔符可能分隔数据。演示一下：下例按逗号拆分（从而解析）字符串，逗号在数据库角色中常见（前面讲的字符串转换工具可以把这里的子串转成数字）：python >>> line = 'Python,3.12,scripting,33' >>> line.split(',') ['Python','3.12','scripting','33'] 分隔符也可以比一个字符更长：python >>> line = 'youPYarePYaPYstringPYcoder' >>> line.split('PY') ['you','are','a','string','coder'] 尽管切片与拆分的解析潜力有限，但它们运行快、能处理基本的文本提取杂活。逗号分隔的文本数据也是 CSV 文件格式的一部分；这方面更高级的工具见 Python 标准库的 csv 模块。

深度理解

·核心概念：解析的两条基本路线——按固定位置（切片）与按分隔符（split）。前者适用于规律数据，后者包容任意位置。

·底层实现：split()（无参）按 Unicode 空白规则划分：把连续空白当作一个分隔点，自动丢弃空串；split(',') 有明确分隔符，则按"分隔符序列"切，分隔符出现之间可能产生空串（'a,,b'.split(',') → ['a','', 'b']）。两者都在 C 层扫描-定位-装配。

·设计原因：用接口把"切"和"拆"统一；位置解析交给切片的半开区间，分隔解析交给 split 的语义化接口，用户按数据形态选择。

·实际问题：CSV/配置文件行解析、日志字段切分、a,b,c 展开、URL query 拆键值（再配合后续字典）。

·初学者误区：(1) 无参 split 与 split('') 不同——前者把多空白缩并且无空串、后者单空格精确切分会产生空串；(2) 以为 split 能正则——不行（那是 re 的活儿）；(3) 位置解析用错列宽（line[4:8] 得'bbb' 含尾空格——一切到哪一列由你算准）。

代码分析

```
>>> line = 'aaa bbb ccc'
>>> line[:3], line[4:8], line[-3:] #
('aaa', 'bbb ', 'ccc')             # col2 'bbb '
>>> 'Python,3.12,scripting,33'.split(',') #
['Python', '3.12', 'scripting', '33']
```

解释器如何处理：切片版本直接按偏移三次 `getitem` 取子串——`line[4:8]` 取下标 4..7 得 `'bbb'`（含第 7 位的空格），这告诫你“固定列宽”要精确到空白。`split(',')` 在 C 层线性扫描逗号位置，把每段截成 `PyUnicode` 子串进列表，得到四个元素的列表，其中 `'3.12'` 可以再用 `float` 转数值。设计思想：解析是“形状已知时切片、形状未知时按分隔符”的选择题，把两种策略都做成 $O(n)$ 的内建路径。

7.6.5 其他常用字符串方法 (Other Common String Methods)

英文原文

Other string methods have more focused purposes—for example, to strip off whitespace at the end of a line of text, perform case conversions, test content, and test for a substring at the end or front:

```
python >> line = "Python's strings are awesome!\n"
>> line.rstrip() # 去掉行尾空白（或其他字符）"Python's
strings are awesome!"

>> line.upper() # 大小写转换"PYTHON'S STRINGS ARE
AWESOME!\n">> line.isalpha() # 内容测试 False >> li
ne.endswith('awesome!\n') # 后缀/前缀测试 True >> li
ne.startswith('Python') True
```

Alternative techniques can also sometimes be used to achieve the same results as string methods—the in membership operator can be used to test for the pre

sence of a substring, for instance, and length and slicing operations can be used to mimic endswith:

```
python >> line.find('awesome')!= -1 # 用方法调用或表达式搜索 True >> 'awesome' in line True >> sub = 'awesome!\n' >> line.endswith(sub) # 用方法调用或切片做结尾测试 True >> line[-len(sub):] == sub True
```

Note that none of the string methods accepts patterns—for pattern-based text processing, you must use the Python `re` standard-library module, an advanced tool that will be introduced briefly in Chapter 37 but is mostly outside the scope of this text. Because of their limitations, though, string methods may run more quickly than the `re` module's tools.

Again, because there are so many methods available for strings, we won't look at every one here. You'll see some additional string examples later in this book, but for more details you can also turn to the Python library manual and other reference resources, or simply experiment interactively on your own. As noted in Chapter 4, `help(S.method)` gives info for a method of any string object `S`; use `help(str.method)` if you have no `S`.

All that being said, one method is noticeably absent from this section's coverage: `format` performs string formatting, which combines many operations in a single step. It's also part of a larger topic in Python, which we turn to next.

中文翻译

其他字符串方法有更聚焦的用途——比如去掉文本行末尾的空白、做大小写转换、测试内容、测试子串在末尾还是在前端：

```
python >>> line = "Python's strings are awesome!\n" >>> line.rstrip() # 去掉行尾空白（或其他字符）
"Python's strings are awesome!" >>> line.upper() # 大小写转换"PYTHON'S STRINGS ARE AWESOME!\n" >>>
line.isalpha() # 内容测试False >>> line.endswith('awesome!\n') # 后缀/前缀测试True >>> line.startswith('Python')
True 某些时候，替代技术也能达到与字符串方法相同的结果——比如 in 成员运算符可用于测试子串是否存在，长度加切片可用来模拟 endswith：python >>> line.find('awesome') != -1 # 用方法调用或表达式搜索True >>> 'awesome' in line
True >>> sub = 'awesome!\n' >>> line.endswith(sub) # 用方法调用或切片做结尾测试True >>> line[-len(sub):] == sub
True 注意：字符串方法一个都不接受模式（pattern）——要做基于模式的文本处理，必须使用 Python 标准库的 re 模块，这是一个高级工具，第 37 章会简要介绍，但基本超出本书范围。也正因它们的能力有限，字符串方法通常比 re 模块的工具运行更快。同样地，因为字符串可用的方法太多，这里不会逐个看。本书后面还会有一些额外的字符串示例，但更完整的细节可以查阅 Python 库手册等参考资料，或自己在交互环境中实验。如第 4 章所述，help(S.method) 给出任何字符串对象 S 的某个方法的信息；如果你没有 S，就用 help(str.method)。说了这么多，本节的覆盖范围里明显缺席了一个方法：format 执行字符串格式化，一步结合多种操作。它也是 Pyt
```

hon 一个更大主题的一部分，我们接下来就讲。

深度理解

- 核心概念：方法刻画"意图"，表达式刻画"机制"——同一结果常有两套达成路径（endswith vs line[-len(sub):]==sub），但语义清晰度不同。
- 底层实现：rstrip 在 C 层从尾部扫描、遇到非空白为止，子串截取返回；带 chars 参数则按字符集合过滤。isalpha 用 Unicode 属性表判定。endswith 支持 (suffix, start, end) 分段里查。
- 设计原因："宁可专用 API，勿要手拼逻辑"——方法自带边界与容错（如 rstrip 无换行时不动行），表达力更强；方法跑得快（C 层）且无模式引擎开销。
- 实际问题：去行尾换行（rstrip）、大小写规范化入库（lower/casfold）、文件扩展名判断（endswith('.txt'))、URL 前缀路由（startswith('/api/'))。
- 初学者误区：(1) .rstrip() 不带参数去空白；带参数 rstrip(';') 会把集合内字符各 rstrip 掉，不是只去掉整个 ';'；(2) isalpha 对含空格/数字的串返回 False（它要求每个字符都是字母）；(3) 正则的 \$、. 不能喂给字符串方法——方法是纯字面量匹配；(4) line[-len(sub):]==sub 在 sub 比行还长时返回 False 而非报错，切片并不会有越界异常——这是切片"容错"的又一体现。

7.7 字符串格式化：铁人三项 (String Formatting: The Triathlon)

英文原文

Although you can get a lot done with the string methods and sequence operations you've already met, Python also provides a more advanced way to combine string processing tasks: string formatting allows us to perform multiple type-specific substitutions on a string in a single step. It's never strictly required, but it can be convenient, especially when laying out text to be displayed to a program's users. We've used string formatting informally in this book already, but it's finally time to dig into its details. As suggested in earlier examples, this story is regrettably convoluted by Python's history: there are today three different string-formatting tools that broadly overlap in functionality. This curious state of affairs reflects a common pattern in software development—new tools arise that boldly promise to be radical improvements over the past, only to be supplanted by even newer tools that boldly make the exact same claims. The net effect handicaps languages with redundancy, and users with unnecessarily steep learning curves. While learning resources could present just one of many options, that would both impose authors' opinions and do a vast disservice to readers: even if you're able to pick just one

of the formatting tools for your own work, the fact that the others have been available for decades and have been used by millions of programmers virtually guarantees that you'll be seeing them in the wild when you begin reusing other people's code. Try as it may, the new cannot erase the old. Hence, this chapter presents all three formatting tools for the sake of inclusiveness. If you're new to Python or programming in general, you probably should focus on the current latest-and-greatest f-string option—not because it's necessarily "better," but because it's more likely to garner development attention and less likely to be deprecated in a backward-incompatible future (its predecessors have been spared this fate to date, but Python has a history here). But that's not to say that the others are out of the race: the expression and method alternatives may feel more comfortable to readers with backgrounds in some other tools, and both are pervasive in the vast realms of Python code written over the last thirty-some years. Learning all three options hedges your bets best.

中文翻译

虽然你已经掌握的字符串方法和序列操作能完成很多工作，但 Python 还提供一种更高级的组合字符串处理任务的方式：字符串格式化（string formatting）允许我们一步对字符串进行多次类型特定的替换。它从来不是必需的，但很方便，尤其是在排版要给程序用户显示的文本时。本书此前已

经非正式地使用过字符串格式化，现在终于到了深挖细节的时候。正如前面的示例所暗示，这个故事被 Python 的历史弄得有点缠结：如今有三种功能大致重叠的字符串格式化工具。这种奇特的状态反映了软件开发中的常见模式——新工具大胆许诺将成为对过去彻底改进，结果又被更新的工具取代，后者同样大胆地兜售一模一样的口号。净效应是给语言带来冗余、给用户带来不必要的陡峭学习曲线。虽然学习资源可以只介绍众多选项之一，但那既是在强加作者的倾向，也是对读者极不负责：即便你能为自己的工作只挑一个格式化工具，其他工具已经存在几十年、被数百万程序员用过的现实，几乎保证你在复用别人代码时会碰上它们。新事物再努力，也抹不掉旧事物。因此，本章出于包容性把三个格式化工具都讲了。如果你是 Python 或编程新手，大概应该把重点放在当前最新最棒的 f-string 上——不是因为它必然“更好”，而是因为它更可能获得持续开发关注，更不可能在未来被不兼容地废弃（它的前辈们迄今还没遭此厄运，但 Python 这里有前科）。但这不意味着其他工具出局了：表达式和方法这两种方案，对其他工具背景的读者可能更顺手，而且两者在过往三十多年海量 Python 代码中无处不在。三个选项都学会，押注最稳。

深度理解

- 核心概念：格式化 = “模板 + 值 → 成品字符串”的一次性替换管线。三种工具（% 表达式、.format 方法、f-string）功能高度重叠，是历史叠加的产物。
- 底层实现：三者的共同底层是格式说明（format spec）解析 + format 协议：把 %d/{:d}/{:d} 解析成“按类型转换 + 按规格排版”，最终调用每个对象类型的 format（数

字实现进制/精度/对齐，字符串实现宽度/对齐）。f-string 在编译期把 {} 内表达式抽取成独立的子表达式字节码，运行时依次求值再装配——所以它是"运行期构建"，不是普通字符串常量。

·设计原因：为什么有三个？% 源自 C 的 printf 模型（Python 诞生即有）；.format 是 3.0 对 % 缺点的改良（位置/关键字、隐式 str）；f-string（3.6）把值直接内联进模板，来源不隔一行、可读性最佳。冗余既是兼容包袱，也是演化的自然结果。

·实际问题：日志（logging 格式化）、报表对齐、数字千分位、错误消息拼装——格式化是"给用户看的字符串"的标准化生产线。

·初学者误区：(1) 三种混在一处不知何去何从——新代码用 f-string，能读懂另两种即可；(2) 认为 f-string 是"模板/配置"，其实它是编译期死代码，不能当数据文件用；(3) %s 与 f-string 里的 f'{x}' 对 str(对象) 的调用路径不同，!r/!s 又是另一码事。

代码对比（三种工具等价实现）

```
kind = 'coder'
'%s is %s' % ('Python', 3.12)           # %
'{} is {}'.format('Python', 3.12)      # {}
name, ver = 'Python', 3.12
f'{name} is {ver}'                      # f-string
```

解释器如何处理：% 走 BINARYOP % 特殊路径，左串解析出 %s 目标表、右侧值按顺序填入，数字类型做 str 化。
.format 是普通方法调用，解析 {} 目标（位置/关键字/

属性/下标) 后逐一格式化。f-string 不同: 编译时 `f'{name} is {ver}'` 被转为类似 `".join([name, ' is', ver]).format()` 的字节码序列——表达式直接进入作用域查找, 运行期取当时绑定值。这解释了一个易忽视点: `%/.format` 的模板可来自字符串数据 (存文件、运行时组装), f-string 不能——它是源码的一部分。

7.7.1 三种格式化选项总览 (String-Formatting Options)

英文原文

As a preview of what we're going to explore in this section, here are today's entries in the string-formatting race:- Formatting expression: `'...%s...%s...' % (value, value)` — The original technique available since Python's inception, this form is loosely based upon the C language's `printf` model and sees widespread use in much existing code. Values on the right replace targets on the left.- Formatting method: `'...{}...{}...'.format(value, value)` — A newer technique added in Python 3.0, this form is derived in part from a same-named tool in C#/.NET. It largely overlaps with the expression's functionality but aims to address usage modes subjectively deemed subpar.- Formatting literal: `f'...{value}...{value}...'` — The very latest (so far) added in Python 3.6, this form is known as f-strings. It shares much with the method but apes a host of languages that support

t string interpolation—substituting inline expressions with their results. You can also format strings manually with string methods, though it's too cumbersome to count. And technically, an additional tool, `string.Template`, predates the method—and drives the formatting set's length up to a whopping four—but it's so scarcely used that it gets less billing than the three primary options above and is relegated to a brief sidebar here (and you'd be excused for pretending it doesn't exist at all, given the heft of the formatting toolbox!). The following sections present all three formatting options above in turn. While it may be tempting to jump straight to whatever the blogosphere may be recommending as you read these words, these sections partly build on each other (e.g., f-strings use the method's format specifier and assume its earlier coverage), so a linear read is suggested.

中文翻译

作为本节的预告，这里列出进入这场字符串格式化竞赛的今日选手：- 格式化表达式 (formatting expression) : `'...%s...%s...' % (value, value)` —— Python 问世即有的原始技术，松散地基于 C 语言的 `printf` 模型，在大量现存代码中广泛使用。右边的值替换左边的目标。- 格式化方法 (formatting method) : `'...{}...{}...'.format(value, value)` —— Python 3.0 加入的较新技术，部分源自 C#/.NET 中同名工具。它与表达式功能大量重叠，旨在解决主观上被视为欠佳的用法模式。- 格式化字面量 (formatting literal)

: f'...{value}...{value}...' ——到目前为止最新的技术，Python 3.6 加入，称为 f-string。它与方法共享很多特性，但模仿了一大批支持字符串插值（string interpolation）的语言——把内联表达式的求值结果替换进去。你也可以手动用字符串方法格式化字符串，只是繁琐得不值一提。严格讲还有一个额外工具 string.Template，它比方法更早——把这套格式化工具的总数推高到惊人的四个——但是它用得极少，受关注度不及上面三件主力，被降格到本节的简短边栏处理（考虑到格式化工具箱的份量，就算你假装它不存在也情有可原！）。下面各节依次介绍上述三个格式化选项。读到这些文字时，你可能会忍不住直接跳去博客圈正在推荐的那个选项，但本节各部分相互衔接（比如 f-string 用方法的那套格式说明符，假定你先读过它的内容），所以建议按顺序线性阅读。

深度理解

·核心概念：三个选项 = 三代产物，共用“替换+排版”的抽象，只是值从哪来（右侧元组 / 方法参数 / 源码内联表达式）与目标怎么标（%s / {} / {}）不同。

·底层实现：%独享 BINARYOP %（同取模运算符按类型分派）；.format 是普通方法；f-string 是语法糖，编译期改写为求值+格式化的字节码。

·设计原因：string.Template 走 \$var 语法、刻意不做排版（安全、够用即止）；三个主力则提供完整格式化能力（对齐、精度、进制、千分位）。

·实际问题：读旧代码必须认识 % 与 .format；写新代码主要用 f-string；模板文件（读入即字符串）只能配 %/.f

ormat/Template。

·初学者误区：把 f-string 的"表达式"能力与模板的"数据驱动"混为一谈；用 f-string 拼不安全的外部模板（安全性问题，见 7.7.4 的 eval 讨论）。

7.8 字符串格式化表达式 (The String-Formatting Expression)

英文原文

Since string-formatting expressions are the original in this department, we'll start with them. Python defines the % binary operator to work on strings. You may recall that this is also the remainder of division, or modulus, operator for numbers. When applied to strings, the % operator provides a simple way to format values as strings according to a format definition. It's a much more concise way to code multiple substitutions than processing parts individually.

中文翻译

既然格式化表达式是这一领域的元老，我们先讲它。Python 定义了 % 二元运算符作用于字符串。你可能记得，它对数字也是除法取余（模）运算符。当用于字符串时，% 运算符提供一种简单方法，按照格式定义把值格式化为字符串。比起逐个部件单独处理，它是编写多次替换更简洁的方式。

深度理解

- 核心概念：`%` 在"字符串操作数"上是格式化运算符（操作符重载复用）。左侧是模板，右侧是值（单值或元组）。
 - 底层实现：`PyUnicodeType` 的 `tpasnumber->nbremainder`？其实字符串对象把它转成 `unicodeformat`——CPython 里 `str.mod` 独立实现（非数字取模）。解析 `%s/%d` 等目标后，对每个目标按 `type code` 转换并填充。
 - 设计原因：`printf` 血统让 C 程序员零门槛；但"值多于/少于目标"的匹配规则、元组怪癖让它被后辈诟病。
 - 实际问题：老代码、某些库日志仍用 `%`；`logging` 模块官方推荐 `%s` 风格惰性格式化。
 - 初学者误区：`%` 对数字是取模——初学者在字符串上误用 `%` 是没问题的，但同一符号双重身份常令人困惑。
-

7.8.1 格式化表达式基础 (Formatting expression basics)

英文原文

To format strings with an expression: 1. On the left of the `%` operator, provide a format string containing one or more embedded conversion targets, each of which starts with a `%` (e.g., `%d`). 2. On the right of the `%` operator, provide the object that you want Python to insert into the format string on the left in place of the conversion target; for multiple targets, provide m

multiple objects in a tuple. For instance, in the following formatting example, the integer 3 replaces the %d in the format string on the left, and the string 'format' replaces the %s. The result is a new string that reflects these two substitutions, which may be printed or saved for use in other roles:

```
python >>> 'There are %d ways to %s!' % (3, 'format')
# 格式化表达式'There are 3 ways to format!'
```

Technically speaking, string formatting in any flavor is usually optional—you can generally do similar work with multiple concatenations and conversions. However, formatting allows us to combine many steps into a single operation. It's powerful enough to warrant a few more introductory examples:

```
python >>> option = 'expression' >>> 'Meet the formatting %s!' % option
# 字符串替换'Meet the formatting expression!'
```

```
>>> '%d %s %g you' % (1, 'formatter', 4.0)
# 类型特定的替换'1 formatter 4 you'
```

```
>>> '%s -- %s -- %s' % (42, 3.14159, [1, 2, 3])
# 所有类型都能匹配 %s'42 -- 3.14159 -- [1, 2, 3]'
```

The first example here plugs a string into the target on the left, replacing the %s marker. In the second example, three values are inserted into the target string. Notice that when you're inserting more than one value, you need to group the values on the right in pare

ntheses—that is, put them in a tuple. The % operator's right side generally expects a tuple of one or more items (or a dictionary of items for key references, covered ahead), but allows a single nontuple item if there is just one substitution target. You'll know which form to use when coding the expression, of course, but this difference was nevertheless deemed a quirk sufficient to justify other formatting options over time. The third example again inserts three values—an integer, a floating-point number, and a list—but notice that all of the targets on the left are %s, which stands for conversion to string. As every type of object can be converted to a string (the one used when printing), every object type works with the %s conversion code. Because of this, unless you need to do special formatting, %s is often the only code you need to remember for the formatting expression. Again, keep in mind that formatting always makes a new string, rather than changing the string on the left; because strings are immutable, it must work this way. As before, assign the result to a variable name if you need to retain it.

中文翻译

用表达式格式化字符串：1. 在 % 运算符左边放一个格式字符串，内含一个或多个嵌入的转换目标（conversion target），每个目标以 % 开头（如 %d）。2. 在 % 运算符右边放一个对象，让 Python 把它插入左边格式字符串中替换转换目标的位置；有多个目标，则在元组里提供多个对象。例

如，下面这个格式化例子：整数 3 替换左边格式字符串中的 %d，字符串'format' 替换 %s。结果是一个反映这两处替换的新字符串，可以打印或保存作其他用途：python >>>'There are %d ways to %s!' % (3,'format') # 格式化表达式'There are 3 ways to format!'严格讲，任何一种字符串格式化通常都是可选的——你可以用多次拼接和转换达到类似效果。但格式化允许我们把许多步骤合并成一次操作。它足够强大，值得再多几个入门示例：python >>> option = 'expression'>>>'Meet the formatting %s!' % option # 字符串替换'Meet the formatting expression!'>>> '%d %s %g you' % (1,'formatter', 4.0) # 类型特定的替换'1 formatter 4 you'>>> '%s -- %s -- %s' % (42, 3.14159, [1, 2, 3]) # 所有类型都能匹配 %s'42 -- 3.14159 -- [1, 2, 3]'第一个示例把一个字符串插进左边目标，替换 %s 标记。第二个示例往目标字符串插入三个值。注意：插入多个值时，需要把右边的值用括号括起来——即放入元组 (tuple) 。% 运算符的右侧一般期望一个含一项或多项的元组（或用于按键引用的字典，后面会讲），但如果只有一个替换目标，也允许单个非元组的项。写代码时你当然知道该用哪种形式，但这一差别仍被认定为怪癖，足以在后来催生其他格式化选项。第三个示例同样插入三个值——整数、浮点数、列表——但注意左边的目标全是 %s，它代表"转成字符串"。因为每种类型的对象都能转成字符串（打印时用的那种），所以每种对象类型都能配 %s 转换码。正因如此，除非需要特殊格式化，%s 往往是格式化表达式里唯一需要记住的码。再次记住：格式化总是构造新字符串，而不是修改左边的字符串；因为字符串不可变，它必然如此。照旧，需要保留结果就把它赋给变量名。

深度理解

·核心概念：`% 表达式 = 模板 % 值元组`。左侧目标的数量与右侧元组元素数量、顺序一一对应。

·底层实现：`BINARYOP %` 分派到 `str.mod`：解析左侧目标序列 (Parsef) 后，若右侧是 tuple 则展开匹配，单个值对应单个目标；目标数≠值数抛 `TypeError: not all arguments converted during string formatting`。`%s` 对任意对象调用 `str()`。

·设计原因：`%s` 作为“万能字符串化”——任何可打印对象都能用，教学简化、记忆负担小；类型码 `%d/%g/%x` 支持精确排版。

·实际问题：拼接日志、生成简单报告、老代码维护——`%s` 全类型通吃是最大卖点。

·初学者误区：(1) 只放一个值时忘了单值不用元组也可（但多个必须元组）；(2) 元组顺序写错导致错位；(3) `%s` 对对象调用 `str()`，想得到可复制形式 (`repr`) 要用 `%r`。

代码分析

```
>>> '%d %s %g you' % (1, 'formatter', 4.0)  #  
'1 formatter 4 you'  
>>> '%s -- %s -- %s' % (42, 3.14159, [1, 2, 3])  # %s str()  
'42 -- 3.14159 -- [1, 2, 3]'
```

解释器如何处理：`%` 先解析左边模板，得到目标表 `[('%d', ...), ('%s', ...), ('%g', ...)]`；右侧 `(1, ...)` 是元组，按位置一一喂给每个目标：`%d` 要求整数→`str(1)`；`%g` 对浮点做“最短表示”（`'4'`，去掉多余的 `.0`）；`%s` 对 `42`、`3.14159`、[

1,2,3] 分别调用其 str。3.14159 的 str 是'3.14159'，而 %g 默认 6 位精度会输出'4'——由此看到"类型码决定排版"的力量。最终''.join 装配返回新串。设计思想：格式与数值分离——模板定义"样貌"，值定义"内容"，一次性替换避免手工 + 拼写。

7.8.2 格式化表达式自定义格式 (Formatting expression custom formats)

英文原文

For more advanced type-specific formatting, you can use any of the conversion type codes listed in Table 7-4 in formatting expressions; they appear after the % character in substitution targets. C programmers will recognize most of these because Python string formatting supports all the usual C printf format codes (but returns the result, instead of displaying it like printf). Some of the format codes in the table provide alternative ways to format the same type; for instance, %e, %f, and %g provide alternative ways to format floating-point numbers.

Table 7-4. Formatting-expression type codes (格式化表达式类型码)

码	含义
s	字符串（或任一对象的 str(X) 字符串）
r	同 s，但用 repr 而非 str

a	同 s, 但用 ascii 而非 str
c	字符 (整数码或字符串)
d	十进制 (有符号十进制整数)
i	整数 (同 d)
u	同 d (已过时: 不再是无符号)
o	八进制整数 (基 8)
x	十六进制整数 (基 16)
X	同 x, 但用大写字母
e	浮点指数形式, 小写
E	同 e, 但用大写字母
f	浮点十进制形式
F	同 f, 但用大写字母
g	浮点 e 或 f 形式
G	浮点 E 或 F 形式
%	字面 % (写成 %%)

All told, conversion targets in the format string on the expression's left side support a variety of conversion operations with a fairly sophisticated syntax all their own. In formal terms, the general structure of conversion targets looks like the following, where [...] denotes an optional part (its two square-bracket characters are not included in the expression's code), and no spaces are allowed between parts (though some parts may contain spaces): `%[(keyname)][flags][width][.precision]typecode` One of the type code characters in the first column of Table 7-4 shows up at the end of this target string's format, at typecode. Between the % and this type code character, you can do any (or none) of the following:- Provide a key name for indexing the dictionary used on the right side of the expressio

n.- List flags that specify zero padding (0), left justification (-), numeric sign (+), or a blank before positive numbers and a - for negatives (a space), where - overrides 0, and + overrides a space.- Give a total but minimum field width for the substituted text.- Set the number of digits (precision) to display after a decimal point for floating-point numbers. Both the width and precision parts can also be coded as a to specify that they should take their values dynamically from the next item in the input values on the expression's right side (useful when this isn't known until your code is run, but unavailable when keynames are used). And if you don't need any of these extra tools, a simple %s in the format string will be replaced by the corresponding value's default print string, regardless of its type.

中文翻译

要做更高级的类型特定格式化，可以在格式化表达式中使用表 7-4 列出的任意转换类型码；它们出现在替换目标的 % 字符之后。C 程序员会认识其中大多数，因为 Python 字符串格式化支持所有常用的 C printf 格式码（但返回结果，而非像 printf 那样直接显示）。表中某些格式码提供了格式化同一种类型的替代方式；例如 %e、%f、%g 是格式化浮点数的三种替代方式。

表 7-4 格式化表达式类型码（全表见上）

总而言之，表达式左侧格式字符串中的转换目标支持各种转换操作，且各自有一套相当精密的语法。形式上，转换目标

的一般结构如下，其中 [...] 表示可选部分（两个方括号字符不包含在表达式的代码里），各部分之间不允许有空格（但某些部分内部可以含空格）：`%[(keyname)][flags][width][.precision]typecode` 表 7-4 第一列的某个类型码字符出现在该目标字符串格式的末尾，即 `typecode` 处。在 % 与该类型码字符之间，你可以做如下任意操作（或什么都不做）：

- 提供一个键名（key name），用于对表达式右侧的字典做索引。
- 列出标志（flags）：指定零填充（0）、左对齐（-）、数值符号（+），或在正数前留空格、负数前加 -（空格标志），其中 - 覆盖 0，+ 覆盖空格。
- 给出被替换文本的总最小字段宽度（field width）。
- 设置浮点数小数点后显示的数字位数（精度 precision）。width 与 precision 两项也可写成，表示它们应从表达式右侧输入值中的下一项动态取值（当运行期才知道数值时很有用，但使用 `keyname` 时不可用）。而且如果你用不上这些额外工具，格式字符串里简单的 `%s` 就会被相应 `value` 的默认打印字符串替换，与类型无关。

深度理解

·核心概念：`%` 目标的完整形态 `%[(key)][flags][width][.precision]typecode`——从“是什么类型”（`typecode`）到“怎么排”（`flags/width/precision`）的一整套声明式排版。

·底层实现：`%` 的 C 实现 `unicodeformat` 解析每个目标：先匹配可选的（`key`），再解析 `flags`、`width`（支持动态取）、`precision`，最后 `type code`，然后据类型调用整数转换（`d/o/x`）、浮点转换（`e/f/g`）或 `repr/str/ascii`。

·设计原因：声明式排版把"对齐、填零、精度"这类易错逻辑编进格式串，避免写辅助循环；支持运行时才知道的尺寸。

·实际问题：打印表格对齐列、生成定宽报告、浮点保留 N 位小数（%.2f 四舍五入）、金额千分位等。

·初学者误区：(1) %d 对浮点会截断或抛错（'%d' % 3.5 会截成 3，但它需要整数或可转整数）；(2) 忘了 %% 转义字面百分号——'100%%' % () 才输出 100%；(3) width 与 precision 位置不能颠倒（%.2f 与 %6.2f 有别）。

代码分析

```
>>> x = 1234
>>> 'integers: ...%d...%-6d...%06d' % (x, x, x)
'integers: ...1234...1234 ...001234'
```

解释器如何处理：三个目标分别演示三种排版：%d 默认右对齐无填充输出 1234；%-6d 左对齐、字段宽 6 → '1234'（右补空格）；%06d 右对齐、宽 6、零填充 → '001234'。C 层按 flags 决定填充字符与对齐方向，不足宽度时填充，超过宽度则原样输出（width 是"最小"）。设计思想：填充与对齐是排版声明，解释器负责机械执行；这些规则与后面 .format 的 :06d、f-string 的 {:06d} 完全同源——理解 % 就等于为后两者打好地基。

7.8.3 高级格式化表达式示例 (Advanced formatting expression examples)

英文原文

Formatting target syntax is documented in full in the Python standard manuals and other reference resources, but to demonstrate common usage, let's explore a few examples. The first formats integers using the default, and then in a six-character field with left justification and zero padding:

```
python >> x = 1234 >> res = 'integers:...%d...%-6d...%06d' % (x, x, x) >> res'integers:...1234...1234...001234'
```

The %e, %f, and %g formats display floating-point numbers in different ways, as the following interaction demonstrates—%E is the same as %e but the exponent is uppercase, and g chooses formats by number content (it's formally defined to use exponential format if the exponent is less than -4 or not less than precision, and decimal format f otherwise, with a default minimum total-digits precision of 6; no, really!):

```
python >> x = 1.23456789 >> x # 默认 REPL 显示 1.23456789 >> '%e | %f | %g' % (x, x, x)'1.234568e+00 | 1.234568 | 1.23457' >> '%E' % x'1.234568E+00'
```

For floating-point numbers, you can achieve a variety of additional formatting effects by specifying left justification, zero padding, numeric signs, total field width, and digits after the decimal point. For simpler tasks, you might get by with simply converting to string

gs with a %s type code or the str built-in function we used earlier:

```
python >> '%-6.2f | %05.2f | %+06.1f' % (x, x, x) '1.23 |  
01.23 | +001.2' >> '%s' % x, str(x) ('1.23456789', '1.234  
56789')
```

When sizes are not known until runtime, you can use a dynamically computed width and precision by specifying them with a in the format string to force their values to be taken from the next item in the inputs to the right of the % operator—the 4 in the tuple here gives precision:

```
python >> '%f, %.2f, %.f' % (1/3.0, 1/3.0, 4, 1/3.0) '0.33  
3333, 0.33, 0.3333'
```

As usual, experiment with some of these examples and operations on your own for more insight.

中文翻译

格式化目标语法在 Python 标准手册与其他参考资源里有完整记载，但为了演示常见用法，我们来探索几个例子。第一个用默认方式格式化整数，然后在六字符字段中左对齐并零填充：python >>> x = 1234 >>> res = 'integers: ...%d ...%-6d...%06d' % (x, x, x) >>> res 'integers: ...1234...1234 ...001234' %e、%f、%g 三种格式以不同方式显示浮点数，如下交互所示——%E 与 %e 相同但指数大写，而 g 按数字内容选择格式（正式定义：若指数小于 -4 或不小于精度，用指数格式 e，否则用十进制格式 f，默认最小总位数精度为 6；没错，就是这样！）：python >>> x = 1.23

456789 >>> x # 默认 REPL 显示1.23456789 >>> '%e | %f | %g' % (x, x, x)'1.234568e+00 | 1.234568 | 1.23457'
>>> '%E' % x'1.234568E+00'对浮点数，你可以通过指定左对齐、零填充、数值符号、总字段宽、小数点后位数，实现各种额外的格式化效果。对更简单的任务，用 %s 类型码或之前用过的 str 内置函数直接转字符串即可：python >>> '%-6.2f | %05.2f | %+06.1f' % (x, x, x)'1.23 | 01.23 | +001.2'>>> '%s' % x, str(x) ('1.23456789','1.23456789')
当尺寸要到运行期才知道时，可以用格式串中的指定动态计算宽度和精度，强制从 % 运算符右侧输入中的下一项取值——这里元组里的 4 提供精度：python >>> '%f, %.2f, %.f' % (1/3.0, 1/3.0, 4, 1/3.0)'0.333333, 0.33, 0.3333'老规矩，多对这些例子和操作亲自做实验，会更有心得。

深度理解

·核心概念：浮点的三种“人格”——%e（科学计数）、%f（定点小数）、%g（智能二选一）。%g 的取舍规则是“指数落进合理区间就用定点，否则用科学计数”，并去掉多余零。

·底层实现：%e/%f 走 C 的 PyOSdoubletostring（风格 g/f），按精度取四舍五入；%g 内部先以最短表示检查指数再决定子格式。%.2f 的精度作用于小数位。

·设计原因：同一数值在不同语境要不同的“长相”（科学论文用 e，报表用 f，宽松展示用 g），全交给一条格式；让宽度/精度能在运行期注入（列宽由数据决定）。

·实际问题：0.1+0.2 显示为 0.300000000000000004 时用 %.2f 安抚用户；金额/百分比对齐列；%+06.1f 生成带

符号定宽报文。

·初学者误区：(1) %g 会丢尾零——想展示固定小数位就用 %f；(2) 想输出百分号忘了 %%；(3) %05.2f 的顺序是 0 填充 + 宽 5 + 精度 2，误记宽度和精度目标。(4) 1/3.0 精度默认 6 位，%f 显示 0.333333。

代码分析

```
>>> '%-6.2f | %05.2f | %+06.1f' % (1.23456789, 1.23456789, 1.23...
'1.23 | 01.23 | +001.2'
>>> '%f, %.2f, %.*f' % (1/3.0, 1/3.0, 4, 1/3.0)
'0.333333, 0.33, 0.3333'
```

解释器如何处理：第一行三目标拆解——%-6.2f：左对齐、宽 6、2 位小数 ('1.23'，右补空格)；%05.2f：右对齐、宽 5、零填充、2 位小数 → '01.23'；%+06.1f：正号显示 +、宽 6、零填充、1 位小数 → '+001.2'。第二行演示：%f 的第 4 顺位值 4 被当作精度，输出 4 位小数 0.3333，而普通 %f 固定 6 位 0.333333、%.2f 固定 2 位 0.33。可见是"运行期才知道的排版参数"的注入管道。设计思想：格式化把"自适应宽度"预留给数据驱动的打印需求。

7.8.4 基于字典的格式化表达式 (Dictionary-based formatting expressions)

英文原文

As a more advanced extension, % string formatting also allows conversion targets on the left to refer to th

e keys in a dictionary coded on the right and use the corresponding values. Syntactically, this form requires () key references on the left of %, and a single dictionary (or other mapping) on the right. Functionally, it allows formatting to be used as a basic template tool. You've met dictionaries only briefly thus far in Chapter 4, but the following demos the idea:

```
python >>> '%(qty)s more %(tool)s' % {'qty': 1, 'tool': 'formatter'}
1 more formatter
```

Here, the (qty) and (tool) in the format string on the left refer to keys in the dictionary literal on the right and fetch their associated values. Programs that generate text such as HTML or XML often use this form: they build up a dictionary of values and substitute them all at once with a single formatting expression, using key references and a template string either loaded from a file or coded in the script. The following demos the idea (notice that its first comment is above the triple quote to keep it out of the string, and "..." prompts may not appear in your REPL):

```
python >>> # 带替换目标的模板
>>> reply = """ ... Hello %(name)s! ... Welcome to %(year)s ... """
>>> values = {'name': 'Pat', 'year': 2024} # 构建要替换的值
>>> print(reply % values) # 执行替换
Hello Pat! Welcome to 2024
```

This trick is also sometimes used in conjunction with the vars built-in function, which returns a dictionary c

containing all the variables that exist in the place it is called:

```
python >>> name = 'Pat'>>> year = 2024 >>> vars()
{'name': 'Pat', 'year': 2024, ...还有其他 Python 设置的内
建名称...}
```

When used on the right side of a format operation, this allows the format string to refer to variables by name—using dictionary-key syntax:

```
python >>> '%(name)s from %(year)s' % vars() # 变量
就是 vars() 里的键'Pat from 2024'
```

Although formatting expressions are positional by nature, dictionaries also allow them to reuse values more than once (see Chapter 5 for another demo of this in action):

```
python >>> '%(value)f, %(value).2f, %(value).f' % ({'va
lue': 1 / 3.0})'0.333333, 0.33, 0'
```

We'll study dictionaries in more depth in Chapter 8. See also "Hex, Octal, and Binary" for examples that convert to hexadecimal and octal number strings with the `%x` and `%o` formatting expression target codes, which we won't repeat here. Additional formatting expression examples also appear ahead as comparisons to the formatting method and f-string—the first of which is this chapter's next topic.

中文翻译

作为更高级的扩展，% 字符串格式化还允许左边的转换目标引用右边字典中的键（key），并使用与之对应的值。语法上，这种形式要求 % 左边有 () 键引用、右边放一个字典（或其它映射）。功能上，它让格式化可当作基础模板（template）工具。你到目前为止只在第 4 章粗浅见过字典，但下面演示了这个思想：

```
python >>> '%(qty)s more %(tool)s' % {'qty': 1, 'tool': 'formatter'}
1 more formatter
```

这里，左边格式字符串中的 (qty) 和 (tool) 引用右边字典变量中的键，并取出其关联值。生成 HTML、XML 等文本的程序常用这种形式：它们构建一个值的字典，用一条格式化表达式一次性替换，键引用配一个模板字符串（从文件加载或在脚本中写成）。下面演示这个思想（注意：第一条注释放在三引号上方以免混进字符串；"..."提示符在你的 REPL 里可能不出现）：

```
python >>> # 带替换目标的模板
>>> reply = """ ... Hello %(name)s! ... Welcome to %(year)s ... """
>>> values = {'name': 'Pat', 'year': 2024} # 构建要替换的值
>>> print(reply % values) # 执行替换
Hello Pat! Welcome to 2024
```

这个技巧有时还结合内置 vars 函数使用，它返回一个包含调用位置上所有变量的字典：

```
python >>> name = 'Pat'
>>> year = 2024
>>> vars()
{'name': 'Pat', 'year': 2024, ... 还有其他 Python 设置的内置名称...}
```

当用在格式化运算右侧时，它允许格式字符串按名字引用变量——用字典键语法：

```
python >>> '%(name)s from %(year)s' % vars()
Pat from 2024
```

虽然格式化表达式本质上是按位置的，但字典也允许它们多次复用某个值（第 5 章有另一个实际演示）：

hon >>> '%(value)f, %(value).2f, %(value).f' % ({'value': 1 / 3.0})'0.333333, 0.33, 0'我们会在第 8 章更深入地研究字典。此外，详见“十六进制、八进制与二进制”一节，那里有如何用格式化表达式目标码 %x、%o 转成十六进制和八进制数字字符串的示例，这里不再重复。更多格式化表达式示例也作为与格式化方法、f-string 的对比出现在后面——其中第一个就是本章下一主题。

深度理解

- 核心概念：%(key)s 让替换按名匹配而非按位匹配——值放字典、键指定位置。天然支持“值的复用”（同一键多处使用）与“模板与数据分离”。
- 底层实现：str.mod 解析左侧 (key) 后，把右侧当作映射 (PyMappingGetItem)，每个目标按键取值再照 type code 格式化；右侧不是字典时抛 TypeError。vars() 返回当前位置命名空间字典（模块作用域下就是 globals()），于是“变量即键”。
- 设计原因：HTML/XML 模板常要把一堆值灌进多处占位，位置匹配不可靠、按键匹配可读且自动复用；vars() 顺手把“局部作用域”当映射喂给模板。
- 实际问题：邮件/报告模板、多语言文案、osgi 风格的超高阶用法少见但存在；把字典展开与 %(name)s 是两种背道而驰的“命名替换”。
- 初学者误区：(1) 忘了 %(qty)s 后面的 s——只有 %(qty)s 才是完整目标，%(qty) 会报错；(2) 右侧必须是字典/映射，单个值不行；(3) vars() 的结果含很多无关名，且不

保证顺序——模板最好只读无歧义键。

代码分析

```
>>> reply = """
... Hello %(name)s!
... Welcome to %(year)s
... """
>>> values = {'name': 'Pat', 'year': 2024}
>>> print(reply % values)
Hello Pat!
Welcome to 2024
```

解释器如何处理：reply % values 中右侧 values 是字典，str.mod 进入"映射模式"：扫描模板发现 %(name)s、%(year)s，分别对字典做键名查找得到'Pat'、2024，再按 s 类型码格式化（2024 被 str 化成'2024'），最后拼接含 \n 的多行字符串，print 原样输出。注意模板内的换行、缩进都被保留——它是三引号文本，不是注释。设计思想：模板是数据、值是数据、格式化是函数——这种"字典驱动替换"是把字符串操作与数据结构（字典）相结合的桥梁，也是第 8 章字典学完之后回头重读会产生更多共鸣的内容。

7.9 字符串格式化方法 (The String-Formatting Method)

英文原文

As noted, Python 3.0 added a second way to format strings that some see as more Python specific. Unlike f

formatting expressions, the formatting method is not as closely based upon the C language's "printf" model; is sometimes more explicit in intent; and avoids the value-or-tuple quirk of %: substituted values are just arguments, whether one or many. On the other hand, the new technique still relies on "printf" concepts like type codes and formatting specifications. Moreover, it largely overlaps with formatting expressions; often yields more verbose code; and in practice can be just as complex in most roles. And if we're all being honest, the value-or-tuple quirk of the % expression is much more of a concern in theory than practice. Luckily, the two are similar enough that many core concepts overlap.

中文翻译

如前所述，Python 3.0 增加了第二种格式化字符串的方式，一些人认为它更"Python 化"。与格式化表达式不同，格式化方法不那么贴近 C 语言的 "printf" 模型；意图有时更显式；并避开了 % 的"单值还是元组"怪癖：被替换的值就是参数，无论是一个还是多个。另一方面，新技术仍然依赖 "printf" 概念，如类型码与格式说明。而且它与格式化表达式大量重叠；代码往往更冗长；实践中在多数角色里可能一样复杂。老实说，% 表达式的"单值还是元组"怪癖更多是理论上的担忧，实践里没那么要紧。好在两者足够相似，许多核心概念相通。

深度理解

- 核心概念：`.format()` 把"模板方法调用化"——模板是主体字符串，值是参数。消灭了 `%` 的元组/单值歧义。
- 底层实现：`str.format` 调用 `PyUnicodeFormatAdvanced`：解析 `{}` 目标（位置/关键字/属性/索引）、按 `!s/!r/!a` 或 `:` 格式说明排版；参数经 `PyObjectCall` 传进来，关键字参数经 `kwargs`。
- 设计原因：命名替换（`{name}`）与属性/下标取值的语法比 `%` 的 `%(name)s` 更接近 Python 本体表达式语法。
- 实际问题：大量遗留代码与某些库的默认格式化仍是 `.format`；"`{}` `{}`" 的简单场景它比 `%` 更清爽。
- 初学者误区：以为 `.format` 比 `%` 一定更好——两者各有适用场景；`{0}` 与 `{}` 混用有规则（相对位置必须按序）。`f-string` 出现后，新代码首选 `f-string`。

7.9.1 格式化方法基础 (Formatting method basics)

英文原文

The string object's `format` method at the heart of this option is based on function call syntax, instead of an expression. Specifically, it uses the call's subject string as a template and takes any number of arguments that represent values to be substituted according to the template.

This option requires knowledge of functions and calls but is mostly straightforward. Within the subject string, curly braces designate substitution targets and arguments to be inserted—either by relative position (`{}`), absolute position (e.g., `{1}`), or keyword (e.g., `{name}`).

As you'll learn when we explore argument passing in depth in Chapter 18, arguments to functions and methods may be passed by position (e.g., value) or keyword (e.g., name=value), and Python's ability to collect arbitrarily many arguments allows for general method-call patterns. As initial examples:

```
python >> template = '{}', '{}', and '{} ' # 相对位置 >> template.format('expr', 'method', 'fstring')
expr, method, and fstring' >> template = '{0}', '{1}', and '{2}' # 绝对位置
>> template.format('expr', 'method', 'fstring')
expr, method, and fstring' >> template = '{first}', '{second}', and '{third}' # 关键字名
>> template.format(first='expr', second='method', third='fstring')
expr, method, and fstring' >> template = '{first}', '{0}', and '{third}' # 组合 ({0} 或 {})
>> template.format('method', first='expr', third='fstring')
expr, method, and fstring'
```

By comparison, the last section's formatting expression can be a bit more concise, but uses dictionaries instead of keyword arguments for named references, and as you'll see in a moment doesn't allow quite as much flexibility for value sources in the template string.

ng itself (which may be an asset or liability, depending on your perspective):

```
python >> template = '%s, %s, and %s' # 等价的 %s 写法
>> template % ('expr','method','fstring')'expr, method, and fstring'
>> template = '%(first)s, %(second)s, and %(third)s'
>> template % dict(first='expr', second='method', third='fstring')'expr, method, and fstring'
```

Note the use of `dict()` to make a dictionary from keyword arguments here, introduced in Chapter 4 and covered in full in Chapter 8; it's an often less cluttered alternative to the `{...}` literal. Naturally, the subject string in the format method call can also be a literal that creates a temporary string, and arbitrary object types can be substituted at targets much like the expression's `%s` code:

```
python >> '{pi}, {} and {years}'.format(62, pi=3.14, years=[1995, 2024])
'3.14, 62 and [1995, 2024]'
```

Just as with the `%` expression and other string methods, `format` creates and returns a new string object, which can be printed immediately or saved for further work (as another reminder, strings are immutable, so `format` really must make a new object). String formatting in any of its forms is not just for display:

```
python >> X = '{pi}, {} and {years}'.format(62, pi=3.14, years=[1995, 2024])
>> X'3.14, 62 and [1995, 2024]'
>> X.split(' and') ['3.14, 62', '[1995, 2024]']
>> Y = X.replace('and','but under no circumstances')
>> Y'3.14, 62 but under no circumstances'
```

but under no circumstances [1995, 2024]'

中文翻译

这一选项的核心——字符串对象的 `format` 方法——基于函数调用语法，而非表达式。具体说，它把调用主体字符串当模板，接受任意数量的参数，代表按模板替换的值。这个选项需要了解函数与调用，但大多很直接。在主体字符串内部，花括号标记替换目标与要插入的参数——可按相对位置（`{}`）、绝对位置（如 `{1}`）或关键字（如 `{name}`）。如第 18 章深入讲解参数传递时你会学到的：函数和方法的参数可按位置（如 `value`）或关键字（如 `name=value`）传递，而 Python 收集任意数量参数的能力支持通用的方法调用模式。先看几个入门示例：

```
python >>> template = '{}',  
{}, and {}' # 相对位置>>> template.format('expr','meth  
od','fstring')'expr, method, and fstring'>>> template  
='{0}, {1}, and {2}' # 绝对位置>>> template.format('ex  
pr','method','fstring')'expr, method, and fstring'>>> t  
emplate = '{first}, {second}, and {third}' # 关键字名>>>  
template.format(first='expr', second='method', third  
='fstring')'expr, method, and fstring'>>> template =  
{first}, {0}, and {third}' # 组合（{0} 或 {}）>>> templat  
e.format('method', first='expr', third='fstring')'expr,  
method, and fstring'相比之下，上一节的格式化表达式可以  
更简洁，但命名引用要用字典而非关键字参数，而且如你  
马上会看到的，它对模板字符串内部“值来源”的灵活性略逊  
（这是优点还是缺点，取决于你的立场）：python >>> t  
emplate = '%s, %s, and %s' # 等价的 %s 写法>>> tem
```

plate % ('expr','method','fstring')'expr, method, and fstring'>>> template = '%(first)s, %(second)s, and %(third)s'>>> template % dict(first='expr', second='method', third='fstring')'expr, method, and fstring'注意这里用 dict() 从关键字参数构造字典——第 4 章介绍、第 8 章全面讲解；它通常是比 {...} 字面量更清爽的选择。自然，format 方法调用中的主体字符串也可以是直接写出的字面量，创建临时字符串；任意对象类型都可以像表达式的 %s 码一样被替换进目标：python >>> '{pi}, {} and {years}'.format(62, pi=3.14, years=[1995, 2024])'3.14, 62 and [1995, 2024]'与 % 表达式和其他字符串方法一样，format 创建并返回一个新字符串对象，可以立即打印或保存起来继续加工（再次提醒：字符串不可变，所以 format 真的必须造新对象）。任何形式的字符串格式化都不只是为了显示：python >>> X = '{pi}, {} and {years}'.format(62, pi=3.14, years=[1995, 2024])>>> X'3.14, 62 and [1995, 2024] '>>> X.split(' and ') ['3.14, 62','[1995, 2024]']>>> Y = X.replace('and','but under no circumstances')>>> Y'3.14, 62 but under no circumstances [1995, 2024]'

深度理解

·核心概念：.format 的目标标识三位一体——{}（相对）、{n}（绝对）、{name}（关键字），且可混用。值来源 = 位置实参 + 关键字实参。

·底层实现：str.format 的实现分两阶段：解析模板生成字段树（每字段含字段名、转换标志、格式说明），然后从实参（args+kwargs 合并视图）取值并排版，最终按字段切分文本拼接。{} 相对位置在解析期被自动编号。

·设计原因：命名与位置并存的替换系统让"模板可读、参数可复"；`.split()/replace()` 对格式化结果继续加工，证明它是普通字符串。

·实际问题：报表、日志、SQL 占位符生成、把格式化的结果当作普通文本再切分/替换的流水线。

·初学者误区：(1) `{}` 相对位置不能跳跃/乱序混用 (`'{1} {}'` 合法但 `'{1} {}'` 后再加 `{0}` 有规则要守)；(2) 关键字名与"字典键"是两回事——想要字典键要写 `{0[key]}`；(3) 忘记 `.format` 需要方法调用括号。

代码分析

```
>>> '{pi}, {} and {years}'.format(62, pi=3.14, years=[1995, 2024])
'3.14, 62 and [1995, 2024]'
>>> X.split(' and ')          #
['3.14, 62', '[1995, 2024]']
```

解释器如何处理：解析模板得到字段：`{pi}`（关键字）、`{}`（相对，编号为 0）、`{years}`（关键字）。实参 62 与 `pi=3.14`、`years=[1995,2024]` 被组合为可查找空间：位置 `0 → 62`，关键字 `pi → 3.14`、`years → 列表`。每个字段按 `str()` 默认格式化（3.14 的 `str` 是 `'3.14'`，列表的 `str` 是 `'[1995, 2024]'`），最后按原模板文本顺序装配。之后 `X.split(' and ')` 把它当普通字符串切分——格式化与后续文本操作无缝衔接。设计思想：模板描述"骨架"，参数提供"血肉"，结果仍是第一等公民字符串。

7.9.2 添加键、属性与偏移 (Adding keys, attributes, and offsets)

英文原文

Like % formatting expressions, format calls can become more complex to support more advanced usage. For instance, format strings can name object attributes and dictionary keys—as in normal Python syntax, square brackets name dictionary keys and dots denote object attributes of an item referenced by position or keyword.

The first of the following examples indexes a dictionary on the key kind and then fetches the attribute platform from the already imported sys module object. The second does the same, but names the objects by keyword instead of position:

```
python >> import sys # 标准库模块 >> 'This {1[kind]} runs {0.platform}'.format(sys, {'kind':'laptop'}) 'This laptop runs darwin' >> 'This {map[kind]} runs {sys.platform}'.format(sys=sys, map={'kind':'phone'}) 'This phone runs linux'
```

Square brackets in format strings can also name offsets to index lists (and other sequences), but only a single positive offset works syntactically within each [], so this feature is not as general as you might think.

To reference negative offsets or slices, or to use arbitrary expression results in general, you must run expressions outside the format string itself, just as you would for % expressions (note the use of parts here to unpack a tuple's items into individual function arguments; you'll learn more about this form when we study function arguments in Chapter 18):

```
python >> somelist = list('HACK') >> somelist ['H','A',  
, 'C', 'K'] >> 'zero={0[0]}, two={0[2]}'.format(somelist) 'z  
ero=H, two=C' >> 'first={}, last={}'.format(somelist[0],  
somelist[-1]) # fmt 内 [-1] 会失败 'first=H, last=K' >> p  
arts = (somelist[0], somelist[-1], somelist[1:3]) # fmt  
内 [1:3] 会失败 >> 'first={}, last={}, middle={}'.format(p  
arts) # 或 {0}, {1}, {2} "first=H, last=K, middle=['A','C']"
```

If you simply cannot do without full generality inside format strings, stay tuned for the f-string and its arbitrary nested expressions (albeit in a code literal instead of a method object, and at the cost of more redundancy and less utility).

中文翻译

与 % 格式化表达式一样，format 调用可以变得更复杂以支持更高级的用法。例如，格式字符串可以命名对象属性和字典键——和普通 Python 语法一样，方括号命名字典键、点号表示按位置或关键字引用的条目的对象属性。下面第一个示例按键 kind 索引一个字典，然后从已导入的 sys 模块对象上取属性 platform。第二个做同样的事，但对象按关键字而非位置命名：python >>> import sys # 标准库模

块>>>'This {1[kind]} runs {0.platform}'.format(sys, {'kind':'laptop'})'This laptop runs darwin'>>>'This {map[kind]} runs {sys.platform}'.format(sys=sys, map={'kind':'phone'})'This phone runs linux'格式字符串中的方括号还可以命名偏移来索引列表（及其他序列），但每个 [] 在语法上只支持单个正偏移，所以这个特性没有你想的那么通用。要引用负偏移或切片，或一般性地使用任意表达式结果，你必须在格式字符串之外运行表达式——正如 % 表达式那样（注意这里用 parts 把元组元素拆成单独的函数参数；第 18 章研究函数参数时会学这种形式）：python >>> somelist = list('HACK') >>> somelist ['H','A','C','K'] >>>'zero={0[0]}, two={0[2]}'.format(somelist)'zero=H, two=C'>>>'first={}, last={} '.format(somelist[0], somelist[-1]) # fmt 内 [-1] 会失败'first=H, last=K'>>> parts = (somelist[0], somelist[-1], somelist[1:3]) # fmt 内 [1:3] 会失败>>>'first={}, last={}, middle={} '.format(parts) # 或 {0}, {1}, {2} "first=H, last=K, middle=['A','C']" 如果你就是离不开格式字符串内部的完全通用性，请继续关注 f-string 及其任意的嵌套表达式（不过是在代码字面量而非方法对象里，代价是更多冗余、更少可复用性）。

深度理解

- 核心概念：.format 字段名支持 0.platform（属性）与 0[key]（下标）两级取址——但下标只能是单个非负整数，不支持负索引、切片、表达式。
- 底层实现：字段名解析器把 0.platform 拆成字段0 → 取属性 platform、0[kind] 拆成字段0 → 按 key 取下标，通过 getattr/getitem 取值；负下标 [-1] 直接解析失败抛

ValueError（不是运行时求值）。

·设计原因：字段名语法刻意保守——字符串模板保持简单可解析；复杂取值（切片、负索引、计算）鼓励先算好再传参，也让模板引擎职责清晰。

·实际问题：从配置字典/对象拼消息（{config[name]}）、从模块取常量（{sys.platform}）、列表元素展平后按位填。

·初学者误区：(1) 试图在格式串里写 {0[-1]}、{0[1:3]}——语法不支持，必须提前算；(2) parts 解包是 Python 语法不是格式串语法，两者不要混淆。

代码分析

```
>>> parts = (somelist[0], somelist[-1], somelist[1:3])    #
>>> 'first={}, last={}, middle={}'.format(*parts)
"first=H, last=K, middle=['A', 'C']"
```

解释器如何处理：parts 在调用点把元组 ('H','K', ['A','C']) 展开为三个位置实参；format 的三个 {} 相对编号 0/1/2 依次取 'H'、'K'、['A','C']，各自 str() 后装配——['A','C'] 的 str 是 "['A','C']"。设计思想：复杂取值在前端完成，模板只做搬运——这保证格式串永远是简单的字符串数据，可安全存储在配置文件里。

7.9.3 格式化方法自定义格式 (Formatting method custom formats)

英文原文

Another similarity with % expressions is that format lets you can achieve more specific layouts with extra format-string syntax. For the formatting method, we use a colon after the possibly empty substitution target's identification, followed by a format specifier that can name the field size, justification, and a specific type code. Here's the formal structure of what can appear as a substitution target in a format string—its four parts are all optional (denoted by surrounding [] here, which aren't coded in the format string) and must appear without intervening spaces: `{[fieldname][component][!conversionflag][:formatspec]}` Text outside a {} substitution target is taken literally and may use doubled {{ and }} to escape braces (each is replaced with a single brace). Within a {} substitution target:- **fieldname**: Is an optional number or keyword identifying an argument, which may be omitted to reference arguments by relative position.- **component**: Is a string of zero or more `.name` or `[index]` (brackets required!) references, which are used to fetch attributes and indexed values of an argument, and may be omitted to use the whole argument value.- **conversionflag**: Starts with a ! if present, which is followed by `s`, `r`, or `a` to call `str`, `repr`, or `ascii` built-in functions on the value, respectively (this may bypass the value's normal formatting).- **formatspec**: Starts with a `:` if present, followed

by text that specifies how the value should be presented, including details such as field width, alignment, padding, decimal precision, and so on, and ends with an optional type code. The format spec component after the colon character has a rich format all its own and is formally described as follows. As you'll see later, this part is reused by f-strings (again, [...] in this denotes an optional component whose square brackets are not coded literally, and spaces aren't allowed between parts but may appear within some): `[[fill]align][sign][z][#][0][width][grouping][.precision][typecode]` Within this format spec part of the `{}` substitution target, the salient parts are these:-

- `fill`: Can be any fill character other than `{` or `}`.
- `align`: May be `<`, `>`, `=`, or `^`, for left alignment, right alignment, padding after a sign character, or centered alignment.
- `sign`: Can be `+` (to sign all numbers), `-` (to sign only negatives), or a space (to use a space for positives).
- `grouping`: May be `,` or `_` to request a comma or underscore separator, added for thousands in decimal number type codes, and for four-digit groups in nondecimal number type codes (which support only `f`).
- `width` and `precision`: Similar to those in the `%` expression of the preceding section, as shown by examples ahead.
- `typecode`: Similar to those in the `%` expression, with the exceptions described below this list.
- `Others`: For numbers, a `0` before width enables sign-aware zero-padding (redundantly with some fill usage), and `#` invokes an alternate form (

e.g., adding 0b and 0X prefixes for binary and hex type codes b and X). The formatspec may also contain nested {} substitution targets for any of its parts, to use argument-list values dynamically (much like the in formatting expressions). These nested {} use fieldname to identify arguments from which values are pulled, and their formatted results are used in place of the nested {}. Nesting may be only one level deep, and f-strings (ahead) use an arbitrary expression instead of fieldname in a nested {}. The method's typecode options largely overlap with those used in % expressions and listed earlier in Table 7-4, but the formatting method adds a b to display integers in binary format (much like using the bin built-in), adds a % to display percentages, uses only d for base-10 integers (i and u are unused), uses ! conversion flags for some cases, and requires a string object for s (to flexibly allow any type like the expression's %s, either omit the type code or formatpec in full, or use a !s conversion flag as described earlier). See Python's library manual for more on substitution syntax that we'll omit here. In addition to the string's format method, a single object may also be formatted with the format(object, formatpec) built-in function (which the method uses internally), and may be customized in user-defined classes with the format operator-overloading method (see Part VI). Different objects may use different format specifiers, but most follow norms.

中文翻译

与 % 表达式的另一相似点：format 允许你用额外的格式串语法实现更具体的排版。对格式化方法，我们在（可能为空的）替换目标标识后加冒号，后面跟一个格式说明符（format specifier），它可以指定字段大小、对齐方式和特定类型码。下面是格式字符串中替换目标可出现的完整形式结构——它的四个部分都可选（此处用周围 [] 表示，方括号并不写在格式串中），且各部分之间不能有空格：{[fieldname][component]![conversionflag][:formatspec]} {} 替换目标之外的文本按字面取用，并可用加倍的 {{ 和 }} 转义花括号（各替换为单个花括号）。在一个 {} 替换目标内部：

- `fieldname`（字段名）：可选的数字或关键字，标识某个参数；省略则以相对位置引用参数。
- `component`（分量）：零个或多个 `.name` 或 `[index]`（方括号必须有！）引用的字符串，用来取参数的属性或索引值；可省略以使用整个参数值。
- `conversionflag`（转换标志）：若存在则以 ! 开头，后面跟 `s`、`r` 或 `a`，分别对该值调用 `str`、`repr` 或 `ascii` 内置函数（这可能绕开该值正常的格式化）。
- `format spec`（格式说明）：若存在则以 : 开头，后接指定该值呈现方式的文本，包括字段宽度、对齐、填充、小数精度等细节，并以可选类型码结尾。冒号后的 `formatspec` 分量自成一套丰富格式，正式描述如下。稍后你会看到，这一部分也被 f-string 复用（再次说明，这里的 [...] 表示可选分量，其方括号不字面写入代码，各部分之间不允许空格，但某些部分内部可以出现空格）：[[`fill`]`align`][`sign`][`z`][`#`][`0`][`width`][`grouping`][`.precision`][`typecode`]在这个 {} 替换目标内的 `formatspec` 部分，要点如下：
- `fill`（填充字符）：可以是

除 {、} 外的任意填充字符。- align (对齐)：可以是 <、>、=、^，分别表示左对齐、右对齐、在符号后填充、居中对齐。- sign (符号)：可以是 + (所有数字都带符号)、- (仅负数带符号)、或空格 (正数用空格占位)。- grouping (分组)：可以是，或，请求千位分隔；十进制数字类型码按千分组，非十进制数字类型码按四位一组 (后者只支持)。- width 与 precision：与上一节 % 表达式中的类似，见后面的示例。- typecode (类型码)：与 % 表达式中的类似，例外情况见本清单之后。- 其他：对数字，width 前的 0 启用“感知符号的零填充” (与某些 fill 用法冗余)，# 调用替代形式 (例如为二进制和十六进制类型码 b、X 添加 0b、0X 前缀)。formatspec 的任一部分还可以包含嵌套的 {} 替换目标，以动态使用参数列表中的值 (很像格式化表达式中的)。这些嵌套 {} 用 fieldname 标识取值来源的参数，其格式化结果用于替代嵌套 {} 的位置。嵌套只允许一层深，而 f-string (在后面) 在嵌套 {} 中使用任意表达式而非 fieldname。方法的 typecode 选项与表 7-4 列出的 % 表达式大部分重叠，但格式化方法还增加了：b 以二进制显示整数 (很像内置 bin)，% 显示百分比，十进制整数只用 d (i、u 不使用)，某些情况用 ! 转换标志，且 s 需要字符串对象 (为了像表达式的 %s 那样灵活容纳任何类型，要么完全省略类型码或 formatspec，要么像前面说的用 !s 转换标志)。更多此处省略的替换语法参见 Python 库手册。除了字符串的 format 方法，单个对象也可以用 format(object, formatspec) 内置函数格式化 (方法内部就用它)，并可在用户自定义类中通过 format 运算符重载方法定制 (见第六部分)。不同对象可以使用不同的格式说明符，但大多遵循惯例。

深度理解

·核心概念：{} 目标的四段式 [fieldname][component][!conv][:spec] 与 format spec 的完整拼图 [[fill]align][sign][z][#][0][width][grouping][.precision][typecode]——这是三种格式化工具共享的排版词表，f-string 直接复用。

·底层实现：str.format 解析后对每个字段调用 object.format(spec)（等价于 format(value, spec)）。int/float/str 各自实现 format：数字按 align/sign/grouping/precision 排版，b/d/x 进制转换，, 千分位；字符串支持宽度与对齐。format 是用户自定义类型格式化能力的入口（返回 NotImplemented 则回退 str）。

·设计原因：把“排版”从“转换”中抽出、统一由 format 协议承载，让格式化成为对象自描述能力的一部分——与 str/repr 同族。

·实际问题：金额千分位 {:,.2f}、二进制标志位展示 {:08b}、表格式对齐 {:<10}、进度百分比 {:.1%}。

·初学者误区：(1) {} 不是语法——分组必须配类型码 ({:,.2f})；(2) !s 与 :s 位置不同（转换标志在冒号前）；(3) {/} 转义常被忘记，输出只剩单个花括号；(4) 以为 z (3.6+ 的负零处理) 常用——其实 {z} 很少见，明白存在即可。

代码分析

```
>>> '{0:10} = {1:10}'.format('text', 123.4567)      # 10
'text = 123.4567'
>>> '{0:>10} = {1:<10}'.format('text', 123.4567)     # /
' text = 123.4567 '
>>> '{:, .2f}'.format(12345.678)                    # +
'12,345.68'
>>> '{:b}'.format(255)                               #
'11111111'
```

解释器如何处理：每例都是 `str.format` → 解析 `format_spec` → 调 `int.format` 或 `float.format`。{0:>10} 中 > 把'text' 顶到右端、左侧补空格；{1:<10} 左对齐右侧补空格——宽度不足时不截断。{:, .2f} 先 round 到 2 位 (12345.68)、再插千分位逗号。{:b} 走整数的二进制转换路径（对应 `bin(255)` 去掉前缀'0b' 的输出）。`format(x, spec)` 内置函数与 `f'{x:spec}'` 最终都走 `type(x).format`——同一排版引擎、三种入口，这就是三种工具“铁人三项”的深层统一。

7.9.4 高级格式化方法示例 (Advanced formatting method examples)

英文原文

As you can tell, the syntax in formatting methods can be complex. Because your best ally in such cases is often the interactive prompt, let's turn to some examples.

In the following, {0:10} means the first positional argument in a field 10 characters wide; {1:<10} means th

e second positional argument left-justified in a 10-character-wide field; ^10 center aligns in 10; and {0.platform:>10} means the platform attribute of the first argument, right-justified in a 10-character-wide field (notice again the use of dict() to make a dictionary from keyword arguments):

```
python >> '{0:10} = {1:10}'.format('text', 123.4567) 'text = 123.4567'
>> '{0:>10} = {1:<10}'.format('text', 123.4567) 'text = 123.4567'
>> '{1[kind]:^10} = {0.platform:^10}'.format(sys, dict(kind='laptop')) 'laptop = darwin'
```

As demoed earlier, you can omit the argument number if you're selecting them from left to right—though this may make your code less explicit, thereby negating one of the purported pluses of format versus %. Code readers must count to match {}s to arguments off to the right (something the f-string's inline expressions wholly avoid):

```
python >> '{:10} = {:10}'.format('text', 123.4567) 'text = 123.4567'
```

Floating-point numbers support the same type codes and formatting specificity in formatting method calls as in % expressions. For instance, in the following {2:g} means the third argument formatted by default according to the "g" floating-point representation, {:.2f} designates the "f" floating-point format with just two decimal digits (and rounding), and {06.2f} denotes a

field with a width of six characters and zero padding on the left:

```
python >>'{:e}, {:.3e}, {2:g}'.format(3.14159, 3.14159, 3.14159)'3.141590e+00, 3.142e+00, 3.14159'>>'{:f}, {:.2f}, {:06.2f}'.format(3.14159, 3.14159, 3.14159)'3.141590, 3.14, 003.14'
```

Hex, octal, and binary formats are supported by the formatting method as well (% has all these except binary). In fact, string formatting is an alternative to some of the built-in functions that format integers to a given base:

```
python >>'{:X}, {:o}, {:b}'.format(255, 255, 255) # 十六进制、八进制、二进制'FF, 377, 11111111'>> hex(255), int('FF', 16), 0xFF # 其他十六进制互转方式 ('0xff', 255, 255)>> oct(255), int('377', 8), 0o377 # 其他八进制互转方式 ('0o377', 255, 255)>> bin(255), int('11111111', 2), 0b11111111 # 其他二进制互转方式 ('0b11111111', 255, 255)
```

The formatting method also supports separator insertions (but % currently does not): you can add commas and underscores between thousands groups in numbers, and underscores between four-digit groups in hex, octal, and binary formats, and absolute argument numbers let you reuse values passed in (% uses dictionaries to do the same):

```
python >>'{:,.2f}'.format(12345.678)'12,345.68'>>'{:0;}{0:}{1:x}{1:b}'.format(2 32, 0x1FFFF)'4,294,967,296 4'
```

```
294967296 1ffff 111111111111111111'
```

Formatting parameters can either be hardcoded in format strings or taken from the arguments list dynamically by nested format syntax—much like the syntax in formatting expressions' width and precision:

```
python >> '{:.4f}'.format(1 / 3.0) # 参数硬编码'0.3333'>
> '%.4f' % (1 / 3.0) # 表达式同样'0.3333'> > '{0:.{1}f}'.format(1 / 3.0, 4) # 从参数里取值'0.3333'> > '%.f' % (4, 1 / 3.0) # 表达式同样'0.3333'
```

In fact, format allows any component of a format specification string to be taken from arguments at runtime, rather than hardcoded at programming time (this is more general than %). In the following, a number is zero filled, left-justified, signed, twelve wide, comma separated, with two decimal digits—both statically and dynamically (and with and without argument numbering that nearly breaches this page's width limits!):

```
python >> '{0:0<+12,.2f}!'.format(1234.564)+'1,234.56000!'
```

```
>> '{0:{1}{2}{3}{4}{5}.{6}{7}}!'.format(1234.564, 0,'<','+', 12,',', 2,'f')+'1,234.56000!'
```

```
>> '{:{0}{1}{2}{3}{4}{5}{6}{7}}!'.format(1234.564, 0,'<','+', 12,',',',', 2,'f')+'1,234.56000!'
```

Finally, Python's built-in format function noted earlier can also be used to format a single item. It may be simpler than using the format method in this case, an

d is roughly similar to formatting one item with the % expression:

```
python >> '{:.2f}'.format(1.2345) # 字符串方法'1.23'>>
format(1.2345, '.2f') # 内置函数'1.23'>> '%.2f' % 1.2345
# 表达式'1.23'>> f'{1.2345:.2f}' # 预告: f-string'1.23'
```

Technically, the format built-in runs the subject object's format method, which the str.format method does internally for each formatted item. It's still more verbose than the original % expression's equivalent here, though, and the f-string alternative may best both when the value is a variable's name—which leads us to the next section.

中文翻译

如你所见，格式化方法中的语法可能很复杂。这种情况下最好的盟友往往是交互式提示符，所以直接看示例。下面：{0:10} 表示第一个位置参数放在宽 10 字符的字段里；{1:<10} 表示第二个位置参数在宽 10 的字段里左对齐；^10 在 10 个字符里居中对齐；{0.platform:>10} 表示第一个参数的 platform 属性在宽 10 的字段里右对齐（再次注意用 dict() 从关键字参数构造字典）：python >>> '{0:10} = {1:10}'.format('text', 123.4567)'text = 123.4567'>>> '{0:>10} = {1:<10}'.format('text', 123.4567)' text = 123.4567'>>> '{1[kind]:^10} = {0.platform:^10}'.format(sys, dict(kind='laptop'))' laptop = darwin'如前面演示的，如果参数是从左到右依次选取，可以省略参数编号——不过这会让代码不太显式，抵消了 format 相对 % 的一个宣称优点。读代码的人必须数一数 {} 对应右侧第几个参数（这正是 f

-string 内联表达式完全避开的) : python >>> '{:10} = {:10}'.format('text', 123.4567)'text = 123.4567'浮点数在格式化方法调用中支持与 % 表达式相同的类型码和格式化细节。例如, 下面 {2:g} 表示第三个参数按 "g" 浮点表示的默认方式格式化, {:.2f} 指定 "f" 浮点格式且只有两位小数 (含四舍五入), {:06.2f} 表示宽度 6、左侧零填充的字段 : python >>> '{0:e}, {1:.3e}, {2:g}'.format(3.14159, 3.14159, 3.14159)'3.141590e+00, 3.142e+00, 3.14159'>>> '{:f}, {:.2f}, {:06.2f}'.format(3.14159, 3.14159, 3.14159)'3.141590, 3.14, 003.14'格式化方法同样支持十六进制、八进制、二进制格式 (% 除了二进制都有)。实际上, 字符串格式化是某些"把整数格式化为指定进制"内置函数的替代品: python >>> '{:X}, {:o}, {:b}'.format(255, 255, 255) # 十六进制、八进制、二进制'FF, 377, 11111111'>>> hex(255), int('FF', 16), 0xFF # 其他十六进制互转方式('0xff', 255, 255) >>> oct(255), int('377', 8), 0o377 # 其他八进制互转方式('0o377', 255, 255) >>> bin(255), int('11111111', 2), 0b11111111 # 其他二进制互转方式('0b11111111', 255, 255) 格式化方法还支持分隔符插入 (而 % 目前不支持) : 你可以在数字的千位组之间加逗号和下划线, 在十六进制、八进制、二进制格式的四位组之间加下划线; 绝对参数编号让你能复用传入的值 (% 用字典做同样的事) : python >>> '{:,.2f}'.format(12345.678)'12,345.68'>>> '{0:,} {0:} {1:x} {1:b}'.format(2 32, 0x1FFFF)'4,294,967,296 4294967296 1ffff 11111111111111111111'格式化参数既可以硬编码在格式串里, 也可以通过嵌套格式语法从参数列表动态取值——很像格式化表达式 width/precision 中的语法: python >>> '{:.4f}'.format(1 / 3.0) # 参数硬

编码'0.3333'>>>'%.4f' % (1 / 3.0) # 表达式同样'0.3333'
>>>'{0:.{1}f}'.format(1 / 3.0, 4) # 从参数里取值'0.3333'
>>>'%.f' % (4, 1 / 3.0) # 表达式同样'0.3333'事实上，fo
rmat 允许把 formatspec 字符串的任意分量在运行期从参
数取值，而非编程期硬编码（这比 % 更通用）。下面这个
数字被零填充、左对齐、带符号、宽 12、逗号分隔、两位
小数——静态和动态各来一遍（并且带与不带参数编号，
后者几乎要顶破本页宽度限制！）：python >>> '{0:0<+1
2,.2f}'!.format(1234.564)+'1,234.56000!'>>> '{0:{1}{2}{
3}{4}{5}.{6}{7}}'!.format(1234.564, 0,'<','+', 12,',', 2,'f')'
'+1,234.56000!'>>> '{:{}{}{}{}{}{}{}'}'.format(1234.564,
0,'<','+', 12,',',',', 2,'f')+1,234.56000!'最后，前面提到的
Python 内置 format 函数也可以用来格式化单个条目。这种
场景下它可能比 format 方法更简单，大致相当于用 %
表达式格式化单个条目：python >>> '{:.2f}'.format(1.23
45) # 字符串方法'1.23'>>> format(1.2345,'.2f') # 内置
函数'1.23'>>>'%.2f' % 1.2345 # 表达式'1.23'>>> f'{1.2
345:.2f}' # 预告：f-string'1.23'严格讲，format 内置函数
运行主体对象的 format 方法，而 str.format 方法在内部
对每个格式化条目也做这件事。不过它仍然比 % 表达式对
应的写法冗长，而当值就是某个变量的名字时，f-string 方
案可能双双胜过它们——这就引向下一节。

深度理解

·核心概念: `.format` 的格式说明与 `%` 同源但更系统: 对齐 `< > ^ =`、符号 `+ -` 空格、分组 `,`、进制 `b o x X`、以及嵌套 `{ }` 动态注入任意 `formatspec` 分量。

·底层实现：嵌套 {} 在解析期被展开成对参数列表的索引取值（{0:{1}...} 中 {1} 取值 0、< 等后拼成完整 spec 再调 format）。format(x, spec) 内置函数与 str.format 共用同一 format 路径。

·设计原因：% 的只能动态化 width/precision，.format 的嵌套 {} 能动态化任意分量（填充、对齐、类型码），这是方法相对表达式更"可编程"之处。

·实际问题：列宽由运行时数据决定的报表（宽度作参数传入）、f-string 场景下的动态宽度、进制转换展示。

·初学者误区：(1) {0:06.2f} 里 0 在宽度前是零填充、06 和 .2f 顺序固定；(2) 嵌套 {} 是"格式串里的参数"，不是花括号转义，读起来困难是常态；(3) format() 与 .format() 两个名字、两种用法，容易写混。

代码分析

```
>>> '{0:{1}{2}{3}{4}{5}.{6}{7}}!'.format(1234.564, 0, '<', '+', ...  
'+', 234.56000!')
```

解释器如何处理：解析时发现外层字段 0 的 spec 内嵌了 7 个嵌套字段 {1}..{7}。运行期先对嵌套字段取值：0、'<'、'+'、12、'.'、2、'f'，拼成完整格式说明 0<+12,.2f，然后对值 1234.564 调用 float.format('0<+12,.2f')：+ 显示符号、< 左对齐、0 用零填充（对齐优先于 0）、宽 12、千分位、2 位小数 → '+1,234.56000'，再接字面 !。第三例'{:}{:}...' 连字段编号都省略，全部由相对位置提供。设计思想：格式说明本身也可以数据化——这套"规格即数据"的能力让 .format 成为三者中最可编程的一个。

7.10 F-String 格式化字面量 (The F-String Formatting Literal)

英文原文

If you've survived the formatting story this far, there's some good news: the third and last variant is mostly just a takeoff on the second. The f-string is a text-string literal that embeds substitution values in the format string itself, rather than listing them separately. The code it uses to specify custom formatting, though, is the same as that used for the formatting method. Hence, much of what you just learned for the method applies here in full. F-strings, added in Python 3.6, perform what's generally called string interpolation—replacing the text of an expression with the result of running it live, when the f-string itself is run. These expressions are coded inside the f-string and can be an arbitrary Python expression code. The effect isn't functionally different from listing replacement values after a % in the expression, or as arguments in the format method. Because they embed values where they are to be substituted, though, f-strings are often shorter and may seem easier to read to some observers. Syntactically, f-strings begin with the letter f (uppercase or lowercase, but usually the latter) before any string-literal form (single, double, or triple quotes). The f prefix

x may be combined with r in any order to code formatted raw strings that ignore backslashes (e.g., rf), and f-strings concatenate implicitly with an adjacent text-string literal of any kind (but use the f prefix on other concatenated literals if you want them to be f-strings too—even on continuation lines). As a preview of Chapter 37, the f cannot be combined with byte-string prefix b (which means that the f-string, like the format method but unlike the % expression, works only for text, not bytes); and cannot be mixed with the backward-compatible and seldom-used u (which would yield prefixes inappropriate for this family-oriented text).

中文翻译

如果你忍着读完了格式化这段故事，听点好消息：第三种也是最后一种变体基本上只是第二种的延伸。f-string (f 字符串) 是一种文本字符串字面量，把替换值嵌入格式字符串本身，而不是单独列出。不过，它用于指定自定义格式的代码与格式化方法相同。因此，你刚为方法学的绝大部分内容在这里完全适用。f-string 在 Python 3.6 加入，执行通常所称的字符串插值 (string interpolation) ——在 f-string 本身运行起来的当下，把某个表达式的文本替换为运行它的实况结果。这些表达式写在 f-string 内部，可以是任意的 Python 表达式代码。其效果与"在表达式里 % 之后列出替换值"或"作为 format 方法的参数"在功能上没有差别。不过，因为 f-string 把值嵌在它们要被替换的位置，它们往往更短，对某些读者来说也似乎更易读。语法上，f-string 以字母 f 开头 (大写小写均可，但通常小写)，置于任何

字符串字面量形式（单、双或三引号）之前。f 前缀可以与 r 以任意顺序组合，编码忽略反斜杠的"格式化原始字符串"（如 rf）；并且 f-string 会与相邻的任意文本字符串字面量隐式拼接（但如果你希望拼接的其他字面量也是 f-string，就要给它们也加上 f 前缀——即便在续行上也要）。作为第 37 章的预告：f 不能与字节串前缀 b 组合（这意味着 f-string 和 format 方法一样、但和 % 表达式不同，只对文本有效，不能作用于字节）；也不能与向后兼容的、极少使用的 u 混用（那样会得出对这个"面向家用的"文本不太合适的组合前缀）。

深度理解

- 核心概念：f-string 是语法糖式的格式化字面量——{} 里放的是表达式而不是参数编号，运行期求值并排版。它是三人竞赛中的"最新选手"，也是 3.6+ 的现代首选。
- 底层实现：词法分析器把 f-string 拆成常量文本片段与 {} 表达式片段，编译期就把表达式编译进字节码（每个 {} 独立求值，走 FORMATVALUE/BUILDSTRING），运行时在当前作用域取变量当前值——所以它有"代码的性质"：延迟求值、作用域敏感、不能存文件当模板。
- 设计原因：代码可读性——值的来源就地可见，不必回头数 % 元组或 .format 参数；任意表达式（方法调用、运算、切片）都可用，突破了 .format 只认字段名语法的限制。
- 实际问题：日志、报错信息、用户提示里直接插入变量；f'{x=}' (3.8+) 实现调试标签；rf 组合写路径与正则。

·初学者误区：(1) 把 f-string 当"模板"，试图运行时从变量/文件组装——它是源码，只能内联表达式；(2) 想在 3.12 之前的版本用 f'{x + '!}' 这类同引号语法——会报错，须换用外圈异号引号；(3) 把它当 docstring——运行时求值、不是纯字面量；(4) 以为它仅能像 print 那样 str 化——通过 :spec 能做所有 .format 能做的高级排版。

代码分析

```
task = f'Learning {what.upper() + '!}' in {tool + str(3.12)}'  
task = f'Learning {what:>' + what + '}'
```

解释器处理流程：读取 f 前缀 → 词法分析把字符串切为常量片段与 {} 表达式 → 语法分析生成包含表达式求值 + FORMATVALUE 的字节码 → 运行时依次求值每个 {}、排版、拼接。f-string 是编译期"计划"、运行期"执行"的代码，不是被动数据。

7.10.1 F-String 格式化基础 (F-string formatting basics)

英文原文

Within the f-string literal, curly braces are used to denote substitutions just like the formatting method. Unlike the method, though, {}s contain inline Python expressions whose formatted runtime results replace the bracketed parts:

```
python >> what = 'coding' >> tool = 'Python' >> f'Lear
```

ning {what} in {tool}'Learning coding in Python'

As usual, the f-string result is a new string that we're letting the REPL display, but it can also be assigned to a name to be used elsewhere in our code. F-strings also frequently appear in print calls to display formatted text, though sometimes more often than they should: there's no reason to use an f-string for simple space-separated prints.

In its simplest form like this, an f-string's expressions enclosed in {} are evaluated and then formatted per their print-string defaults. This isn't much different from the equivalent expression or method, but is slightly easier on your keyboard and may be slightly easier on your eyes:

```
python >> 'Learning %s in %s' % (what, tool)'Learning
coding in Python' # Expression equivalent
>> 'Learning {} in {}'.format(what, tool)'Learning coding in Python'
# Method equivalent
```

Importantly, any expression can be used in the curly braces and works as it would outside the f-string:

```
python >> task = f'Learning {what.upper() + '!}' in {tool + str(3.12)}'
>> task'Learning CODING! in Python3.12'
```

Moreover, {} expressions are evaluated both where they appear (subject to the name-scoping details you'll meet later in this book) and when the f-string is run

to make a string (not when your code is first read by Python). Like the formatting expression and method, it's a runtime operation that uses the current values of any variables it names:

```
python >> what = 'f-strings'>> task # F-strings built
when run (only)'Learning CODING! in Python3.12'>>
task = f'Learning {what.upper()} + '!}' in {tool + str(3.12
)}}>> task'Learning F-STRINGS! in Python3.12'
```

As a preview, this runtime nature also means that f-strings, unlike all other text-string literals, don't work as Chapter 15's docstrings. This makes sense if you keep in mind that f-strings are runtime code, more like the % expression and format method calls. When coded in a function, for example, an f-string won't be run until that function is called.

One syntax quirk here: as this example demos, you can embed quotes within an f-string's {} even if they are the same as the quotes used for the f-string at large—but this is new as of Python 3.12. In earlier Python s, backslash escapes didn't help for nested quotes, though other enclosing-quote tricks did, and none of this applies outside a {}:

```
python f'Learning {what + '!}' # OK as of Python 3.12
f'Learning {what + '!}' # An error before Python 3.12
f'Learning {what + '\!\'}' # And this doesn't make it work
f"Learning {what + '!}" # OK before (and after) Python 3.12
f'"Learning {what + '!}'"' # Ditto f'Learning'{w
```

```
hat +'!')' # An error in 3.12+ f'Learning \'{what +'!')\'
# OK if escape quotes outside {}
```

Also new as of Python 3.12, a backslash can be used in an f-string's {} part and works just like it does outside the {}, as do both comments and newlines (the following may stretch the limits of f-strings, but prove these points):

```
python >> f'{\n'.join([what] 3) + '\x21')'f-strings\nf-strings\nf-strings!
```

```
>> f'Learning { # Your comment here
```

```
... what.upper() +'!... } in {tool + str(3.12)}'Learning F-STRINGS! in Python3.12'
```

In other words, if you like f-strings, you'll like them best in Python 3.12+. As this book is based on 3.12 (and 3.13 is right around the corner), it will generally use 3.12's f-string rules; mod examples' quotes for older Pythons if needed.

中文翻译

在 f-string 字面量内部，花括号同样用来表示替换，如同格式化方法那样。但与方法不同，{} 里装的是内联的 Python 表达式，它们的格式化运行结果会替换掉括号所在的片段：
python >>> what = 'coding'>>> tool = 'Python'>>> f'Learning {what} in {tool}'Learning coding in Python'照例，f-string 的结果是一个我们让 REPL 显示的新字符串，不过它也可以赋给一个名字，以便在代码其它地方使用。f-string 也经常出现在 print 调用中显示格式化文本，

虽然有时出现得比应有的还多：对简单的空格分隔打印，根本没理由用 f-string。在像这样最简单的形式里，f-string 的 {} 中的表达式被求值，然后按其打印字符串的默认格式排版。这与等价的表达式或方法差别不大，但对你敲键盘略省力，对你的眼睛也略友好：python >>> 'Learning %s in %s' % (what, tool) 'Learning coding in Python' # 表达式等价写法 >>> 'Learning {} in {}'.format(what, tool) 'Learning coding in Python' # 方法等价写法 重要的是，花括号里可以放任意表达式，它的行为和 f-string 外完全一样：python >>> task = f'Learning {what.upper()} + '!}' in {tool + str(3.12)}' >>> task 'Learning CODING! in Python3.12' 再者，{} 表达式既在它们出现的位置被求值（受你稍后会学到的名字作用域细约束），也在 f-string 被运行生成为字符串时被求值（而不是在你代码首次被 Python 读取时）。和格式化表达式与方法一样，这是一次使用它所引用的任何变量当前值的运行期操作：python >>> what = 'f-strings' >>> task # f-string 只在运行时构建 'Learning CODING! in Python3.12' >>> task = f'Learning {what.upper()} + '!}' in {tool + str(3.12)}' >>> task 'Learning F-STRINGS! in Python3.12' 顺带预告：这种运行期的本质也意味着，与其他所有文本字符串字面量不同，f-string 不能当作第 15 章的 docstring。只要记住 f-string 是运行期代码、更像 % 表达式和 format 方法调用，这就说得通了。例如，写在函数里时，f-string 要等那个函数被调用才会运行。这里有个语法怪癖：正如这个示例演示的，你可以在 f-string 的 {} 内嵌入引号，哪怕它们与 f-string 整体所用的引号相同——但这是 Python 3.12 的新特性。在更早的 Python 里，反斜杠转义对嵌套引号不管用，不过其它

"换外侧引号"的技巧仍然有效，而且这些都不适用于 {} 之外：`python f'Learning {what + '!}'` # 从 Python 3.12 起可行
`f'Learning {what + '!}'` # 在 Python 3.12 之前是错误
`f'Learning {what + '\!}'` # 而这并不能让它可行
`f"Learning {what + '!}"` # 在 Python 3.12 之前（及之后）都可行
`""Learning {what + '!}""` # 同理
`f'Learning'{what + '!}'` # 在 3.12+ 中会报错
`f'Learning \'{what + '!}\'` # 在 {} 之外转义引号即可
Python 3.12 还有另一项新特性：反斜杠可以出现在 f-string 的 {} 部分，行为与它在 {} 之外时完全一样，注释和换行同样如此（下面的代码可能把 f-string 的极限撑得有点开，但恰好证明这些要点）：
`python >>> f'\n'.join([what] 3) + '\x21' f-strings\nf-strings\nf-strings!>>> f'Learning { # 这里放你的注释... what.upper() + '!...' } in {tool + str(3.12)} Learning F-STRINGS! in Python3.12'`
换句话说，如果你喜欢 f-string，在 Python 3.12+ 下你会最喜欢它们。由于本书基于 3.12（而 3.13 也近在眼前），它将普遍采用 3.12 的 f-string 规则；在更老的 Python 下运行，需要时请改写示例的引号。

深度理解

·核心概念：f-string 的三个关键词——表达式（{} 内是任意 Python 表达式，不是字段名）、运行期（求值只在字符串被构建时发生）、当前值（用变量此刻的值，而非第一次读取时的值）。

·底层实现：f-string 经词法分析被拆成"常量文本片段 + 表达式片段"，并在编译期展开成字节码序列；每次运行都重新求值表达式、调用其 format 排版再拼接。因为它"是

代码”，所以才能在函数体内延迟求值，也因此不能被用作 docstring（docstring 是模块/类/函数体首条真正的字面量语句）。

·设计原因：把“要显示什么”和“值在哪”放同一处，消除 % 元组的顺序错位和 .format 的字段编号负担；同时保留方法的自定义格式语法，学习成本低。

·实际问题：3.12 放宽了 {} 内引号、反斜杠、注释与换行的限制（PEP 701），写正则、路径、多行表达式更方便；旧版本只能用“外侧引号与内层不同”规避。

·初学者误区：(1) 误以为 f-string 在函数定义时就求值——不，它在函数被调用、语句执行到那一刻才求值；(2) 想用 \n、\x21 等在 3.12 之前的 {} 内转义——只有 3.12 + 才行；(3) 把 f-string 当 docstring 用会报语法/悬挂问题；(4) 对“运行时才构建”没有概念，导致 debug 打印看不到最新值而困惑。

代码分析

```
>>> what = 'coding'; tool = 'Python'
>>> task = f'Learning {what.upper()} + '!' in {tool + str(3.12)}'
>>> task
'Learning CODING! in Python3.12'
```

处理流程分解：f 前缀触发 f-string 语法 → 词法器分别识别常量'Learning'、表达式 what.upper() + '!'、常量'in'、表达式 tool + str(3.12) → 字节码依次：取全局名 what → 调用 upper() → 与'!' 相加；取 tool → 调用 str(3.12) → 相加成'Python3.12' → 最后用 BUILDSTRING 拼接四段为新字符串。注意第二行 >> 回显之前先改了 what，但

旧 task 值不变——因为旧值早已在上一次运行该语句时固化。

7.10.2 F-String 自定义格式 (F-string custom formats)

英文原文

As noted, f-strings use the same custom-format syntax as string methods, so there's not much new to learn here. In the abstract, f-strings are coded with a format like this (as usual, [...] means an optional part here and its square brackets are not part of the f-string's code, but spaces are generally allowed between the parts here): `f'...literaltext... {expression [=] [!s, !r, or !a] [:formatspec]} ...literaltext...'` As in the method, text outside a `{}` is taken literally and uses `{{` and `}}` to escape braces, and the `{}` part can be repeated to embed multiple values. In each, the expression part is any Python expression code (including nested f-strings) and is run to produce the substitution value before formatting it. As in the formatting method, the optional `!s`, `!r`, or `!a` render the expression in user-friendly, as-code, or ASCII-with-escapes form—which is the same as calling `str`, `repr`, or `ascii` for the entire expression enclosed by `{}`. Within a `{}`, the `formatspec` that is coded after a `:` is (nearly) identical to that used in the string method and presented earlier, so we won't repeat its

syntax here; see "Formatting method custom formats" for the options that f-strings share with the method . As a minor convenience for developers, f-strings also allow an = character after expression to add the expression's text and an "=" as a label before its value formatted with a repr default.

中文翻译

正如前面所说，f-string 使用与字符串方法相同的自定义格式语法，所以这里没什么新东西可学。抽象地说，f-string 的编码格式如下（照例，[...] 表示可选项，这里的方括号不属于 f-string 的代码，但各部分之间通常允许留空格）：
f'...**字面文本**... {表达式 [=] [**!s**、**!r** 或 **!a**] [**:格式说明**]} ...**字面文本**...'与方法一样，{} 之外的文本按字面处理，用 {{ 和 }} 转义花括号，{} 部分可以重复以嵌入多个值。在每一处中，表达式部分可以是任意 Python 表达式代码（包括嵌套的 f-string），并在格式化之前先运行以产生替换值。与格式化方法一样，可选的 **!s**、**!r** 或 **!a** 把表达式渲染成对用户友好的、代码形式的、或带转义的 ASCII 形式——这等同于对整个 {} 所括的表达式调用 str、repr 或 ascii。在 {} 之内，跟在 : 后的格式说明与字符串方法中用的（几乎）完全相同，前面已经讲过了，所以这里不再重复其语法；f-string 与方法共享的选项请参见"格式化方法自定义格式"。对开发者有个小便利：f-string 还允许在表达式后加一个 = 字符，把表达式的文本和"="作为标签加在其值之前，值用 repr 的默认格式排版。

深度理解

- 核心概念：f-string 的 {} 结构 = expression + (可选) = + (可选) !s!/r!/a + (可选) :formatspec——四段依次排开，前两段是 f-string 特有的，后两段与方法共用。
- 底层实现：= 调试标签是词法/语法层面的"显示增强"：把表达式的源代码文本原样抓出，加上 =，再接对表达式值的 repr（默认）结果——3.8 引入（PEP 498 的延后补充）。!s!/r!/a 则在运行期对已求值对象调用 str()/repr()/ascii()。
- 设计原因：= 解决"打印变量是为了看它叫什么"的高频调试需求，省去手写 f'x={x!r}'；格式说明复用方法，让两种工具思维统一。
- 实际问题：f'{intval = }' 会输出 intval = 42 这样的带空格形式；= 后面仍可跟 : 和转换标志，顺序固定为 expression = !conv :spec。
- 初学者误区：(1) 把 = 误解为"等于"或赋值——它只是标签语法；(2) 在 3.8 之前用 = 会报错；(3) 忘记 !s!/r!/a 与 : 的先后顺序（必须先 ! 后 :）；(4) 以为 {{ }} 可以替代 {} 转义后的插值——它们只是输出一个字面大括号。

代码分析

```
f'{c = :>20}'      # + 3.8+
f'{c=!r:10}'       # + repr +
```

内部处理：先求值 c 并得到其文本标签 'c ='（或 c=）；若带 !r 则先调用 repr(c) 得到候选字符串；再套用 :20 或

:10 宽度与对齐规则；最后把"标签 + 格式化结果"一次性拼入外层字符串。

7.10.3 F-String 高级示例 (Advanced f-string examples)

英文原文

All of which is easier to explain by example than narrative, so let's get back to running code. Numbers can be formatted in a variety of ways spelled out for the formatting method earlier—including defaults, fixed decimal digits, comma and underscore separators, exponents, signs, and leading zeroes:

```
python >> a = 3.14156 >> b = 1234567 >> f'{a} and {b}' # Defaults '3.14156 and 1234567' >> f'{a:.2f} and {b:09}' # Decimals, padding '3.14 and 001234567' >> f'{a:1000:,.2f} and {b:,} and {b:}' # Comma and underscore separators '3,141.56 and 1,234,567 and 1234567' >> f'{a:1000:e} and {b:+012,}' # Exponents, signs, and padding '3.141560e+03 and +001,234,567' >> f'{b:X} and {b:o} and {b // 64:b}' # Hex, octal, binary, underscores '12D687 and 4553207 and 100101101011010'
```

Adding an `=` after the expression may be useful when you're debugging code, as it labels data automatically (though this may be more readable for simple variable names than larger expressions):

```
python >> f'{a=:e} and {b=:+012,}' # Labeled'a=3.141
560e+00 and b=+001,234,567'>> f'{a + 1=:e} and {b
2=:+012,}'a + 1=4.141560e+00 and b 2=+002,469,1
34'
```

String formats can vary according to the s/r/a flag all so coded before the format specifier (but after an =). To demo, the following uses a non-ASCII character (a n "A" with either an umlaut mark or diaeresis):

```
python >> c = 'h\xc4ck' # \xc4 (a.k.a \u00c4) is non-A
SCII Ä >> f'{c} and {c} and {c}' # Defaults'hÄck and hÄ
ck and hÄck'>> f'{c!s} and {c!r} and {c!a}' # Display m
ode, two ways'hÄck and'hÄck' and'h\\xc4ck'>> f'{str
(c)} and {repr(c)} and {ascii(c)}'hÄck and'hÄck' and'h\
\xc4ck'>> f'{c=!s} and {c=!r} and {c=!a}' # Labeled"c
=hÄck and c='hÄck' and c='h\\xc4ck'>> f'{c=!s:8} an
d {repr(c)} and {c:0>8}' # Width, fill, alignment"c=hÄc
k and'hÄck' and 0000hÄck"
```

Advanced tip: as in the formatting method, parts of the format specifier can be fetched dynamically at runtime instead of being hardcoded, by using nested {} expressions. The nested expression's formatted result is used where it appears:

```
python >> width = 8 >> f'{a:.8f} and {c:0>8}' # Hardc
oded parameters'3.14156000 and 0000hÄck'>> f'{a:.{
width}f} and {c:0>{width}}' # Dynamic parameters'3.1
4156000 and 0000hÄck'>> f'{a=:.{width}f} and {c 2:0
>{width 3}}'a=3.14156000 and 0000000000000000hÄ
```


ckhÄck'

Like the format method, any part of a formatspec in a {} can be a nested {}—though they contain expressions in the f-string, not argument identifiers. This is why f-string formats are just "(nearly)" identical. Other parts of f-strings don't allow {}s (forward reference: for economy, the following assigns many names positionally with sequence-assignment syntax covered formally in Chapter 11—it's like `a=0`, `b='<'`, and so on):

```
python >> what = 1234.564 >> a, b, c, d, e, f, g, h =  
0, '<', '+', 12, ',', 2, 'f' >> f'{what:0<+12,.2f}!' + 1,234.56  
000!'
```

```
>> f'{what:{a}{b}{c}{d}{e}{f}{g}{h}}!' + 1,234.56000!' # B  
ut don't try this at home?
```

Usage tip: because f-strings run expressions that reference variables, they may not be easy to use as templates when substitution values are collected in a container at runtime—as they often would be. In the following, `a` converts dictionary keys to keyword arguments, as you'll learn later in this book:

```
python >> values = dict(tool='Python', role='scriptin  
g') # Collected values >> 'Use %(tool)s for %(role)s.'  
% values # Expression: keys'Use Python for scripting.'  
>> 'Use {tool} for {role}.'.format(values) # Method: ke  
ywords'Use Python for scripting.'
```

```
>>'Use {0[tool]} for {0[role]}'.format(values) # Method: reused-arg keys'Use Python for scripting.'
```

```
>> f'Use {values['tool']} for {values['role']}.' # F-string: expressions'Use Python for scripting.'
```

For more impressive f-string results, assign substitution values to same-scope variables when possible:

```
python >> tool = 'Python'>> role = 'scripting'>> f'Use {tool} for {role}.' 'Use Python for scripting.'
```

But also bear in mind that f-strings may not be easy to use when templates are loaded from external files at runtime—as they often would be. While the format expression and method can treat such templates as simple text data, f-strings are Python program code, and hence may have to be run post load with the eval function of Chapter 5—and trusted to not contain code that will do damage (e.g., erasing files is fairly easy in a Python expression):

```
python >> fs = """f'Use {tool} for {role}.'""" # As if loaded from a file >> eval(fs) # Run as code – and trust! 'Use Python for scripting.'
```

F-strings are a powerful tool, but their nested expressions make them geared more toward in-program formatting than data-based roles. For more details on the f-string, see its full disclosure in Python's language reference manual.

中文翻译

这一切靠例子比靠叙述更好懂，所以我们回到运行的代码上来。回调到前面为格式化方法铺陈过的方式，数字可以用多种形式排版——包括默认、固定小数位数、逗号与下划线分隔符、指数、符号、前导零：

```
python >>> a = 3.14156
>>> b = 1234567 >>> f'{a} and {b}' # 默认'3.14156 a
nd 1234567'>>> f'{a:.2f} and {b:09}' # 小数位数、补零'
3.14 and 001234567'>>> f'{a 1000:,.2f} and {b:,} and {
b:}' # 逗号与下划线分隔符'3,141.56 and 1,234,567 and
1234567'>>> f'{a 1000:e} and {b:+012,}' # 指数、符号
与补零'3.141560e+03 and +001,234,567'>>> f'{b:X} a
nd {b:o} and {b // 64:b}' # 十六进制、八进制、二进制、
下划线'12D687 and 4553207 and 100101101011010'
在表达式后加 = 在调试代码时可能很有用，它会自动给数
据加标签（不过对简单的变量名比对大的表达式更好读些）
：python >>> f'{a=:e} and {b=:+012,}' # 带标签'a=3.1
41560e+00 and b=+001,234,567'>>> f'{a + 1=:e} an
d {b 2=:+012,}' 'a + 1=4.141560e+00 and b 2=+002,4
69,134'字符串格式还可以随 s/r/a 标志（也写在格式说明
之前、但在 = 之后）而变化。为了演示，下面用一个非 AS
CII 字符（带元音变音号或分音符的"A"）：python >>> c
='h\xc4ck' # \xc4（也叫 \u00c4）是非 ASCII 的 Ä >>>
f'{c} and {c} and {c}' # 默认'hÄck and hÄck and hÄck'>
>> f'{c!s} and {c!r} and {c!a}' # 显示模式，两种写法'hÄ
ck and'hÄck' and'h\\xc4ck'" >>> f'{str(c)} and {repr(c)
} and {ascii(c)}' 'hÄck and'hÄck' and'h\\xc4ck'" >>> f'{
c=!s} and {c=!r} and {c=!a}' # 带标签'c=hÄck and c='h
```

Äck' and c='h\\xc4ck'" >>> f'{c!=s:8} and {repr(c)} and {c:0>8}' # 宽度、填充、对齐"c=hÄck and'hÄck' and 000hÄck" 进阶提示：和格式化方法一样，格式说明的各部分可以从运行期动态取得，而不是写死，办法是使用嵌套的 {} 表达式。嵌套表达式的格式化结果就用在它出现的位置：

```
python >>> width = 8 >>> f'{a:.8f} and {c:0>8}' # 写死的参数'3.14156000 and 0000hÄck'>>> f'{a:.{width}f} and {c:0>{width}}' # 动态参数'3.14156000 and 0000hÄck'>>> f'{a:.{width}f} and {c 2:0>{width 3}}' 'a=3.14156000 and 0000000000000000hÄckhÄck'
```

和 format 方法一样，{} 中格式说明的任何部分都可以是一个嵌套的 {}——不过在 f-string 里它们装的是表达式，而不是参数标识符。这就是为什么 f-string 的格式"（几乎）完全相同"。f-string 的其它部分不允许 {}（前瞻说明：为省篇幅，下面用第 11 章才正式讲的序列赋值语法一次性按位赋值多个名字——相当于 a=0、b='<', 等等）：

```
python >>> what = 1234.564 >>> a, b, c, d, e, f, g, h = 0, '<', '+', 12, ',', 2, 'f'>>> f'{what:0<+12,.2f}!'+'1,234.56000!'>>> f'{what:{a}{b}{c}{d}{e}{f}{g}{h}}!'+'1,234.56000!' # 但请勿在家中模仿？用法提示：因为 f-string 运行的是引用变量的表达式，当替换值是在运行期被收集进一个容器里——正如它们经常如此时，f-string 会不好当作模板用。下面是"用把字典键转换成关键字实参"的示范，你会在本书后面学到它：
```

```
python >>> values = dict(tool='Python', role='scripting') # 收集好的值>>>'Use %(tool)s for %(role)s.' % values # 表达式：键'Use Python for scripting.'>>>'Use {tool} for {role}.'.format(values) # 方法：关键字'Use Python for scripting.'>>>'Use {0[tool]} for {0[role]}.'.f
```

format(values) # 方法：复用实参的键'Use Python for scripting.'>>> f'Use {values['tool']} for {values['role']}.' # f-string: 表达式'Use Python for scripting.'想得到更漂亮的 f-string 效果，尽量把替换值赋给同作用域的变量：python >>> tool = 'Python'>>> role = 'scripting'>>> f'Use {tool} for {role}.' 'Use Python for scripting.'但也要记住：当模板在运行期从外部文件载入——正如它们经常如此时，f-string 可能不好用。格式表达式与方法可以把这类模板当作纯文本数据，而 f-string 是 Python 程序代码，因此在载入后可能不得不用第 5 章的 eval 函数来运行它——并且要相信它不会包含能造成损害的代码（例如，在一个 Python 表达式里删文件相当容易）：python >>> fs = """f'Use {tool} for {role}.'""" # 仿佛从文件载入>>> eval(fs) # 以代码运行——并信任它！'Use Python for scripting.' f-string 是个强大的工具，但它的嵌套表达式使它更偏向程序内格式化，而不是数据驱动的角色。关于 f-string 的更多细节，请参见 Python 语言参考手册里对它全面披露的内容。

深度理解

- 核心概念：f-string 覆盖了方法的所有 format-spec 能力（对齐、填充、精度、类型码、分隔符），并叠加两个独有能力——= 调试标签和嵌套 {} 动态格式。表达式内联让它能做 values['tool']、a 1000 这类方法做不到的活。
- 底层实现：嵌套 {} 在解析期进行递归分析：内层 {} 先被单独编译求值，其结果作为外层 format-spec 的文本再被解析。!r!a 是对对象调用 repr()/ascii()（后者把非 ASCII

编码成 `\xNN/\uNNNN` 转义)，所以 `c!a` 变成 `'h\\xc4ck'`。

·设计原因：让“一眼看懂输出”成为默认——`{a=}` 省的不仅是打字，还省去上下文猜测；动态宽度来自变量，适合表格化、对齐列宽等真实需求。

·实际问题：日志里用 `f'{exc=}'` 快速查看异常对象；生成表格时列宽按数据动态计算；`b // 64:b` 证明 `formatspec` 前那段也可以是任意表达式（这里是整除）。

·初学者误区：(1) 试图在 f-string 里动态“拼接一个格式说明变量”来模拟模板——那就回到 `.format/%` 的作用域了；(2) 对 `eval` 文件模板的安全隐患掉以轻心；(3) 以为嵌套 `{}` 可以出现在 `expression` 部分之外——不行，只有 `:formatspec` 内允许；(4) 写 `f'{values['tool']}'` 在 3.12 前的 Python 会因引号问题报错，须用字典变量或外层异号引号。

代码分析

```
>>> a, b, c, d, e, f, g, h = 0, '<', '+', 12, ',', '.', 2, 'f'
>>> f'{what:{a}{b}{c}{d}{e}{f}{g}{h}}!'
'+1,234.56000!'
```

执行剖析：内层 `{}` 逐个求值得到 `('0','<','+', 12,',', '.', 2,'f')`，其 str 拼接成 `'0<+12,.2f'`，该字符串成为外层 `{}` 的完整 `formatspec`，等价于写死 `f'{what:0<+12,.2f}!'`。三个要点：内层求值先于外层排版；`what` 的值 1234.564 按，千分位、`.2f` 两位小数、`+` 强制符号、`<` 左对齐、`0` 填充到宽 12；外层除了 `:formatspec` 之外不允许再嵌套 `{}`。

7.10.4 谁是赢家? (And the Winner Is...)

英文原文

The previous edition of this book went to considerable lengths (about seven pages) to show how the formatting method, newest at the time, was functionally redundant with the expression, in order to underscore the downsides of feature bloat in programming languages. Given that the number of primary formatting tools in Python has climbed from two to three since then, that message may not have entirely hit its mark. Consequently, this edition has dropped most of the rhetoric, and opted to leave this race's call up to you. But if you're looking for a guideline, there is no killer argument for dismissing any formatting option out of hand for all use cases. As stated at the start of this section, f-strings may be best in most new code, on logistical grounds alone: newer is less likely to be culled by Python sooner. In fact, we'll be using them regularly in the rest of this book where warranted, so expect more examples ahead. Even so, you will also see the expression and method often in existing code and may have to use them in roles that f-strings don't address (e.g., for substitutions in text loaded from files). More fundamentally, this book is not in the business of telling you what to do, and you are welcome to use any formatting option you prefer. Imposing new tools on programmers is exclusive, divisive, and probab

ly rude. Despite norms in the software field today, the choice of development options should be yours—and yours alone—to make. As for the bloat: change is not always bad, but it can be when it creates redundancy or incompatibility. In fact, you've just had a front-row seat to one of its worst consequences: N functionally equivalent options can multiply newcomers' learning requirements by N . We'll return to this and other perils of ego-fueled churn and convolution in software development at the end of this book. For now, we'll close by simply noting that this stuff still matters. With any luck, a future edition won't have yet another formatting tool to doc, and future learners won't have yet another one to grok.

中文翻译

本书上一版本花了很大篇幅（约七页）去说明格式化方法（当时最新的一种）与表达式在功能上如何冗余，目的是强调编程语言中特性膨胀（feature bloat）的弊端。鉴于自那以来 Python 的主要格式化工具数量已从两个升到三个，那条信息也许没完全达到它的目标。因此，这一版删掉了大部分说教，把这场比赛的裁决权留给了你。但如果你想要一条准则：没有任何一个“通杀”的理由能让你对所有使用场景一概抛弃某一格式化选项。就像本节开头所说，单就“后勤”理由，f-string 在大多数新代码中可能是最好的：更新的东西更不可能被 Python 更早淘汰。事实上，本书余下部分在合适之处会经常用它，所以后面还会有更多示例。即便如此，你也会在现有代码里常看到表达式和方法，并可能不得不在

f-string 覆盖不到的角色里用它们（例如，对从文件载入的文本做替换）。更根本的是，本书并不负责告诉你该怎么做事，你完全可以选用任何你偏爱的格式化选项。把新工具强加给程序员是排他的、分裂的，而且大概也不够礼貌。尽管如今软件领域有此风气，开发选项的选择权应该在你——也仅仅在你手上。至于膨胀（bloat）：改变不总是坏事，但当它制造冗余或不兼容时就成了坏事。事实上，你刚刚就坐在它最恶劣后果的最前排：N 个功能等价的选项，会把新手的必需学习量放大 N 倍。书末我们还会回头谈这一点以及软件开发中“自我驱动”的折腾与过度复杂的其他危害。现在，我们用一句话收尾：这些东西仍然重要。运气好的话，未来再版不会再多一个需要文档化的格式化工具，未来的学习者也不会再多一个需要吃透的工具。

深度理解

- 核心概念：三种工具是共存生态而非替代关系：f-string（新代码主流，程序内插值）、% 表达式（老代码固有，文本模板）、.format（方法与 f-string 的桥梁，动态字段名）。
- 底层实现：作者借此抛出“功能冗余 = 膨胀”的软件设计议题：三种工具底层都要走“求值 → format → 拼接”，但多出的语法面增加了初学者 3 倍的学习量。
- 设计原因：Python 演进的历史遗留——% 从 C 语言借来，.format 为修 % 的缺陷而生，f-string 为省代码而生；每个都不完美，但每个都仍在服务大量存量代码。
- 实际问题：读老代码能认出三种写法各自来历；选型时优先 f-string，但涉及“模板来自文件/数据”的场景回退 .for

mat 或 %。

·初学者误区：(1) 把"推荐 f-string"理解成"其他都该改"——存量代码不必要重写；(2) 认为 .format 已死——它至今是唯一能"动态装配格式字符串"的内置方案之一；(3) 忽略作者的核心方法论：工具选择权在自己，理解冗余背后的成本同样重要。

代码分析

```
# name/version
>>> f'{name} is {version}'           # f-string
>>> '{} is {}'.format(name, version) #
>>> '%s is %s' % (name, version)    #
```

三者字节码层面的差异：f-string 由编译器直接展开、无需运行时解析模板；.format 在运行时解析模板字符串再逐个 format；% 则走 C 的格式化逻辑。性能与可读性各有取舍，但对日常输出，三者结果一致。

7.11 还有一位：string.Template (PLUS ONE MORE: STRING.TEMPLATE)

英文原文

Technically speaking, there are four (not three) formatting tools built into Python today, if we include the obscure string module's Template tool mentioned earlier. Now that you've seen the other three, you can tell how it compares.

The expression, method, and f-string can all be used as templating tools, referring to substitution values by name using dictionary keys, keyword arguments, or variables (the ";" in the following separates multiple statements needed to give the f-string variables to reference):

```
python >> 'The %(num)s %(tool)s' % dict(num=4, tool='formatter')
The 4 formatters' >> 'The {num} {tool}s'.format(num=4, tool='formatter')
The 4 formatters' >> num=4; tool='formatter'; f'The {num} {tool}s'
The 4 formatters'
```

The module's templating system allows values to be referenced by name too, prefixed by a \$, as either dictionary keys or keywords, but does not support all the utilities of the other two methods—a limitation that yields simplicity, the prime motivation for this tool:

```
python >> import string >> t = string.Template('The $num ${tool}s')
>> t.substitute(num=4, tool='formatter')
The 4 formatters' >> t.substitute(dict(num=4, tool='formatter'))
The 4 formatters'
```

See Python's manuals for more details. It's possible that you may see this alternative (as well as additional tools in the third-party domain) in Python code too; thankfully this technique is simple and is used rarely enough to warrant its limited coverage here.

中文翻译

严格来说，Python 今天内置了四个（不是三个）格式化工具——如果我们把前面提过的那个鲜为人知的 `string` 模块里的 `Template` 工具也算进去的话。现在你已经见过另外三个，能分辨出它的定位了。表达式、方法和 `f-string` 都可以当模板工具用，用字典键、关键字实参或变量按名字引用替换值（下面的分号把给 `f-string` 提供可引用变量的多条语句分隔开）：

```
python >>> 'The %(num)s %(tool)s' % dict(num=4, tool='formatter')
'The 4 formatters'
>>> 'The {num} {tool}s'.format(num=4, tool='formatter')
'The 4 formatters'
>>> num=4; tool='formatter'; f'The {num} {tool}s'
'The 4 formatters'
该模块的模板系统同样允许按名字引用值，用 $ 做前缀，既可用字典键也可用关键字，但不支持另外两者的全部功能——这种局限反而带来简单性，这正是这个工具的首要动机：
```

```
python >>> import string
>>> t = string.Template('The $num ${tool}s')
>>> t.substitute(num=4, tool='formatter')
'The 4 formatters'
>>> t.substitute(dict(num=4, tool='formatter'))
'The 4 formatters'
更多细节见 Python 的手册。你也可能在 Python 代码中看到这个替代品（以及第三方领域的其它工具）；好在它足够简单、又用得够少，在此只作有限篇幅的介绍。
```

深度理解

·核心概念：`string.Template` 用 `$name` 或 `${name}` 占位，专为“把模板当数据处理”而生——它不解析 Python 语法，只做纯文本替换。

·底层实现：Template 用正则把 \$ 标识符替换成 %(name)s 后走 % 格式化；substitute 对缺失键直接报 KeyError，而另有 safesubstitute 版本不会抛错（适合部分填充）。

·设计原因：当模板来自用户输入或不完全可信的文件时，%/.format 的格式说明其实也是一种“语言”；Template 把语法面压缩到最小，天然防注入、易做 i18n（国际化）。

·实际问题：邮件正文、配置文件、i18n 模板等“模板与代码分离”的场景；\$ 冲突时用 \$\$ 转义字面 \$。

·初学者误区：(1) 以为它速度更快——其实只是更简单更安全；(2) 想要格式化（如宽度、小数位）却来用 Template——它不做这些；(3) 忘记 \$ 后花括号的作用——\${tool}s 是为了让 s 紧跟其后不被吞进名字。

代码分析

```
>>> t = string.Template('The $num ${tool}s')
>>> t.substitute(num=4, tool='formatter')
'The 4 formatters'
```

执行流程：Template 构造时把模板转换为内部 % 风格（\$num → %(num)s, \${tool} → %(tool)s, \$num 后面紧跟的 s 属于字面文本，因为 \${tool} 用花括号界定了标识符边界）→ substitute 按映射填充缺失检查严格性 → 返回新字符串。整个过程不执行任何 Python 表达式，所以天然没有代码注入风险。

7.12 通用类型范畴 (General Type Categories)

英文原文

Now that we've explored the first of Python's collection objects, the string, let's close this chapter by defining a few general type concepts that will apply to most of the types we'll look at from here on. With regard to built-in types, it turns out that operations work the same for all the types in the same category, so we'll only need to define most of these ideas once. We've examined only numbers and strings so far, but because they are representative of two of the three major type categories in Python, you already know more about several other types than you might think.

中文翻译

既然我们已经探索了 Python 的第一个集合对象——字符串，就让我们通过定义几个通用类型概念来结束本章；这些概念将适用于我们从此往后的几乎所有类型。就内置类型而言，结果表明：同一范畴中的所有类型，其操作以相同方式工作，所以我们只需要把这些思想中的大部分定义一次。到目前为止我们只研究了数字和字符串，但由于它们代表了 Python 三大类型范畴中的两个，你其实已经比你以为的了解更多几个其它类型了。

7.12.1 类型按范畴共享操作集合 (Types Share Operation Sets by Categories)

英文原文

As you've learned, strings are immutable sequences: they cannot be changed in place (the immutable part), and they are positionally ordered collections that are accessed by offset (the sequence part). It so happens that all the sequences we'll study in this part of the book respond to the same sequence operations shown in this chapter at work on strings—concatenation, indexing, iteration, and so on. More formally, there are three major type (and hence operation) categories in Python that have this generic nature:— Numbers (integer, floating-point, decimal, fraction, others): Support addition, multiplication, etc.— Sequences (strings, lists, tuples): Support indexing, slicing, concatenation, etc.— Mappings (dictionaries): Support indexing by key, etc. Python's byte strings mentioned at the start of this chapter fall under the general "strings" label here; sets are something of a category unto themselves (they don't map keys to values and are not positionally ordered sequences); and we haven't yet explored mappings on our in-depth tour (we will in the next chapter). However, many of the other types we will encounter will be similar to numbers and strings. For example, for any sequence objects X and Y:— $X + Y$ make

s a new sequence object with the contents of both operands joined. - `X * N` makes a new sequence object with `N` copies of the sequence operand `X`. In other words, these operations work the same way on any kind of sequence, including strings, lists, tuples, and some user-defined object types. The only difference is that the new result object you get back is of the same type as the operands `X` and `Y`—if you concatenate lists, you get back a new list, not a string. Indexing, slicing, and other sequence operations work the same on all sequences, too; the type of the objects being processed tells Python which flavor of the task to perform (and if that sounds like polymorphism again, it should).

中文翻译

正如你已学到的，字符串是不可变序列：它们不能在原处修改（不可变这部分），并且它们是按位置有序、按偏移访问的集合（序列这部分）。恰好，本书这一部分我们将研究的所有序列，都响应本章展示在字符串上的那些序列操作——拼接、索引、迭代等等。更正式地说，Python 中有三大具有这种通用性质的类型（因而也是操作）范畴：- 数字（Numbers）（整数、浮点数、小数、分数及更多）：支持加法、乘法等。- 序列（Sequences）（字符串、列表、元组）：支持索引、切片、拼接等。- 映射（Mappings）（字典）：支持按键索引等。本章开头提到的 Python 字节串在此归入广义的“字符串”标签之下；集合（set）多少自成一体（它们不把键映射到值，也不是按位置有序的序列）；而我们的深入巡礼中尚未探索映射（下一章就会）。不过，

我们将遇到的许多其它类型都会与数字、字符串相似。比如，对任意序列对象 X 和 Y ：- $X + Y$ 生成一个新序列对象，内容是两个操作数的拼接。- $X N$ 生成一个新序列对象，内容是序列操作数 X 的 N 份副本。换句话说，这些操作在任意种类的序列上以相同方式工作，包括字符串、列表、元组以及一些用户自定义对象类型。唯一区别是：你拿回的新结果对象与操作数 X 、 Y 同型——如果你拼接列表，得到的是一份新列表，而不是字符串。索引、切片以及其它序列操作在所有序列上同样以相同方式工作；被处理对象的类型告诉 Python 该执行哪种风味的任务（如果这又让你想起多态（polymorphism），那就对了）。

7.12.2 可变类型可在原处修改 (Mutable Types Can Be Changed in Place)

英文原文

The string's immutable classification is an important constraint to be aware of, yet it tends to trip up new users. If an object type is immutable, you cannot change its value in place; Python raises an error if you try. Instead, you must run code to make a new object containing the new value. The major core types in Python break down as follows:- Immutables (numbers, strings, tuples, frozensets): None of the object types in the immutable category support in-place changes, though we can always run expressions to make new objects and assign their results to variables as needed.- Mutables (lists, dictionaries, sets, bytearray): Convers

ely, the mutable object types can always be changed in place with operations that do not create new objects. Although such objects can be copied manually, in-place changes support direct modification. Generally, immutable types give some degree of integrity by guaranteeing that an object won't be changed by another part of a program. For a refresher on why this matters, see the discussion of shared object references in Chapter 6. To see how lists, dictionaries, and tuples participate in type categories, we need to move ahead to the next chapter.

中文翻译

字符串的不可变归类是一个需要留意的、重要的约束，却容易绊倒新用户。如果一种对象类型不可变，你就不能在原处改变它的值；你若尝试，Python 会报错。相反，你必须运行代码生成一个包含新值的新对象。Python 的主要核心类型分类如下：

- 不可变 (Immutableables) (数字、字符串、元组、frozenset)：不可变范畴里的对象类型都不支持原处修改，尽管我们总可以运行表达式生成新对象，并按需把结果赋给变量。
- 可变 (Mutables) (列表、字典、集合、bytearray)：反过来说，可变的对象类型总是可以用不创建新对象的操作在原处修改。虽然这类对象也可以手动复制，但原处修改支持直接改动。通常，不可变类型通过保证“对象不会被程序的另一部分改变”来提供某种程度的完整性。至于为什么这很重要，请回顾第 6 章对共享对象引用的讨论。要看列表、字典和元组如何参与类型范畴，我们需要进入下一章。

深度理解

·核心概念：本节把本章的观察上升为对象模型总纲——三大类型范畴（numbers/sequences/mappings）+ 可变与不可变两分类。字符串是"不可变序列"，是这两条线在同一个类型上的叠合。

·底层实现：不可变对象的内部缓冲只读，任何"修改"式操作要么返回新对象、要么抛错误；可变对象（如列表、by bytearray）暴露可写的内部存储，setitem 等协议就地生效、id 不变。

·设计原因：不可变保证共享安全（第 6 章共享引用的隐患因不可变而消解）、可用作字典键、可被缓存；可变让"就地增删改"无需整份拷贝，是性能与安全的权衡。

·实际问题：判断 API 行为的关键——`S += 'x'` 对 str 会生成新对象，对 list 则原地扩展；bytes/bytearray 是可不可变对照的活教材。

·初学者误区：(1) 混淆"对象不可变"与"变量不可重绑"；(2) 以为所有内置集合都可变——str/tuple 不可变；(3) 在不可变对象上指望"改一部分"，却忘记必须整体生成新值。

代码分析

```
>>> S = 'spam'
>>> S[0] = 'z'                                     #
TypeError: 'str' object does not support item assignment
```

对照可变类型（第 8 章全面展开）：

```
>>> L = ['s', 'p', 'a', 'm']
>>> L[0]          #
's'
>>> L[0] = 'z'    #
>>> L
['z', 'p', 'a', 'm'] # id
```

解释器视角：setitem 是通用序列协议，str 的实现直接抛 TypeError（它不可写），list 的实现则改写指针数组。这也点出下章列表“可变序列”的身份——同样支持 +//索引/切片，但多了原处修改能力。

本章小结 (Chapter Summary)

英文原文

In this chapter, we took an in-depth, second-pass tour of the string object type. We learned about coding string literals, and we explored string operations, including sequence expressions, string method calls, and string formatting in its expression, method, and literal flavors. Along the way, we studied a variety of concepts in depth, such as slicing, method call syntax, and triple-quoted block strings. We also defined some core ideas common to a variety of types: sequences, for example, share an entire set of operations demoed here for strings. In the next chapter, we'll continue our types tour with a look at the most general object collections in Python—lists and dictionaries. As you'll find, much of what you've learned here will apply to tho

se types as well. And as mentioned earlier, in the final part of this book we'll return to Python's string model to flesh out the details of Unicode text and binary data, which are of interest to some, but not all, Python programmers, and depend on tools we haven't yet studied in full. Before moving on, though, here's another chapter quiz to review the material covered here.

中文翻译

本章我们对字符串对象类型做了一次深入的、第二遍的巡礼。我们学习了字符串字面量的编写，并探索了字符串操作——包括序列表达式、字符串方法调用，以及表达式、方法、字面量三种风味的字符串格式化。一路上我们还深入研究了一系列概念，比如切片、方法调用语法、三引号块字符串。我们也定义了一些各类类型通用的核心思想：例如，序列共享一整组操作，本章就用字符串做了演示。下一章我们将继续类型之旅，考察 Python 中最通用的对象集合——列表和字典。你会发现，这里所学的许多东西对那两类同样适用。而且如前所述，本书最后一部分会回到 Python 的字符串模型，补全 Unicode 文本和二进制数据的细节——这些对一部分（而非全部）Python 程序员有意义，并且依赖我们尚未完整研究过的工具。不过在继续之前，这里又是一章测验，来复习本章覆盖的内容。

测试你的知识：测验 (Test Your Knowledge: Quiz)

1. Can the string find method be used to search a list?

字符串的 find 方法能被用来搜索列表吗?

2. Can a string slice expression be used on a list?

字符串切片表达式能用在列表上吗?

3. How would you convert a character to its ASCII integer code? How would you convert the other way, from an integer code to a character?

如何把一个字符转换成它的 ASCII 整数编码? 又如何反过来, 从整数编码转换回字符?

4. How might you go about changing a string in Python?

在 Python 里, 你会如何着手"改变"一个字符串?

5. Given a string S with the value 'c,od,e', name two ways to extract the two characters in the middle.

给定值为 'c,od,e' 的字符串 S, 说出两种提取中间那两个字符的办法。

6. How many characters are there in the string "a\nb\x1f\000d"?

字符串 "a\nb\x1f\000d" 里有几个字符?

7. Write an expression, method call, and f-string to format 'Python' and 3.12 at a string's beginning and end.

分别用表达式、方法调用和 f-string, 写出把'Python' 和 3.12 格式化到字符串开头和结尾的代码。

测试你的知识：答案 (Test Your Knowledge: Answers)

第 1 题

英文原文: No, because methods are always type specific; that is, they only work on a single object type. Expressions like `X + Y` and built-in functions like `len(X)` are generic, though, and may work on a variety of types. In this case, for instance, the `in` membership expression has a similar effect as the string `find`, but it can be used to search both strings and lists. Python makes some attempt to name similar methods consistently (many objects have a `copy` method, for example, and mutable objects may share method names like `pop`), but methods are still more type specific than other operation sets.

中文翻译: 不能。因为方法总是类型特定的; 也就是说, 它们只作用于单一对象类型。而 `X + Y` 这样的表达式和 `len(X)` 这样的内置函数是通用的, 可能作用于多种类型。在本例中, `in` 成员关系表达式与字符串的 `find` 效果类似, 但它既能搜索字符串也能搜索列表。Python 曾试图让相似的方法命名一致 (例如许多对象都有 `copy` 方法, 可变

对象可能共享 pop 这类方法名)，但方法仍然比其他操作集合更类型特定。

第 2 题

英文原文：Yes. Unlike methods, expressions are generic and apply to many types. In this case, the slice expression is really a sequence operation—it works on any type of sequence object, including strings, lists, and tuples. The only difference is that when you slice a list, you get back a new list.

中文翻译：能。与方法不同，表达式是通用的，可以应用于许多类型。本例中，切片表达式本质上是一个序列操作——它作用于任意类型的序列对象，包括字符串、列表和元组。唯一的区别是：当你切片一个列表时，得到的是新列表。

第 3 题

英文原文：The built-in ord(S) function converts from a one-character string to an integer character code; chr(I) converts from the integer code back to a string. Keep in mind, though, that these integers are only ASCII codes for text whose characters are drawn only from the ASCII character set. In the Unicode model, text strings are really sequences of Unicode code point identifying integers, which may fall outside the 7-bit range of numbers reserved by ASCII (we pr

viewed Unicode in Chapter 4 and will revisit it in Chapter 37).

中文翻译：内置函数 `ord(S)` 把单字符字符串转换成整数形式的字符编码；`chr(l)` 把整数编码转换回字符串。不过请记住：只有当文本字符全部来自 ASCII 字符集时，这些整数才只是 ASCII 码。在 Unicode 模型中，文本字符串实际上是 Unicode 码点（code point）——即标识整数——的序列，可能落在 ASCII 预留的 7 位数字范围之外（我们曾在第 4 章预告过 Unicode，并将在第 37 章重访）。

第 4 题

英文原文：Strings cannot be changed; they are immutable. However, you can achieve a similar effect by creating a new string—by concatenating, slicing, using a method call like `replace`, or running formatting operations—and then assigning the result back to the original variable name.

中文翻译：字符串不能被改变；它们是不可变的。不过，你可以通过创建新字符串达到类似效果——用拼接、切片、调用像 `replace` 这样的方法，或运行格式化操作——然后把结果赋回原来的变量名。

第 5 题

英文原文：You can slice the string using `S[2:4]` or `split` on the comma and index the string using `S.split(',')`

)[1]. Try these interactively to see for yourself.

中文翻译：你可以用 `S[2:4]` 切片字符串，或用逗号分割后用 `S.split(',')[1]` 索引这个字符串。请交互式地试试这两种方法，亲自验证。

第 6 题

英文原文：Six. The string `"a\nb\x1f\000d"` contains the characters a, newline (`\n`), b, literal value 31 (as hex escape `\x1f`, which is a code point that stands for the nonprintable control character US), literal value 0 (an octal escape `\000`), and d. Pass the string to the built-in `len` function to verify this and print each of its characters' `ord` results to see the actual code point (identifying number) values. See Table 7-2 for more details on escapes.

中文翻译：六个。字符串 `"a\nb\x1f\000d"` 包含字符 a、换行符 (`\n`)、b、字面值 31（十六进制转义 `\x1f`，它是一个代表不可打印控制字符 US 的码点）、字面值 0（八进制转义 `\000`）和 d。把这个字符串传给内置 `len` 函数验证一下，并打印它每个字符的 `ord` 结果，就能看到实际的码点（标识数字）值。转义码的更多细节参见表 7-2。

第 7 题

英文原文：There's no right answer for what goes in the middle of the result string, but as examples: `'%s is %s' % ('Python', 3.12)` works for the expression, an

`d'{} is {}'.format('Python', 3.12)` suffices for the method call. There's almost no reason to use an f-string if all parts are known and formatted per defaults (`f'Python' is {3.12}'` seems silly), but f-strings are more useful if values are first assigned to variables: `x='Python'; y=3.12; f'{x} is {y}'` or similar garners full points.

中文翻译：结果字符串中间放什么没有标准答案，但举个例子：表达式用 `'%s is %s' % ('Python', 3.12)` 即可，方法调用用 `'{} is {}'.format('Python', 3.12)` 就足够了。如果所有部分都是已知的、且按默认格式排版，几乎没理由用 f-string (`f'Python' is {3.12}'` 看起来有点傻)；但如果先把值赋给变量，f-string 就更有用：`x='Python'; y=3.12; f'{x} is {y}'` 或类似写法即可得满分。

本章总结

技术拓展 (Technical Expansion)

- 三种格式化工具的选型矩阵：f-string（新代码首选，3.6+，表达式内联、= 调试标签、支持嵌套 {}）、.format 方法（动态字段名、可当文本模板）、% 表达式（老代码存量、字典键 %(name)s）；string.Template 为第四种，专攻“把模板当纯文本数据”的安全场景。
- 3.12 的 f-string 关键变更（PEP 701）：{} 内可用与外部相同的引号、可用反斜杠转义、可写注释与多行表达式

；旧版本需用"外侧换异号引号"规避，兼容旧环境时优先 `f"..."` 包裹单引号表达式。

·性能与安全：`f-string` 编译期展开为字节码，比 `%.format` 的运行时解析更快；`eval` 执行文件加载的 `f-string` 模板有代码注入风险，外部模板应改用 `string.Template` 或 `.format(values)`。

·进阶预告：`bytearray`（可变字节串）与 `bytes` 的 Unicode 模型在第 37 章展开；`str.formatmap`、`str.maketrans/translate` 等高频工具可查阅官方文档自学；切片步长、负索引在列表、元组上完全一致（第 8 章）。

·术语对照：`interpolation`（插值）/ `format specifier`（格式说明）/ `conversion flag`（转换标志）/ `alignment`（对齐）/ `fill`（填充）/ `precision`（精度）/ `template`（模板）/ `docstring`（文档字符串）/ `immutable`（不可变）/ `mutable`（可变）/ `sequence`（序列）/ `mapping`（映射）。

学习建议 (Learning Advice)

·本章定位：五星重要、五星难度。字符串是 Python 几乎所有程序的基石，本章的知识点（切片、方法、三种格式化）将在后续每章反复出现，务必一次学透。

·练习方法：(1) 亲手把本章每个 REPL 示例敲一遍，尤其是三种格式化对同一输入的各等价写法；(2) 用切片做 S 形漫步：`S[::2]`、`S[::-1]`、`S[1:9:2]`，在纸上画下标确认边界；(3) 把 Quiz 七题当作业独立作答，再对照答案。

·易错提醒：(1) % 格式化里 %s 与 %(name)s 混用会报错；(2) f-string 不能当 docstring、不能从文件加载模板；(3) 在 3.12 前的环境里写同引号嵌套会语法错误；(4) 字符串不可变，任何"修改"都要生成新对象再重新赋值。

·进阶路径：学完本章可顺手掌握列表、元组的序列操作（第 8 章）；关注 str 方法清单（表 7-3）中尚未用到的 zfill、partition、maketrans 等，日后文本处理时检索即可。

·心态建议：字符串格式化是新手最容易"抄了不会用"的知识点——三种写法并存是历史原因，不是设计玄学。先固定用 f-string，再读懂老代码里的 %，最后遇到模板场景自然想到 .format/Template。工具之间能互译，就说明你真的懂了。